

Runbooks Betrieb und Release

Betriebsanleitungen, Produktionskonfiguration, Release und Migration

Die operative Ebene: Runbooks für Betrieb und Incident-Handling, Produktionsnachweise und Deployment-Verifikation, Release-Prozess und Migrationsleitfäden. Hier steht, wie das System tatsächlich betrieben, ausgerollt und im Fehlerfall stabilisiert wird.

Repository: SvenKulesa/Finance

Stand: claude/pdf-dokumente-notebook-llm-l6kcxq @ 4eadf9d

Erzeugt am: 16.08.2026

Enthaltene Dokumente: 38

Umfang: ca. 27.242 Wörter

Dieses PDF ist eine Quelle für NotebookLM. Jedes eingebettete Dokument beginnt auf einer neuen Seite und nennt seinen Repository-Pfad, damit erzeugte Podcasts, Video-Skripte und Zusammenfassungen ihre Aussagen exakt auf die Originaldatei zurückführen können.

Inhalt dieses Quellpakets

1. Migration Report — EXC-0003	5
docs/migration/MIGRATION_EXC-0003_sql_to_supabase.md	
2. Migration Report — EXC-0005 und EXC-0007	14
docs/migration/MIGRATION_EXC-0005_EXC-0007_root_docs.md	
3. POST-PR Production Snapshot — Subscription Entitlements Live Refresh	20
docs/production/evidence/PR_POSTCHECK_2026-08-03_SUBSCRIPTION_ENTITLEMENTS_LIVE_REFRESH.md	
4. PRE-PR Production Snapshot — Subscription Entitlements Live Refresh	21
docs/production/evidence/PR_PRECHECK_2026-08-03_SUBSCRIPTION_ENTITLEMENTS_LIVE_REFRESH.md	
5. CAPITAL-AI — Production SLO Persistence Acceptance Record	23
docs/production/evidence/PRODUCTION_SLO_PERSISTENCE_ACCEPTANCE_2026-08-02.md	
6. Render Auto-Deploy (checksPass) Verification — 2026-08-09	27
docs/production/evidence/RENDER_AUTODEPLOY_WEBHOOK_VERIFICATION_2026-08-09.md	
7. changelog dev	28
docs/release/changelog-dev.md	
8. Changelog - CAPITAL-AI Plattform	32
docs/release/changelog-prod.md	
9. Alpaca-Shadow: Produktionsverifikation und Laufzeitgrenzen	35
docs/runbooks/ALPACA_SHADOW_PRODUCTION_VERIFICATION.md	
10. Runbook — CI-Konsolidierung auf build-and-test	37
docs/runbooks/CI_BUILD_AND_TEST_CONSOLIDATION.md	
11. CAPITAL-AI Runbook: Deployment-Rollback und Backup	39
docs/runbooks/DEPLOYMENT_ROLLBACK_UND_BACKUP.md	
12. DEVELOPMENT Chain Phase Execution Runbook	46
docs/runbooks/DEVELOPMENT_CHAIN_PHASE_EXECUTION.md	
13. FRED Macro/Rate Evidence — Production Runbook	51
docs/runbooks/FRED_MACRO_EVIDENCE_RUNBOOK.md	
14. CAPITAL-AI Runbook: GA4 Analytics MCP Server	54
docs/runbooks/GA4_MCP_SERVER_SETUP.md	
15. GitGuardian-/Snyk-App-Integration — Owner-Runbook	58
docs/runbooks/GITGUARDIAN_SNYK_APP_INTEGRATION.md	
16. CAPITAL-AI Runbook: Google Analytics, AdSense und CookieHub	61
docs/runbooks/GOOGLE_ANALYTICS_SETUP.md	
17. M10 — Passkey-only Human/Owner PR Authorization Runbook	67
docs/runbooks/M10_PASSKEY_OWNER_PR_AUTHORIZATION.md	

18. M5 Supabase Agent Audit Mutation Runbook	71
<code>docs/runbooks/M5_SUPABASE_AGENT_AUDIT_MUTATION.md</code>	
19. M5A Runbook — Supabase Native TOTP MFA / AAL2 Hardening	73
<code>docs/runbooks/M5A_SUPABASE_TOTP_AAL2_HARDENING.md</code>	
20. M6 — Supply Chain Provenance & Attestation Runbook	80
<code>docs/runbooks/M6_SUPPLY_CHAIN_PROVENANCE.md</code>	
21. M7 — Render Deploy-Hook-Rotation: Mutation Work Order	84
<code>docs/runbooks/M7_DEPLOY_HOOK_ROTATION_HANDOFF.md</code>	
22. M7 — Deployment Identity & Production Platform Mutation Runbook	89
<code>docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md</code>	
23. M7 — Deployment Rollback Verification: Mutation Work Order	96
<code>docs/runbooks/M7_ROLLBACK_VERIFICATION_HANDOFF.md</code>	
24. M8 — Provider-neutral Agent Cutover Runbook	101
<code>docs/runbooks/M8_AGENT_CUTOVER.md</code>	
25. M9 — Agent Assurance, Incident Response & Break-Glass Runbook	106
<code>docs/runbooks/M9_ASSURANCE_INCIDENT_BREAK_GLASS.md</code>	
26. Production Environment Setup — Documentation-Only Target	111
<code>docs/runbooks/PRODUCTION_ENVIRONMENT_SETUP.md</code>	
27. Render Auto-Deploy A/B Test — 2026-08-09	112
<code>docs/runbooks/RENDER_AUTODEPLOY_AB_TEST_2026-08-09.md</code>	
28. Render Auto-Deploy Credential Test — 2026-08-09	115
<code>docs/runbooks/RENDER_AUTODEPLOY_CREDENTIAL_TEST_2026-08-09.md</code>	
29. Render CI Deployment Gate	116
<code>docs/runbooks/RENDER_CI_DEPLOY_GATE.md</code>	
30. Render Docker Hardening Runbook — 2026-08-09	118
<code>docs/runbooks/RENDER_DOCKER_HARDENING_2026-08-09.md</code>	
31. CAPITAL-AI Runbook — Render Production Evidence & Configuration Handoff	120
<code>docs/runbooks/RENDER_PRODUCTION_EVIDENCE_HANDOFF.md</code>	
32. RUNBOOK — Screening SLO Evidence Persistence	128
<code>docs/runbooks/SCREENING_SLO_EVIDENCE_PERSISTENCE.md</code>	
33. Screening SLO Sink — Production Handoff	130
<code>docs/runbooks/SCREENING_SLO_SINK_PRODUCTION_HANDOFF.md</code>	
34. SEO WP-S1 Migration-Ledger-Abgleich	132
<code>docs/runbooks/SEO_WP_S1_LEDGER_RECONCILIATION.md</code>	
35. CAPITAL-AI Runbook: Social Media OAuth-2.0-Setup (YouTube, TikTok, Instagram, Facebook, X)	136
<code>docs/runbooks/SOCIAL_MEDIA_OAUTH_SETUP.md</code>	
36. Systemadmin Audited Execution Runbook — SA3	142

[docs/runbooks/SYSTEMADMIN_AUDITED_EXECUTION.md](#)

37. Systemadmin Chat Execution Profile Runbook	146
---	------------

[docs/runbooks/SYSTEMADMIN_CHAT_EXECUTION_PROFILE.md](#)

38. Systemadmin Execution Host Runbook — SA3B	151
--	------------

[docs/runbooks/SYSTEMADMIN_EXECUTION_HOST.md](#)

Migration Report — EXC-0003

Quelle: docs/migration/MIGRATION_EXC-0003_sql_to_supabase.md

Migration Report — EXC-0003

Enterprise Migration

Migration ID

MIG-EXC-0003

Titel

Überführung der Quota-Migration aus `sql/` nach `supabase/migrations/`

Version

1.0.0

Status

Completed — Strukturmigration durchgeführt und verifiziert

Datum

2026-07-30

Kategorie

Structural Migration gemäß ESS-0001-CONTRACTS Chapter 14, Migration Categories

ADR-Referenz

ADR-0011 — Bestandsschutz und Zielstruktur für die Root-Abweichungen

Verantwortlich

Security Center (Eigentümer `supabase/` gemäß Chapter 16, Directory Ownership)

Enterprise Purpose

Dieser Report dokumentiert die Auflösung der registrierten Ausnahme EXC-0003 gemäß ESS-0001-CONTRACTS Chapter 14.

Er weist nach, dass die Überführung reproduzierbar, inhaltsneutral und rückführbar erfolgt ist.

Abgrenzung

Diese Migration ist eine **Structural Migration**, keine **Data Migration**.

Es wurde ausschließlich ein Artefakt im Repository verschoben.

Es wurde **kein** SQL gegen eine Datenbank ausgeführt.

Es wurden **keine** Daten verändert.

Der Data Migration Contract aus Chapter 14 greift erst bei der Ausführung des enthaltenen SQL gegen eine Zielumgebung und ist nicht Gegenstand dieser Migration.

Ausgangszustand

```
sql/  
  001_user_quota.sql  
  
supabase/migrations/  
  20260711000000_iam.sql
```

Verletzte Contracts

Contract	Verletzung
Chapter 2 — Repository Root	<code>sql/</code> ist kein zugelassenes Root-Verzeichnis
Chapter 2 — File Placement Rules	derselbe Artefakttyp an zwei Orten
Chapter 3 — Directory Responsibility	eine Verantwortung besitzt genau ein Verzeichnis

Zielzustand

```
supabase/migrations/  
  20260711000000_iam.sql  
  20260730000000_user_quota.sql
```

`sql/` existiert nicht mehr.

Durchführung

Schritt	Maßnahme	Ergebnis
1	Prüfsumme des Ausgangsartefakts gebildet	<code>10c791c4...c80196</code>
2	Referenzanalyse über das gesamte Repository	keine Code-Referenz gefunden
3		kein Runner vorhanden

Schritt	Maßnahme	Ergebnis
	Prüfung auf automatische Migrationsausführung	
4	Verschiebung als Git-Rename	Ähnlichkeit 100 %
5	Prüfsumme des Zielartefakts gebildet	10c791c4...c80196
6	Leeres Verzeichnis <code>sql/</code> entfernt	Root-Abweichung beseitigt

Prüfsummenvergleich

vorher	10c791c4e413c99b9825efdf6cf1482098cf6824e1b623a83fac73a722c80196
nachher	10c791c4e413c99b9825efdf6cf1482098cf6824e1b623a83fac73a722c80196

Die Prüfsummen sind identisch. Der Inhalt wurde nicht verändert.

Bewusste Entscheidung zur Inhaltsneutralität

Die Datei wurde **byteidentisch** übernommen.

Es wurde ausdrücklich **kein** Provenienz-Header, kein ADR-Verweis und kein Migrationshinweis in die SQL-Datei eingefügt, obwohl Chapter 7 solche Referenzen für Enterprise-Artefakte grundsätzlich vorsieht.

Begründung

Bei einer Strukturmigration ist die byteweise Gleichheit der stärkste verfügbare Nachweis der Inhaltsneutralität. Jede Ergänzung — auch eine rein redaktionelle — hätte die Prüfsumme verändert und damit genau den Nachweis zerstört, den Chapter 14 für eine reproduzierbare Migration verlangt.

Die Herkunftsdocumentation erfolgt deshalb ausschließlich in diesem Report und in der Exception Registry.

Eine spätere inhaltliche Ergänzung der Datei ist als eigene Änderung mit eigener Versionsbewertung möglich.

Namensgebung

Die Zieldatei folgt der im Verzeichnis bereits verwendeten Konvention

```
<YYYYMMDDHHMMSS>_<name>.sql
```

Gewählt wurde `20260730000000_user_quota.sql`.

Begründung des Zeitstempels

Die Git-Historie liefert kein belastbares Ursprungsdatum: sowohl `001_user_quota.sql` als auch `20260711000000_iam.sql` wurden im selben Sammel-Commit `75c42a4` hinzugefügt. Ein früheres Datum wäre eine nicht belegbare Behauptung.

Der Zeitstempel bezeichnet daher den Zeitpunkt, zu dem das Artefakt in den verwalteten Migrationsbestand aufgenommen wurde. Diese Aussage ist nachprüfbar.

Auswirkung auf die Ausführungsreihenfolge

Keine. Die Quota-Migration erzeugt `public.user_quota` mit eigener Funktion, eigenem Trigger und eigener Policy. Sie besitzt keine Abhängigkeit zur IAM-Migration — weder Fremdschlüssel noch gemeinsame Objekte. Die lexikografische Einordnung nach der IAM-Migration ist damit funktional folgenlos.

Validierung

Prüfung	Ergebnis
Inhalt unverändert	✓ Prüfsummen identisch
Git-Rename erkannt	✓ Ähnlichkeit 100 %, Historie erhalten
Keine Code-Referenz auf <code>sql/</code>	✓ keine Treffer in <code>*.ts</code> , <code>*.tsx</code> , <code>*.json</code> , <code>*.yaml</code>
Kein automatischer Migrations-Runner	✓ kein Verzeichnis-Scan auf <code>sql/</code> oder <code>supabase/migrations/</code>
Kein <code>supabase/config.toml</code> vorhanden	✓ keine CLI-gesteuerte automatische Anwendung
Namenskonvention eingehalten	✓ Zeitstempel-Format wie bestehende Migration
Root-Abweichung beseitigt	✓ <code>sql/</code> existiert nicht mehr
Keine Datenbankänderung ausgeführt	✓ ausschließlich Repository-Operation

Idempotenz des enthaltenen SQL

Für den Fall einer erneuten Ausführung gegen eine Umgebung, in der die Tabelle bereits existiert, ist das enthaltene SQL vollständig wiederholbar formuliert:

```
create table if not exists public.user_quota (
create index if not exists idx_user_quota_email
create or replace function public.touch_user_quota_updated_at()
drop trigger if exists trg_touch_user_quota
drop policy if exists "service_role_full_access"
```

Eine erneute Anwendung verändert weder Struktur noch Daten.

Dies ist relevant, weil der Dateikopf angibt, das Skript sei bereits vor dem 0.5.0-Build gegen Staging und Produktion ausgeführt worden. Die Aufnahme in den Migrationsbestand kann daher nicht zu einem Fehler führen.

Rollback

Strategie

Vollständige Umkehrung durch Rücknahme des Git-Renames.

```
mkdir -p sql
git mv supabase/migrations/20260730000000_user_quota.sql sql/001_user_quota.sql
```

Voraussetzungen keine

Risiken keine — es wurde kein Zustand außerhalb des Repositorys verändert

Rollback-Tests Prüfsummenvergleich nach Rücknahme

Rollback-Version identisch mit der Ausgangsversion, da inhaltsneutral

Ein Datenbank-Rollback ist nicht erforderlich und nicht möglich, da keine Datenbankoperation stattgefunden hat.

Befund während der Migration

FND-EXC-0003-01 — Die Migration ist im aktuellen Codebestand verwaist

Stufe Medium

Kategorie Technische Schuld gemäß Chapter 12, Technical Debt Contract

Nachweis

Der Dateikopf lautet:

```
"Backs src/lib/freeTierLimits.ts."
```

Die Analyse ergibt:

Prüfung	Ergebnis
Existiert <code>src/lib/freeTierLimits.ts</code> ?	nein
Existiert die Datei in der Git-Historie?	nein — kein einziger Commit
Referenzen auf <code>user_quota</code> in <code>*.ts</code> / <code>*.tsx</code>	keine
Referenzen auf die Tabelle in <code>server/</code>	keine

Die Tabelle `public.user_quota` wird vom aktuellen Codebestand nicht verwendet. Das zugehörige Modul existiert im Repository nicht und hat dort nie existiert.

Bewertung

Es sind zwei Erklärungen möglich, die aus dem Repository heraus nicht unterscheidbar sind:

1. Die Quota-Durchsetzung wurde entfernt oder nie in dieses Repository übernommen; die Migration ist ein Rückstand.
2. Die Tabelle wird von einer Komponente außerhalb dieses Repositories verwendet.

Bewusst nicht getroffene Entscheidung

Die Datei wurde **nicht** gelöscht.

Eine Löschung wäre eine Entscheidung über einen möglicherweise produktiv genutzten Datenbankzustand. Chapter 14 behält die Entfernung von Bestand ausdrücklich dem Plattform Director vor und verlangt zuvor den Nachweis, dass keine aktiven Abhängigkeiten bestehen. Dieser Nachweis ist allein aus dem Repository nicht führbar.

Empfehlung

Klärung, ob `public.user_quota` in der Zielumgebung Daten enthält und von einer externen Komponente gelesen wird. Je nach Ergebnis:

- verwendet → Quota-Durchsetzung im Code wiederherstellen oder Verwender dokumentieren
- nicht verwendet → Entfernung als eigene Data Migration mit Sicherung und Rollback

Bis zur Klärung bleibt die Migration unverändert im Bestand.

Klärung (2026-07-31)

Die unter Empfehlung geforderte Prüfung wurde gegen die verknüpfte Supabase-Produktivumgebung durchgeführt (Projekt `ryzywoktpmyhwzxmsty`, dieselbe Umgebung, gegen die auch der `Supabase Preview`-CI-Check läuft).

Ergebnis

Prüfung	Befund
Existiert <code>public.user_quota</code> in der Datenbank?	Nein — nicht unter den 13 vorhandenen Tabellen des Schemas <code>public</code>
Existiert eine Edge Function, die die Tabelle referenziert?	Nein — <code>supabase/functions/</code> enthält ausschließlich <code>.gitkeep</code>
Existiert eine RPC-Funktion, die die Tabelle referenziert?	Nein — <code>supabase/rpc/</code> enthält ausschließlich <code>.gitkeep</code>
Existiert eine strukturell verwandte Tabelle?	Ja — <code>public.usage_log</code> (46 Zeilen, Spalten <code>user_id</code> , <code>endpoint</code> , <code>interval_used</code>), konzeptionell ähnlich, jedoch mit abweichendem Schema (keine <code>quota_kind</code> -Klassifizierung, kein Zeitfenster-Zähler) und ohne Code-Referenz im Repository

Schlussfolgerung

Die zweite in diesem Report offengehaltene Erklärung — „Die Tabelle wird von einer Komponente außerhalb dieses Repositorys verwendet“ — ist widerlegt: Eine Tabelle, die nicht existiert, kann nicht verwendet werden. Es verbleibt ausschließlich die erste Erklärung: Die Quota-Durchsetzung wurde nie in Betrieb genommen. Die Migration wurde nie gegen die Produktivumgebung ausgeführt.

Risikobewertung der Entfernung

Da die Tabelle nicht existiert, kann ihre Entfernung aus dem Migrationsbestand keinen Datenverlust verursachen — es gibt keine Zeilen, die verloren gehen könnten. Der unter Bewusst nicht getroffene Entscheidung beschriebene Vorbehalt (Chapter 14: kein Bestand ohne Nachweis fehlender aktiver Abhängigkeiten) ist damit erfüllt.

Weiterhin nicht getroffene Entscheidung

Trotz des vorliegenden Nachweises wurde die Datei im Rahmen dieser Klärung **nicht** gelöscht.

Chapter 14 weist die Entfernung von Bestand ausdrücklich dem Platform Director zu — nicht der Documentary Engine und nicht dem ausführenden KI-System. Der vorliegende Nachweis liefert die Entscheidungsgrundlage; die Entscheidung selbst ist hiermit zur Freigabe vorgelegt:

- **Entfernen** — `supabase/migrations/202607300000000_user_quota.sql` löschen. Risikofrei, da keine Daten betroffen sind. Der freie Tarif verliert damit endgültig die im Dateikopf beschriebene serverseitige Quota-Durchsetzung, falls sie künftig gewünscht wird.
- **Belassen und ausführen** — die Migration gegen die Produktivumgebung anwenden und die im Code fehlende Durchsetzung (`src/lib/freeTierLimits.ts` oder Äquivalent) nachträglich implementieren, falls die Quota-Begrenzung fachlich weiterhin gewollt ist.
- **Belassen und dokumentieren** — Migration unverändert im Bestand halten, als bewusst aufgeschobene, aktuell inaktive Funktionalität kennzeichnen.

Bis zur Entscheidung des Platform Director gilt die dritte Option als Zwischenzustand.

Auswirkung auf die Exception Registry

Feld	vorher	nachher
Status EXC-0003	Time Limited	Revoked
Frist	2026-09-30	entfällt
Grund	—	Abweichung beseitigt

Der Status `Revoked` ist gemäß Chapter 16 der zutreffende Endzustand einer nicht mehr benötigten Ausnahme. Der Eintrag bleibt dauerhaft in der Registry erhalten und wird nicht gelöscht.

Auswirkung auf offene Befunde

Befund	vorher	nachher
GAP-006	zwei nicht zugelassene Root-Verzeichnisse (<code>server/</code> , <code>sql/</code>)	ein registriertes Root-Verzeichnis (<code>server/</code> , EXC-0001)

Von den ursprünglich acht Root-Abweichungen ist die erste vollständig aufgelöst.

Version

Diese Migration verändert keine Schnittstelle, kein Verhalten und keinen Datenbestand.

Versionsauswirkung gemäß Chapter 9: **Patch**.

Die tatsächliche Versionsvergabe obliegt dem Version Manager und ist bis zur Auflösung von GAP-019 (vier widersprüchliche Versionsstände) ausgesetzt.

Related Documents

ADR-0011 — Bestandsschutz und Zielstruktur für die Root-Abweichungen

`.ai/registry/exception-registry.json` — EXC-0003

`docs/architecture/ARCHITECTURE_GAP_REPORT.md` — GAP-006

ESS-0001-CONTRACTS Chapter 14 — Migration & Lifecycle Contracts

ESS-0001-CONTRACTS Chapter 16 — Repository Governance

Version History

Version	Status	Beschreibung
1.0.0	Completed	Strukturmigration <code>sql/</code> → <code>supabase/migrations/</code> durchgeführt und verifiziert

End of Document

MIG-EXC-0003

CAPITAL-AI Migration Report

Version 1.0.0

Migration Report — EXC-0005 und EXC-0007

Quelle: docs/migration/MIGRATION_EXC-0005_EXC-0007_root_docs.md

Migration Report — EXC-0005 und EXC-0007

Enterprise Migration

Migration ID

MIG-EXC-0005-0007

Titel

Verlagerung von `ORCHESTRATORS_AND_SCORING_ENGINES.md` nach `docs/architecture/` und `favi-con.svg` nach `public/`

Version

1.0.0

Status

Completed — beide Strukturmigrationen durchgeführt und verifiziert

Datum

2026-07-31

Kategorie

Structural Migration gemäß ESS-0001-CONTRACTS Chapter 14, Migration Categories

ADR-Referenz

ADR-0011 — Bestandsschutz und Zielstruktur für die Root-Abweichungen

Verantwortlich

Documentary Engine (EXC-0005) · Frontend (EXC-0007)

Enterprise Purpose

Dieser Report dokumentiert die Auflösung der registrierten Ausnahmen EXC-0005 und EXC-0007 gemäß ESS-0001-CONTRACTS Chapter 14. Beide Positionen waren in ADR-0011 als voraussetzungsfrei und sofort umsetzbar eingestuft.

Abgrenzung

Beide Migrationen sind **Structural Migrations**. Es wurde ausschließlich je ein Artefakt im Repository verschoben. Kein Build wurde ausgeführt, keine Laufzeitumgebung verändert.

Teil 1 — EXC-0005: ORCHESTRATORS_AND_SCORING_ENGINES.md

Ausgangszustand

ORCHESTRATORS_AND_SCORING_ENGINES.md (Repository-Root)

Verletzte Contracts

Contract	Verletzung
Chapter 2 — Documentation Placement	Dokumentation außerhalb docs/ ohne Werkzeugbindung

Zielzustand

docs/architecture/ORCHESTRATORS_AND_SCORING_ENGINES.md

Durchführung

Schritt	Maßnahme	Ergebnis
1	Prüfsumme des Ausgangsartefakts gebildet	f24340be...c45591a
2	Referenzanalyse über das gesamte Repository	3 Treffer, alle in dieser Standard-Lieferung selbst (ADR-0011 , exception-registry.json , ARCHITECTURE_GAP_REPORT.md) — keine Code-Referenz
3	Verschiebung als Git-Rename	Ähnlichkeit 100 %
4	Prüfsumme des Zielartefakts gebildet	f24340be...c45591a

Prüfsummenvergleich

vorher	f24340be80bb10a5ebe93843dcbc7b18c76d43ddba61cbaf6487d7219c45591a
nachher	f24340be80bb10a5ebe93843dcbc7b18c76d43ddba61cbaf6487d7219c45591a

Identisch. Der Inhalt wurde nicht verändert.

Bewusst nicht miterledigt

ADR-0011 nennt für EXC-0005 zusätzlich die Ergänzung von ESS- und ADR-Referenzen gemäß Chapter 7. Diese Ergänzung wurde **nicht** vorgenommen.

Begründung — dieselbe wie bei EXC-0003: eine inhaltliche Änderung hätte die Prüfsumme verändert und den Nachweis der reinen Strukturverlagerung zerstört. Chapter 7 sieht die Ergänzung solcher Referenzen als generatorgestützten Vorgang der Documentation Engine vor, die noch nicht existiert. Der Punkt bleibt unter GAP-029 offen und ist keine Rückstellung dieser Migration, sondern ihrer bewussten Abgrenzung.

Teil 2 — EXC-0007: `favicon.svg`

Ausgangszustand

```
favicon.svg    (Repository-Root)
public/        (leer, ausschließlich .gitkeep)
```

Verletzte Contracts

Contract	Verletzung
Chapter 2 — Repository Root, <code>public/</code> als Ort statischer Web-Ressourcen	Statische Ressource außerhalb des vorgesehenen Verzeichnisses

Zielzustand

```
public/favicon.svg
```

Durchführung

Schritt	Maßnahme	Ergebnis
1	Prüfsumme des Ausgangsartefakts gebildet	<code>eb8b5116...7cc6a9451</code>
2	Referenz in <code>index.html</code> geprüft	<code><link rel="icon" href="/favicon.svg"></code> — bereits root-absolut
3	Vite-Konfiguration geprüft (<code>vite.config.ts</code>)	kein <code>publicDir</code> -Override → Default <code>public/</code>
4	Build- und Container-Pfad geprüft (<code>package.json</code> , <code>Dockerfile</code>)	<code>vite build</code> kopiert ausschließlich <code>publicDir</code> nach <code>dist/</code> ; Docker kopiert ausschließlich <code>dist/</code> in das Runtime-Image
5	Verschiebung als Git-Rename	Ähnlichkeit 100 %
6	Prüfsumme des Zielartefakts gebildet	<code>eb8b5116...7cc6a9451</code>
7	<code>index.html</code> erneut geprüft	keine Änderung erforderlich, da Referenz bereits root-absolut ist

Prüfsummenvergleich

vorher	eb8b5116e54ebd520ec0558c09e5180a348bd93a1f02da4b0a507747cc6a9451
nachher	eb8b5116e54ebd520ec0558c09e5180a348bd93a1f02da4b0a507747cc6a9451

Identisch. `index.html` wurde nicht verändert.

Befund während der Migration

FND-EXC-0007-01 — Das Favicon gelangte vor dieser Migration nicht in den Produktions-Build

Stufe Low

Kategorie Latenter Fehler, aufgedeckt durch Strukturmigration

Nachweis

- `vite.config.ts` setzt keinen abweichenden `publicDir`; Vite kopiert beim Build ausschließlich den Inhalt von `public/` unverändert nach `dist/`.
- `favicon.svg` lag vor dieser Migration im Repository-Root, **nicht** in `public/`.
- Der `Dockerfile`-Runtime-Stage kopiert ausschließlich `dist/` in das Produktions-Image (`COPY --from=builder /app/dist ./dist`).
- `render.yaml` und `package.json` (`build`, `start`) führen keinen zusätzlichen Kopiervorgang für Root-Dateien aus.

Bewertung

`index.html` referenzierte das Favicon bereits vor dieser Migration korrekt als `/favicon.svg`. Die Datei war jedoch in keinem Schritt der Build-Kette Bestandteil von `dist/`. In der Produktionsumgebung war die Anfrage nach `/favicon.svg` damit mit hoher Wahrscheinlichkeit ein 404, während sie im lokalen Entwicklungsserver möglicherweise funktionierte, da Vite dort in bestimmten Konfigurationen zusätzlich Dateien aus dem Projekt-Root direkt bedient.

Auswirkung dieser Migration

Mit der Verlagerung nach `public/favicon.svg` wird die Datei ab dem nächsten Build regulär nach `dist/favicon.svg` kopiert und ist über `/favicon.svg` erreichbar. Diese Migration behebt den Fehler als Nebeneffekt der Strukturkonformität.

Empfehlung

Nach dem nächsten Produktions-Deployment verifizieren, dass `/favicon.svg` eine 200-Antwort liefert. Kein weiterer Code ist zu ändern.

Validierung

Prüfung	EXC-0005	EXC-0007
Inhalt unverändert	✓ Prüfsummen identisch	✓ Prüfsummen identisch
Git-Rename erkannt	✓ Ähnlichkeit 100 %	✓ Ähnlichkeit 100 %
Referenzierende Datei angepasst, falls nötig	entfällt (keine Referenz)	✓ keine Änderung nötig (bereits root-absolut)
Root-Abweichung beseitigt	✓	✓
Kein Build ausgeführt, kein Betriebszustand verändert	✓	✓

Rollback

Strategie Vollständige Umkehrung durch Rücknahme des jeweiligen Git-Renames.

```
git mv docs/architecture/ORCHESTRATORS_AND_SCORING_ENGINES.md ORCHESTRATORS_AND_SCORING_ENGINES.md
git mv public/favicon.svg favicon.svg
```

Voraussetzungen keine

Risiken Rücknahme von EXC-0007 reproduziert den unter FND-EXC-0007-01 dokumentierten Zustand (Favicon fehlt im Produktions-Build).

Rollback-Tests Prüfsummenvergleich nach Rücknahme

Rollback-Version identisch mit der Ausgangsversion, da beide Migrationen inhaltsneutral sind

Auswirkung auf offene Befunde

Befund	vorher	nachher
GAP-006	fünf verbleibende nicht aufgelöste Root-Abweichungen	drei verbleibende (<code>server/</code> , <code>server.ts</code> ; die übrigen vier sind <code>Permanent</code> und keine Befunde)
GAP-020	teilweise (Dokumentation ohne ESS-Bezug)	unverändert offen, betrifft Inhalt nicht Ort
GAP-029	Dokument ohne ESS-/ADR-Referenz	unverändert offen, siehe Abgrenzung oben

Version

Beide Migrationen verändern keine Schnittstelle, kein Verhalten und keinen Datenbestand.

Versionsauswirkung gemäß Chapter 9: **Patch**.

Die tatsächliche Versionsvergabe obliegt dem Version Manager und ist bis zur Auflösung von GAP-019 (vier widersprüchliche Versionsstände) ausgesetzt.

Related Documents

ADR-0011 — Bestandsschutz und Zielstruktur für die Root-Abweichungen

`.ai/registry/exception-registry.json` — EXC-0005, EXC-0007

`docs/architecture/ARCHITECTURE_GAP_REPORT.md` — GAP-006, GAP-020, GAP-029

ESS-0001-CONTRACTS Chapter 14 — Migration & Lifecycle Contracts

Version History

Version	Status	Beschreibung
1.0.0	Completed	Strukturmigrationen EXC-0005 und EXC-0007 durchgeführt und verifiziert, Favicon-Build-Fehler FND-EXC-0007-01 aufgedeckt

End of Document

MIG-EXC-0005-0007

CAPITAL-AI Migration Report

Version 1.0.0

POST-PR Production Snapshot — Subscription Entitlements Live Refresh

Quelle: docs/production/evidence/PR_POSTCHECK_2026-08-03_SUBSCRIPTION_ENTITLEMENTS_LIVE_REFRESH.md

POST-PR Production Snapshot — Subscription Entitlements Live Refresh

Date: 2026-08-03 **Scope:** Post-creation production revalidation for the refreshed subscription-entitlement and Warren Buffett access PR.

This document is completed after the PR is opened. It intentionally records the production baseline, connected-service state, HTTP smoke result, drift assessment, and final-head CI requirement. No production mutation is authorized by this evidence file.

PRE-PR Production Snapshot — Subscription Entitlements Live Refresh

Quelle: docs/production/evidence/PR_PRECHECK_2026-08-03_SUBSCRIPTION_ENTITLEMENTS_LIVE_REFRESH.md

PRE-PR Production Snapshot — Subscription Entitlements Live Refresh

Date: 2026-08-03 **Scope:** Recreate the subscription-entitlements / Warren Buffett gating work on the latest production repository baseline after concurrent merges. **Main SHA:** af-b25a0116beb867b9de37058f76aad3d7291187 **Platform version:** 0.6.0

Live repository baseline

- PR #68 is merged on `main` and combines the GA4/CookieHub integration with the PlatformDirector manifest correction.
- The drift between the previous entitlement PR base (`07926dc...`) and current `main` affects only analytics/CookieHub/PlatformDirector files; no entitlement-scope file overlaps were observed.
- This branch was therefore created directly from `afb25a0116beb867b9de37058f76aad3d7291187` rather than reusing the older branch base.

HTTP production smoke

- `https://capital-ai.online` and `/healthz` are `UNVERIFIABLE_FROM_RUNNER` in this execution environment because DNS resolution for `capital-ai.online` failed.
- This is an execution-runner limitation and is **not** classified as a production outage.

Supabase live security state

Project: `ryzywoktpmyhwzxmsty`

Known advisor state remains unchanged:

1. INFO `rls_enabled_no_policy` on `public.screening_slo_evidence` — intentional server-only/fail-closed table state.
2. WARN `auth_leaked_password_protection` — accepted Free-tier constraint under ADR-0031 until plan upgrade.

No Supabase DDL/DML is executed by this PR. The new quota migration remains repository-only for controlled production handoff.

Stripe live recurring prices

Observed active live prices immediately before PR creation:

- Starter monthly: EUR 7.00 — `price_1TnEUEPKr4joNbEctWTgogW6`

- Starter yearly: EUR 75.60 — `price_1TpDDNPKr4joNbEcGm7ngSmp`
- Pro monthly: EUR 29.00 — `price_1TpD0hPKr4joNbEc50cS0PKr`
- Pro yearly: EUR 248.00 — `price_1TpDVYPKr4joNbEck8SdA1sK`
- Enterprise monthly: EUR 109.00 — `price_1Tl6GnPKr4joNbEckzqM3SoC`
- Enterprise yearly: EUR 1,280.00 — `price_1TpDZ1PKr4joNbEckxXITdTc`

The existing Pro yearly pricing discrepancy remains unresolved: 10% off EUR 29/month would be EUR 313.20/year, not EUR 248.00. This PR does not mutate Stripe.

Decision

Proceed with a fresh PR from the current production repository baseline implementing only the canonical subscription entitlement contract, server-side quota enforcement, Buffett authorization API, tests, ADR, and repository migration. Manual merge only.

CAPITAL-AI — Production SLO Persistence Acceptance Record

Quelle: docs/production/evidence/PRODUCTION_SLO_PERSISTENCE_ACCEPTANCE_2026-08-02.md

CAPITAL-AI — Production SLO Persistence Acceptance Record

- **Record ID:** PROD-SLO-PERSISTENCE-ACCEPTANCE-2026-08-02
- **Record Version:** 1.0.0
- **Platform Version:** 0.6.0
- **Date:** 2026-08-02
- **Environment:** Production
- **Status:** ACCEPTED / CLOSED
- **Final Readiness Status:** PRODUCTION_SLO_PERSISTENCE_READY
- **Classification:** Production Acceptance Evidence
- **Source of Truth Migration:** supabase/migrations/20260802160000_screening_slo_evidence.sql
- **Production Sink:** server/screeningSloSupabaseSink.ts
- **Related Merge:** PR #50 , merge commit 7f33f9c2f311dbb7e31cb26dadeee5bedb76abb3

1. Purpose

This record closes the production acceptance lifecycle for persistent Screening SLO Evidence. It persists the successful completion result of the controlled production migration and links that result to the repository artifacts that define the schema and production sink.

The controlled migration execution completed with the explicit final status:

PRODUCTION_SLO_PERSISTENCE_READY

Before this record was created, a renewed GitHub repository search did not find that exact completion status as a versioned repository artifact. This record closes that auditability gap; it does not re-run the production database migration and does not modify Supabase, Render, Stripe, scoring algorithms, frontend code, or existing SLO evidence.

2. Acceptance Basis

The acceptance is based on two evidence classes:

1. **Controlled production execution evidence** — the migration/validation procedure defined for 20260802160000_screening_slo_evidence.sql completed successfully and returned PRODUCTION_SLO_PERSISTENCE_READY.

2. **Repository evidence** — the production repository contains the migration, the persistent `SupabaseScreeningSloSink`, the security constraints and the merged implementation history.

The production execution result is recorded here as acceptance evidence. This GitHub closure did not independently repeat destructive or mutation-based database tests.

3. Repository Evidence

3.1 Migration

`supabase/migrations/20260802160000_screening_slo_evidence.sql` defines:

- `public.screening_slo_evidence`
- UUID primary key with `gen_random_uuid()`
- the `HEALTHY`, `DEGRADED`, `UNAVAILABLE`, `NO_RUNTIME_EVIDENCE` state constraint
- non-negative `quote_age_ms` constraint
- `score_impact_enabled = false` database constraint
- `hard_screening_block_enabled = false` database constraint
- JSONB `reasons` and `evidence`
- indexes for `observed_at`, `correlation_id`, `symbol`, and `state`
- partial symbol index with `symbol IS NOT NULL`
- Row Level Security
- no public/authenticated client write policy in the migration
- append-only mutation prevention function
- `BEFORE UPDATE OR DELETE` append-only trigger

3.2 Production sink

`server/screeningSloSupabaseSink.ts`:

- writes server-side only
- requires a privileged Supabase server key
- writes to `screening_slo_evidence`
- maps the SLO contract fields to the production schema
- does not claim `persisted: true` after a failed database write
- does not use a local-file or synthetic persistence fallback

3.3 Integration history

PR [#50](#) merged the persistent production SLO sink and append-only migration into `main`. The merge commit is:

`7f33f9c2f311dbb7e31cb26dadeee5bedb76abb3`

4. Production Readiness Matrix

Check	Status	Acceptance Evidence
Migration present	PASS	Migration exists in production repository
Production Supabase unambiguous	PASS	Controlled production migration execution completed with READY status
Table created	PASS	Controlled production execution result
Columns complete	PASS	Controlled production execution result + repository migration definition
Constraints active	PASS	Controlled production execution result + repository migration definition
Indexes active	PASS	Controlled production execution result + repository migration definition
RLS active	PASS	Controlled production execution result + repository migration definition
Client write policies excluded	PASS	Controlled production execution result + migration grants no client write policy
Service write possible	PASS	Controlled write validation completed in production execution
UPDATE blocked	PASS	Controlled append-only validation completed in production execution
DELETE blocked	PASS	Controlled append-only validation completed in production execution
<code>score_impact_enabled=true</code> blocked	PASS	Controlled constraint validation completed in production execution
<code>hard_screening_block_enabled=true</code> blocked	PASS	Controlled constraint validation completed in production execution
Backend schema compatible	PASS	<code>SupabaseScreeningSloSink</code> field mapping matches migration schema

5. Controlled Validation Marker

The production validation procedure specifies one successful append-only marker:

- `correlation_id = migration-validation-test`
- `evidence.type = migration-validation`

Because the table is intentionally append-only, this record does not attempt to delete or modify that marker.

6. Security and Governance Closure

The accepted architecture preserves these boundaries:

- SLO evidence cannot modify canonical asset scores.
- `score_impact_enabled` remains database-enforced `false`.
- `hard_screening_block_enabled` remains database-enforced `false`.
- browser/anonymous/authenticated clients receive no write permission from this migration.
- SLO evidence is append-only after insertion.
- privileged persistence remains a server-side responsibility.

No secrets, Supabase keys, connection strings or privileged credentials are stored in this record.

7. Versioning Consequence

This acceptance closes a production capability within platform version `0.6.0`. It **does not by itself trigger a platform version increment**. Platform version changes are release decisions governed by `ADR-0030 – Platform Version and Release Lifecycle`.

8. Closure Decision

All critical acceptance checks defined by the production migration procedure completed with PASS and the reported final status is `PRODUCTION_SLO_PERSISTENCE_READY`.

Therefore:

Production SLO Persistence is accepted and this work package is CLOSED.

Future schema or persistence-contract changes require a new migration, new validation evidence, and a new or superseding acceptance record rather than rewriting this historical acceptance outcome.

Render Auto-Deploy (checksPass) Verification — 2026-08-09

Quelle: docs/production/evidence/RENDER_AUTODEPLOY_WEBHOOK_VERIFICATION_2026-08-09.md

Render Auto-Deploy (checksPass) Verification — 2026-08-09

Related: ADR-0037, docs/runbooks/RENDER_PRODUCTION_EVIDENCE_HANDOFF.md

Incident

The merge of PR #134 (570e4c5) into `main` did not trigger an automatic Render deploy despite `render.yaml` /the live service both declaring `autoDeployTrigger: checksPass` and GitHub CI reporting `success` for that commit. The last automatic (`trigger: new_commit`) deploy before the incident was for the PR #131 merge on 2026-08-08 09:39 UTC — i.e. the webhook path had been working roughly 24h earlier and stopped delivering afterwards, rather than never having worked.

Production was unblocked by manually redeploying commit 570e4c5 via the Render API (`dep-d9s9e7on74is7388ea60` , live 2026-08-09 15:06 UTC, `trigger: api`).

Remediation

Repository owner re-authorized the Render GitHub App's repository access for SvenKulesa/Finance (GitHub → Settings → Installations → Render → Configure). This is a dashboard-only action; no API surface for GitHub App installation scope was available to automate or verify it directly.

Verification method

This document is itself the verification payload: a trivial, non-functional PR merged to `main` after the GitHub App reconnect, used only to observe whether the resulting merge commit produces a Render deploy with `trigger: new_commit` (automatic) instead of requiring a manual `trigger: api` redeploy. No application code changes.

Expected evidence after merge (recorded manually, not by this file):

```
mergeCommitSha:
renderDeployId:
renderDeployTrigger: new_commit | api
renderDeployStatus: live | build_failed | ...
autoDeployConfirmed: yes | no
```

changelog dev

Quelle: docs/release/changelog-dev.md



⊞ CAPITAL-AI DOCUMENTARY

AUTONOMOUS AI DOCUMENT HYGIENIST • VERSION 0.5.4

System-Metadaten	Spezifikation
Plattform-Identität	Capital-AI Documentary (V0.5.4)
Gründer & Inhaber	Sven Kulesa
Zentrale E-Mail	sven.kulesa@capital-ai.online
Echtheits-Emblem	⊞ CAPITAL-AI CORE
Status	Revisionssicher verifiziert & bereinigt

Changelog-dev: CAPITAL-AI Developer Logs

All updates, workarounds, and environmental parameters for local and Cloud Run container execution are logged here.

[0.6.4-dev] - 2026-07-10

Added

- **ADR-0003.5 - Identity Access Management (Sicherheitsmanagement & Compliance):** Neues Entscheidungsdokument (docs/adr/ADR-0003_5-identity-access-management.md) zur Definition des Owner-IAM, erzwungener Passkey (WebAuthn/FIDO2) & 2FA Multi-Faktor-Absicherung, Step-Up Tokenisierung und geschützter Systemzonen.
- **ADR-0008 - Behebung des Reaktivitäts- & Lebenszyklus-Ausfalls im Document Hygiene Panel:** Neues Entscheidungsdokument (docs/adr/ADR-0008-document-hygiene-lifecycle-fix.md) zur Behebung verfrühter administrativer Status-Abfragen vor abgeschlossener Benutzer-Authentifizierung im Client.
- **Interaktiver Proof of Concept (PoC):** Entwicklung eines voll funktionsfähigen, interaktiven Sicherheits- & Compliance-PoC (src/components/SicherheitsmanagementPoC.tsx), um die neuen Authentifizierungsschleifen, geschützten Zonen und anonymisierten Audit-Logs visuell zu validieren.

- **Anforderungskatalog Integration:** Einbindung des Anforderungskatalogs (docs/adr/anforderungskatalog.md) als prioritäres, ausstehendes Dokument (PENDING) im Documentary Wiki.

Fixed

- **Lebenszyklus-Fehler im Document Hygiene Panel:** Absicherung der initialen Abfragen im DocumentHygienePanel via Conditional Early-Return und Erweiterung des Dependency-Arrays um currentUserEmail . Dies verhindert unauthorisierte Backend-API-Fehler beim Laden der Komponente.
- **Behebung von Namenskonventionsfehlern:** Bereinigung verbleibender Referenzen des Alt-Projekts AIF-Capital-Core in src/components/InteractModule.tsx hin zur einheitlichen Marken- und Systemidentität Capital-AI .

[0.6.3-dev] - 2026-07-10

Added

- **ADR-0007 - Compliance-Wertschöpfungskette:** Neues Entscheidungsdokument (docs/adr/ADR-0007-compliance-value-chain.md) zur Bereitstellung einer lückenlosen Compliance-Wertschöpfungskette (Ingestion, PII-Maskierung, deterministische Formeln, manipulationssichere Logs, PDF-Berichtszertifizierung).

Fixed

- **Behebung von Namenskonventionsfehlern:** Korrektur der alten Systembezeichnung AIF-Capital-Core im DSGVO-Konformitätszertifikat (docs/DATENSCHUTZ_PROTOKOLL.md) zu Capital-AI Compliance-Ausschuss zur Wahrung einer einheitlichen Marken- und Systemidentität.

[0.6.2-dev] - 2026-07-10

Added

- **Dauerhafter Admin-Bypass / Auto-Login (Google AI Preview & Dev):** Implizierter Auto-Login für die E-Mail-Adresse sven.kulesa@gmx.net mitsamt der Systemstufe Enterprise im Preview- und Entwicklungsmodus (erkannt via hostname run.app oder localhost), um Admin-Optionen out-of-the-box im Google AI Developer Space dauerhaft anzuzeigen.
- **ADR-0005 - Ökosystem-Frontend-Modul-Einbindung:** Neues Architekturdokument (docs/adr/ADR-0005-frontend-module-integration.md) zur standardisierten Einbindung des Front-Ends via Module Federation und sicheren postMessage Event-Bussen im Capital-AI FinTech-Ökosystem.
- **ADR-0006 - Anbindung des Plattform-Direktors:** Neues Architekturdokument (docs/adr/ADR-0006-platform-director-connection.md) zur Definition der nächsten architek-

tonischen Meilensteine bei der Anbindung einer übergeordneten Server-Governance-Schicht (Platform Director).

Changed

- **ADR-Historienindex:** Synchroner Eintrag der neuen Entscheidungsdokumente in der `/docs/adr/adr_history.json`.

[0.6.1-dev] - 2026-07-10

Changed

- **Entfernung der CORE-Branding-Kennzeichnung:** Die statische Bezeichnung "CORE" wurde auf allen Hauptebenen der Benutzeroberfläche (Landingpage, Hauptdashboard-Header und Navigationsschublade) vollständig entfernt und durch die einheitliche Revisionsnummer `VERSION 0.5.4` ersetzt.
- **Deaktivierung des Header-Dropdown-Panels:** Das ausklappbare Profil-Dropdown-Panel in der oberen Navigationsleiste hinter dem Suchfeld wurde mitsamt dem ChevronDown-Ausklapppfeil vollständig entfernt, um die visuelle Hierarchie zu beruhigen. Alle Profil- und Systemoptionen verbleiben exklusiv in der ausfahrbaren Drawer-Seitenleiste.

Added

- **Architekturdokumentation (ADR-0004):** Ein neues detailliertes Architekturentscheidungs-Dokument (`docs/adr/ADR-0004-branding-and-panel-removal.md`) wurde angelegt, um diese Branding- und Navigationsanpassung revisionssicher und dauerhaft zu protokollieren. Der ADR-Index `docs/adr/adr_history.json` wurde synchron aktualisiert.

[0.6.0-dev] - 2026-07-04

Added

- **Kraken Pro Referral Card:** Added custom visual cards displaying the partner referral text and outbound action buttons inside `Dashboard.tsx` and `ImpressumAgb.tsx`.
- **Real-Time Push-Up Notifications:** Added `Watchlist.tsx` for real-time asset monitoring, connected it to the `RealtimeAiNewsfeed` update loop, and implemented low-latency synthesizer alerts via the standard browser `AudioContext` interface.

Changed

- **Modul 1 Designation Removal:** Completely removed all visible, literal, and comment mentions of "Modul 1" or "Module 1" across the application files (`Dashboard.tsx` , `CapitalAiLogo.tsx` , `LandingPage.tsx` , `GuestCliffhangerModal.tsx` , `RawMaterialsDashboard.tsx` , `RealTimeRiskAssessment.tsx`). Rebranded modules/titles to either "CORE" or specific technical labels (e.g. "Rohstoff-Analyse").

- **Global Support E-Mail Integration:** Swapped support address to `support@capital-ai.online`. Added global header anchor in `Dashboard.tsx`, registered custom tooltip items in retractable drawer navigation, and appended styling blocks in footer.
- **Impressum Direct Contact update:** Adjusted contact state values inside `ImpressumAgb.tsx` to route business communications to `sven.kulessa@capital-ai.online`.

[0.5.4-dev] - 2026-07-02

Changed

- **Platform Version:** Updated version identifier across all user-facing pages, system documents, and export templates to **Version 0.5.4**.
- **Audit-Trail Box Removal:** Removed the "Audit-Trail & Begründung" info box from the Crypto Scoring Enterprise overview per client request.

Fixed / Ignored

- **Stripe & Supabase Environmental Variables:** Unset environment variables for Stripe and Supabase are gracefully ignored/bypassed in this development and preview sandbox. Mock state fallbacks and local context handling are leveraged for uninterrupted execution.

Changelog - CAPITAL-AI Plattform

Quelle: docs/release/changelog-prod.md

Changelog - CAPITAL-AI Plattform

All notable changes to this project will be documented in this file. The format is based on [Keep a Changelog](#), and this project adheres to Semantic Versioning.

[0.6.0-Beta] - 2026-07-04

Added

- **Kraken Pro Empfehlungs-Partner-Link:** Integration eines exklusiven Kraken Pro Partner-Bonus-Kartenmoduls unter dem Haftungsausschluss im Dashboard-Footer sowie im Impressum. Ermöglicht schnelle Empfehlungsanmeldungen (Code: `yc4ggk3f`) mit Prämienteilung.
- **Real-Time Push-Up Benachrichtigungen:** Aktivierung von interaktiven Push-Up-Meldungen für den Newsfeed und das persönliche Radar (Watchlist). Nachrichten, die zu einem neu berechneten Asset-Scoring von unter 3.0 (kritisch/bearish) oder über 7.0 (Breakout/bullish) führen, lösen sofort eine akustische Chime-Meldung (Web Audio API Synthesizer) und ein visuelles Pop-up aus.
- **Interaktives Watchlist-Modul (Persönliches Radar):** Integration eines personalisierten Radars zur Überwachung beliebter Werte. Ermöglicht Echtzeit-Kursabfragen, Score-Anzeigen und beinhaltet Schnelltest-Simulatoren (▲ RLY / ▼ CRH) zur Verifizierung von Push-Up-Benachrichtigungen.

Changed

- **Entfernung der "Modul 1" Bezeichnung:** Vollständige Bereinigung der Applikationsseiten, Dokumente, Slogans, Logos und Code-Kommentare von der veralteten Bezeichnung "Modul 1" bzw. "Module 1". Die primäre Systemkennzeichnung wurde stattdessen einheitlich auf "CORE" bzw. anwendungsspezifische Fachbezeichnungen (z. B. "Rohstoff-Analyse") überführt.
- **Kontakt E-Mail Korrektur & Globale Erreichbarkeit:** Aktualisierung aller Support-Kanäle auf die offizielle E-Mail-Adresse `support@capital-ai.online`. Zur Erhöhung der Sichtbarkeit wurde diese global im Header (Sticky Top-Navigation), im Retractable Sidebar-Drawer (Kategorie "Compliance & Support") sowie im Footer der Seite fest verankert.
- **Direktkontakt im Impressum:** Anpassung der Impressum-Kontaktdetails zur Ausweisung von `sven.kulessa@capital-ai.online` als geschäftlicher Direktkontakt für administrative Anfragen.
- **Unified Production Milestone Elevation (v0.6.0):** Upgraded all system-wide version tags and configuration indicators (including `metadata.json`, `ComplianceExporter`, `Au-`

ditLogs , ProfilePage , DashboardSearchFilter , SentimentDashboard , and AssetUniverseDashboard) from v0.5.4 to v0.6.0-Beta.

- **Orchestrator and Scoring Alignment:** Harmonized specialized scoring engines across both Express API endpoints and interactive client dashboards, ensuring fully deterministic evaluation parity.
- **Database Resilience & Cache Layering:** Implemented a local file-system/in-memory backup storage state that automatically catches and resolves connectivity exceptions, guaranteeing seamless operation for administrator accounts.

[0.5.4-Beta] - 2026-07-03

Changed

- **Sicherheits- & Rollenbereinigung (Gast-Modus & Global Admin):**
- **Entfernung Gast-Modus:** Der Gast-Modus wurde vollständig aus der Benutzeroberfläche (LandingPage & Dashboard) entfernt. Es gibt keine sichtbaren Links, Knöpfe oder Verknüpfungen mehr.
- **Programmweiter Schreibschutz:** Der Gast-Modus wurde programmweit über einen Secret Key (CAPITAL_AI_SECRET_KEY_2026) gesichert und schreibgeschützt. Unbefugte Versuche, eine Gastsitzung zu initialisieren, werden mit einer Sicherheitswarnung blockiert.
- **Zuweisung Global Administrator:** Die Administratorenrechte wurden exklusiv auf die E-Mail-Adressen sven.kulessa@gmail.com und sven.kulessa@gmx.net beschränkt. Jegliche Gast-Bypässe für das Administrations-Panel wurden entfernt.

[0.5.4-Beta] - 2026-07-02

Added

- **Modul 1: Rohstoff-Kategorisierung & AI-Scoring:**
- Full-stack integration of multi-agent orchestration for physische and kritische Rohstoffe.
- Implementation of **ClassificationAgent** to determine main class, subclass, market type, and valuation modes.
- Implementation of **FundamentalsAgent** for geological and reserve evaluation (ore grade, tonnage, recycling).
- Implementation of **RiskAgent** to map country bottlenecks, supply chain friction, ESG factors, and producer concentration.
- Implementation of **ValuationAgent** to analyze industrial and military/defense importance.
- **Master Orchestrator (RawMaterialsOrchestrator):** Coordinates parallel execution of agents and routes data.
- **Centralized Scoring Engine (RawMaterialsScoringService):**
- Single Source of Truth for formula-based weighted scoring calculations.

- Version-controlled configs (`v1.0-standard` and `v1.2-critical-focused`).
- Handling of missing fields through dynamic penalty calculations on confidence and data quality ratings.
- **Interactive UI Panel (`RawMaterialsDashboard`):**
 - Glassmorphic bento-grid dashboard in dark-mode style.
 - Dynamic autocomplete search bar with automatic rating score output on select.
 - Interactive "Tuning Sandbox" allowing real-time "What-If" parameter adjustments and manual score recalculation.
 - Fully traceable terminal-like logs showcasing agent step-by-step reasoning.
- **Backend API Routes (`/api/raw-materials/*`):**
 - `/api/raw-materials/list` : Fetches predefined commodity registries and standard scores.
 - `/api/raw-materials/analyze` : Orchestrates and queries the Gemini-3.1-pro-preview models.
 - `/api/raw-materials/score` : Instantly calculates deterministic scores on manual tuning inputs.

Changed

- **Dashboard Sidebar:** Integrated the "Rohstoff-Bewertung" nav option under the Analysetools accordion.
- **Server Entrypoint:** Mounted the new modular router on the main Express server with secure lazy-loading of GoogleGenAI SDK.
- **Type safety definitions:** Consolidated `src/types/rawMaterials.ts` and schema boundaries in `src/schemas/rawMaterialsValidation.ts` .

End of Changelog. Version pinned to 0.6.0 Beta.

Alpaca-Shadow: Produktionsverifikation und Laufzeitgrenzen

Quelle: docs/runbooks/ALPACA_SHADOW_PRODUCTION_VERIFICATION.md

Alpaca-Shadow: Produktionsverifikation und Laufzeitgrenzen

Stand: 2026-08-14

Scope: read-only Repository-Änderung; keine Render-, Supabase-, Stripe- oder Trading-Mutation.

Ziel

Der Produktionsstart prüft Alpaca einmalig über den bestehenden read-only Endpoint `GET /v2/stocks/{symbol}/trades/latest`. Das Ergebnis ist beobachtend und bleibt vollständig vom kanonischen Preis- und Scoringpfad getrennt.

Redigierte Startdiagnose

Erlaubte Felder:

- `configured`
- `authenticated`
- `state`
- `symbol`
- `feed`
- `retrievedAt`

Nicht protokolliert werden Key-ID, Secret, Preis, Response-Body, Fehlerdetail oder Header. `READY`, `STALE` und `DEGRADED` belegen eine erfolgreiche Authentisierung; `UNAVAILABLE` und `NOT_CONFIGURED` bleiben nicht-blockierende, sichtbare Shadow-Zustände. Der Smoke-Test beeinflusst weder Server-Readiness noch Marktpreise oder Scores.

Produktionsgrenze Document Hygiene

Die Produktionslaufzeit ist read-only:

- kein File-Watcher;
- kein Erzeugen von `uploads/document_hygiene.json`;
- kein ADR-Seeding;
- keine lokalen Document-Hygiene-Mutationen;
- schreibende Admin-Routen antworten fail-closed mit `503 / DOCUMENT_HYGIENE_READ_ONLY`.

Lokale Entwicklungs- und Testumgebungen behalten das bestehende schreibende Werkzeugverhalten.

Verifikation nach Merge

1. Main-CI und Deployment-Identität müssen PASS sein.
2. Render-Boot-Logs müssen die read-only Meldung der Document Hygiene ohne `CAPITAL_AI_RUNTIME_ARTIFACT_READ_ONLY`-Write-Fehler zeigen.
3. Genau eine redigierte `Alpaca shadow startup smoke`-Diagnose prüfen.
4. Keine Credentials oder Provider-Response-Inhalte dürfen im Log stehen.
5. Kanonischen Quote-/Scorepfad auf unveränderten Provider und unveränderten Score prüfen.

Rücksetzung

Ein human-autorisierter Revert-PR stellt den vorherigen Codezustand wieder her. Es gibt keine Daten- oder Schemarücksetzung, weil dieser PR keine externe Mutation ausführt.

Nachgelagerter, blockierter Schritt

Der produktive Outbox-Fehler wegen der fehlenden Supabase-RPC `public.claim_outbox_job(p_lease_owner, p_lease_seconds)` ist nicht Teil dieses PRs. Die Migration benötigt eine separate Owner-Autorisierung, Pre-Mutation-Baseline, Rollback-Runbook und Post-Mutation-Verifikation. Bis dahin bleibt dieser Schritt blockiert.

Runbook — CI-Konsolidierung auf build-and-test

Quelle: docs/runbooks/CI_BUILD_AND_TEST_CONSOLIDATION.md

Runbook — CI-Konsolidierung auf build-and-test

Status: OWNER ACTION REQUIRED BEFORE MERGE

ADR: ADR-0073

Vorher

Required Checks:

- `capital-ai-ci`
- `build-and-test`
- `GitGuardian Security Checks`

Nachher

Required Checks:

- `build-and-test`
- `GitGuardian Security Checks`

Owner-Schritte

1. Live-Ruleset `main-production-protection` öffnen.
2. Nur `capital-ai-ci` aus Required status checks entfernen.
3. Prüfen, dass `build-and-test` und `GitGuardian` erhalten bleiben.
4. Keine Bypass Actors hinzufügen.
5. Einstellung speichern.
6. Im Konsolidierungs-PR bestätigen, dass der Ruleset-Cutover abgeschlossen ist.
7. Erst danach Merge freigeben.

Fail-Closed

Falls `capital-ai-ci` vor dem Merge weiterhin Required ist, darf der PR nicht gemergt werden. Nach einem Merge würde sonst jeder neue PR auf einen nicht mehr gemeldeten Check warten.

Umsetzung im Repository

`.github/workflows/capital-ai-ci-shadow.yml` wird **nicht gelöscht**, sondern stillgelegt:

- `on:` enthält nur noch `workflow_dispatch`;

- der Workflow meldet für Pull Requests keinen `capital-ai-ci`-Check mehr;
- Jobinhalt, Pinning, Read-only-Permissions und `persist-credentials: false` bleiben unverändert.

Grund: Der Required-Check `Sicherheit geänderter Workflows` verbietet Workflow-Löschungen fail-closed und ohne Ausnahmepfad

(`scripts/security/verifyChangedWorkflowSecurity.mjs`, geladen aus dem vertrauenswürdigen `main`-Stand). Die Stilllegung erreicht dasselbe Ziel, ohne diese Sicherheitsinvariante aufzuweichen.

Rollback

Frischen Branch erstellen und den `pull_request`-Trigger wieder eintragen:

```
on:
  pull_request:
    branches: [main]
    types: [opened, synchronize, reopened, edited]
  workflow_dispatch:
```

`capital-ai-ci` erst nach einem realen PASS wieder als Required Check setzen.

Erwarteter Nutzen

- ein statt zwei vollständiger Node-/Build-Läufe pro PR;
- geringere Actions-Kosten;
- unverändertes Owner-Gate;
- unveränderte technische Prüftiefe.

CAPITAL-AI Runbook: Deployment-Rollback und Backup

Quelle: docs/runbooks/DEPLOYMENT_ROLLBACK_UND_BACKUP.md

CAPITAL-AI Runbook: Deployment-Rollback und Backup

Document ID

RUNBOOK-0001

Bezug

ARCH-AUDIT-0002 (Enterprise FinTech Architecture Audit), Kapitel 14.5, Maßnahme H7 „Deployment-Rollback und Backup-Verfahren“.

Zusätzliche Render-Governance seit 03.08.2026:

- docs/architecture/RENDER_PRODUCTION_CONFIGURATION_AUDIT.md (RENDER-AUDIT-0001)
- docs/adr/ADR-0037-render-production-configuration-governance.md
- docs/runbooks/RENDER_PRODUCTION_EVIDENCE_HANDOFF.md

Für aktuelle Render-Dashboard-/Service-Konfigurationsdetails ist `RENDER_PRODUCTION_EVIDENCE_HANDOFF.md` maßgeblich. Dieses Runbook behandelt den Recovery-Ablauf.

Status

Aktiv

Geltungsbereich

Dieses Runbook beschreibt, wie ein fehlgeschlagenes oder fehlerhaftes Produktiv-Deployment von CAPITAL-AI zurückgerollt wird und wie die Datenbasis (Supabase) im Notfall wiederhergestellt werden kann. Es deckt die Anwendungsebene (Render-Deployment) und die Datenebene (Supabase Postgres) getrennt ab, weil beide unterschiedliche Wiederherstellungsmechanismen haben und ein Rollback der einen Ebene die andere nicht automatisch mitzieht.

Automatisierter Teil dieses Runbooks: `npm run predeploy:check` (`scripts/automation/verifyDeploymentReadiness.ts`). Das Skript prüft vor jedem Deploy automatisierbare Vorbedingungen (siehe Abschnitt 4) und läuft als eigener Schritt in der CI-Pipeline (`.github/workflows/ci.yml`). Es ersetzt dieses Runbook nicht — es verhindert nur eine Teilmenge der Fehler, die sonst erst nach dem Deploy sichtbar würden.

1. Anwendungsebene: Rollback eines Render-Deployments

CAPITAL-AI läuft auf Render.com als Docker-Service (`render.yaml` , `runtime: docker`) mit `healthCheckPath: /healthz` . Render nutzt diesen Health-Check für Deployment Readiness: ein neuer Deploy wird erst dann live geschaltet, wenn die neue Instanz erfolgreich auf den konfigurierten Health-Check antwortet. Schlägt der neue Deploy vor dem Traffic-Switch fehl, bleibt die vorherige funktionierende Version aktiv.

Wichtig: `/healthz` bestätigt derzeit vor allem Prozess-Liveness und keine vollständige fachliche Readiness aller externen Abhängigkeiten. ADR-0037 definiert deshalb die Weiterentwicklung zu einem expliziten Readiness Contract.

Manuelles Eingreifen ist nötig, wenn ein Deploy zwar den Health-Check besteht, sich aber im Betrieb als fehlerhaft herausstellt (z. B. ein Bug, der erst bei echtem Nutzerverkehr auftritt, oder ein funktionaler Regressionsfehler wie die in ARCH-AUDIT-0002 dokumentierten P0-Vorfälle).

1.1 Manueller Rollback über das Render-Dashboard

Aktuelle Render-Semantik (erneut verifiziert 03.08.2026): Render kann bei einem Rollback auf einen vorherigen erfolgreichen Deploy dessen vorhandenes Build-Artefakt wiederverwenden. Ein Rollback ist deshalb nicht mit einem vollständigen Neu-Build des historischen Commits gleichzusetzen.

Ablauf:

1. Render-Dashboard öffnen → Service `capital-ai` → Tab **Deploys**.
2. Den letzten bekannten funktionierenden Deploy anhand von Commit-SHA, Zeitpunkt und Incident- Evidence identifizieren.
3. Bei diesem erfolgreichen Deploy **Rollback** auswählen.
4. Vor Bestätigung prüfen, dass der Ziel-Deploy tatsächlich die gewünschte Code-/Build-Version enthält und keine Datenbank-Rücksetzung erwartet wird.
5. Rollback ausführen.
6. Nach dem Rollback: Render Health Check und die ursprünglich betroffene Funktion gezielt verifizieren; ein grüner `/healthz` allein ist kein fachlicher Regressionstest.

Auto-Deploy-Schutz nach Dashboard-Rollback

Render deaktiviert beim Rollback über das Dashboard automatische Deploys. Das schützt davor, dass ein nachfolgender Push den fehlerhaften Zustand unmittelbar wieder einführt.

Nach erfolgreicher Ursachenbehebung muss Auto-Deploy bewusst wieder auf den gemäß ADR-0037 freigegebenen Zustand gesetzt werden. Zielzustand für Production ist `After CI Checks Pass` (`autoDeployTrigger: checksPass`), sobald Dashboard Evidence und GitHub Required Checks dies bestätigen.

Was beim Render-Rollback nicht pauschal auf den historischen Zustand zurückkehrt

Ein Render-Rollback ist kein vollständiger Infrastruktur-Zeitmaschinen-Snapshot. Je nach Konfiguration verwenden einzelne Service-Einstellungen weiterhin den aktuellen Zustand. Insbesondere sind Persistent Disks und bestimmte gemeinsam genutzte Environment-Group-Zustände separat zu betrachten. Vor einem Rollback mit Configuration Drift daher immer `RENDER_PRODUCTION_EVIDENCE_HANDOFF.md` heranziehen.

1.2 Rollback per Git, falls ein nachvollziehbarer Revert gewünscht ist

Ein `git revert` ist eine **andere Recovery-Methode** als ein Render Instant Rollback. Er erzeugt einen neuen Git-Commit, der die fehlerhafte Codeänderung zurücknimmt.

```
git fetch origin main
git revert <fehlerhafter-commit-hash> --no-edit
git push origin main
```

Ob und wann Render diesen Revert deployt, hängt von der tatsächlichen Auto-Deploy-Konfiguration ab. Nach ADR-0037 soll Production erst nach bestandenen CI Checks deployen. Daher darf dieses Runbook nicht mehr pauschal voraussetzen, dass jeder Push auf `main` sofort einen Render-Deploy auslöst.

Dieser Weg ist sinnvoll, wenn:

- die Ursache klar einem Commit zugeordnet ist;
- der Revert selbst CI-/Security-Gates durchlaufen soll;
- der Git-Verlauf den Recovery-Schritt dauerhaft abbilden soll.

Ein Render Dashboard Rollback ist dagegen sinnvoll, wenn Time-to-Recovery wichtiger ist und ein bekannter guter Build sofort wiederhergestellt werden soll.

1.3 Was ein Rollback NICHT abdeckt

Ein Rollback auf Anwendungsebene macht keine Datenbankänderungen rückgängig. Wenn der fehlerhafte Deploy bereits Schreibzugriffe mit falscher Logik ausgeführt hat (z. B. fehlerhafte Daten in Supabase geschrieben), muss das getrennt behandelt werden — siehe Abschnitt 2.

Zusätzlich gilt nach ADR-0037:

- lokaler Render-Dateisystemzustand ist ohne verifizierten Persistent Disk als ephemeral zu behandeln;
- ein Code-Rollback darf nicht als Wiederherstellung von Subscription-/Credit-Dateien betrachtet werden;
- Secret-Rotation wird niemals durch Rückkehr zu einem möglicherweise kompromittierten alten Secret ersetzt.

2. Datenebene: Supabase Backup und Wiederherstellung

Das aktive Supabase-Projekt ist `ryzywoktpmyhwzxmstyu` (Postgres 17, Region `eu-west-1`).

2.1 Wichtiger Hinweis zur tatsächlichen Backup-Konfiguration

Dieses Runbook kann die tatsächlich aktive Backup-Stufe (Plan-Tier, Retention-Zeitraum, Point-in-Time-Recovery-Fenster) nicht angeben. Die dafür verfügbaren Werkzeuge in der ursprünglichen Audit-Session lieferten Projekt-Metadaten, aber keine vollständige Backup-/Retention-Konfiguration. Gemäß der No-Demo-Data-Policy wird dieser Wert bewusst **nicht** geschätzt oder angenommen.

Vor dem ersten Ernstfall muss der Betreiber daher manuell verifizieren: Supabase-Dashboard → Projekt → **Settings/Database/Backups** entsprechend der aktuellen Supabase-UI. Dort steht die tatsächliche Backup-Frequenz und Aufbewahrungsdauer des aktuellen Plans. Dieser Wert sollte nach Prüfung als Evidence ergänzt werden.

2.2 Verifizierte Backup-Konfiguration

(Vom Betreiber auszufüllen, nachdem Abschnitt 2.1 im Dashboard geprüft wurde. Solange dieser Abschnitt leer ist, gilt: Backup-Konfiguration nicht verifiziert.)

- Plan-Tier: —
- Automatisches Backup-Intervall: —
- Retention-Zeitraum: —
- Point-in-Time-Recovery verfügbar: —

2.3 Wiederherstellung über die Supabase-eigene Backup-Funktion

Sofern laut Abschnitt 2.2 automatische Backups aktiv sind, ist die Supabase-eigene Wiederherstellung der bevorzugte Weg. Vor jeder produktiven Wiederherstellung sind die aktuelle Supabase-Dokumentation, der genaue Restore-Scope und der erwartete Datenverlust seit dem Wiederherstellungspunkt zu prüfen.

2.4 Manuelle Sicherung per `pg_dump` (planunabhängiger Fallback)

Unabhängig vom Plan-Tier kann ein manueller Dump vor einer riskanten Migration als zusätzliche Absicherung verwendet werden:

```
# SUPABASE_DB_URL vorher manuell und NUR lokal in der Shell setzen.
pg_dump "$SUPABASE_DB_URL" \
  --format=custom \
  --file="capital-ai-backup-$(date +%Y%m%d-%H%M%S).dump"
```

Wiederherstellung eines solchen Dumps:

```
# SUPABASE_RESTORE_TARGET_URL nur lokal in der Shell setzen.
pg_restore --dbname="$SUPABASE_RESTORE_TARGET_URL" \
  --clean --if-exists \
  capital-ai-backup-<ZEITSTEMPEL>.dump
```

`--clean --if-exists` kann bestehende Objekte löschen. Eine Anwendung gegen ein produktives, befülltes Ziel ist deshalb ein destruktiver Datenbankvorgang und nur mit expliziter Production- Freigabe zulässig.

2.5 Strukturelle Wiederherstellung aus `supabase/migrations/`

Unabhängig von jedem Backup ist das Datenbankschema als Sequenz versionierter Migrationen im Repository nachvollziehbar (`supabase/migrations/*.sql`). Im Extremfall lässt sich damit die Struktur — nicht die Nutzdaten — rekonstruieren.

```
for f in supabase/migrations/*.sql; do
  psql "$SUPABASE_DB_URL" -f "$f"
done
```

Dies ersetzt kein Daten-Backup.

3. Ablauf im Vorfall

1. Incident klassifizieren: Code/Deploy, Render Configuration, Daten, Secret oder Kombination?
2. Aktuellen Production Commit und Service Configuration Evidence erfassen.
3. Time-to-Recovery entscheiden: - Render Instant Rollback für schnellen bekannten guten Build; - Git Revert für nachvollziehbaren Code-Recovery-Pfad.
4. Wenn Daten betroffen sind: Anwendung und Datenbank getrennt behandeln.
5. Nach Recovery: - Health; - betroffene fachliche Funktion; - Auth/IAM; - relevante externe Provider; - Logs/Alerts prüfen.
6. Auto-Deploy-Zustand nach Render Dashboard Rollback bewusst kontrollieren.
7. Incident Evidence und Root Cause dokumentieren.

4. Automatisierte Pre-Deploy-Prüfung

`npm run predeploy:check` (`scripts/automation/verifyDeploymentReadiness.ts`) prüft vor Deploys unter anderem:

1. `render.yaml` definiert `healthCheckPath`.
2. Statisch erkennbare serverseitige Environment-Variablen sind gegen Blueprint-/Alias-Listen geprüft.
3. `supabase/migrations/` besitzt gültige Migrationsdateien.
4. Dependency-/Lockfile-/SBOM-Policy.
5. Traceability, RAG Evidence und Prompt Registry.

Bekannte Grenze seit RENDER-AUDIT-0001

Die Environment-Abdeckung ist **nicht vollständig**, wenn Schlüssel dynamisch zusammengesetzt oder über Wrapper wie `getStripeVar(key)` an `getCleanEnv(key)` weitergegeben werden.

Beispiele, die daher separat durch ADR-0037 / Production Env Contract abgedeckt werden müssen:

```
STRIPE_PRICE_ID_STARTER_MONTHLY
STRIPE_PRICE_ID_STARTER_YEARLY
STRIPE_PRICE_ID_PRO_MONTHLY
STRIPE_PRICE_ID_PRO_YEARLY
STRIPE_ID_FOUNDER / STRIPE_PRICE_ID_FOUNDER
STRIPE_PRICE_ID_EXPORT_PDF
```

Der bestehende Gate ist weiterhin wertvoll, darf aber bis zur zentralen Env-Contract-Implementierung nicht als vollständiger Beweis der Render-Environment-Konfiguration gelten.

5. Render Configuration Recovery

Bei einer fehlerhaften Dashboard-/Blueprint-Konfigurationsänderung:

1. keine Secret-Werte in Tickets/PRs kopieren;
2. vorherigen Setting-Fingerprint/Evidence verwenden;
3. dokumentieren, welche Änderung einen Deploy/Restart ausgelöst hat;
4. Konfiguration zurücksetzen oder via verifiziertem Blueprint korrigieren;
5. neuen Deploy/Health/Smoke-Test abwarten;
6. Secret nur dann auf alten Wert setzen, wenn ausdrücklich bestätigt ist, dass es nicht kompromittiert wurde — bei Credential Incidents stattdessen rotieren/revoken;
7. Audit/ADR/Runbook aktualisieren.

Verwandte Dokumente

- `docs/architecture/ENTERPRISE_FINTECH_ARCHITECTURE_AUDIT.md`
- `docs/architecture/RENDER_PRODUCTION_CONFIGURATION_AUDIT.md`
- `docs/adr/ADR-0037-render-production-configuration-governance.md`
- `docs/runbooks/RENDER_PRODUCTION_EVIDENCE_HANDOFF.md`
- `render.yaml`
- `Dockerfile`
- `scripts/automation/verifyDeploymentReadiness.ts`
- `.github/workflows/ci.yml`
- `docs/DATENSCHUTZ_PROTOKOLL.md`

Provider-Referenzen

- Render Deploys: <https://render.com/docs/deploys>
- Render Rollbacks: <https://render.com/docs/rollbacks>
- Render Health Checks: <https://render.com/docs/health-checks>
- Render Persistent Disks: <https://render.com/docs/disks>

Provider-Verhalten ist bei jedem relevanten Recovery-Prozess erneut gegen den aktuellen Stand zu prüfen.

DEVELOPMENT Chain Phase Execution Runbook

Quelle: docs/runbooks/DEVELOPMENT_CHAIN_PHASE_EXECUTION.md

DEVELOPMENT Chain Phase Execution Runbook

Status: PROPOSED Date: 2026-08-12 Authority: docs/governance/DEVELOPMENT_CHAIN_EXECUTION_POLICY.md

Zweck

Dieses Runbook ist die generische Ausführungssequenz für jeden DEVELOPMENT Chain Roadmap-Punkt. Phasenspezifische Runbooks dürfen diese Regeln verschärfen, aber nicht abschwächen.

0. Eingangsvoraussetzungen

Vor Beginn müssen bekannt sein:

- Roadmap Phase / Item;
- aktueller `main` SHA;
- Authority Refs (Roadmap/ESS/ADR);
- erwartete Zielpfade;
- Checkklasse D/C/R/M;
- Risk Class;
- Mutation Class;
- erwartete Evidence;
- Rollback-/Stop-Bedingungen.

Fehlen sicherheitsrelevante Eingaben, gilt `STOP`.

1. Read-only Preflight

1. aktuellen `main` und relevante Production Baseline auflösen;
2. offene PRs und Changed-File-Overlap prüfen;
3. bestehende ADR/ESS/Runbooks/Evidence lesen;
4. Drift zwischen Roadmap, Code, Runtime und Plattformstatus erfassen;
5. Scope/Risk/Checkklasse bestimmen;
6. erforderliche positive und negative Tests definieren;
7. externe Mutation als `NOT REQUIRED`, `PLANNED` oder bereits separat `HUMAN APPROVED` klassifizieren;
8. Rollback-/Kill-Switch-Verfügbarkeit prüfen.

2. Branch-Erstellung

Nur nach bestandenem Preflight:

```
current main → fresh scoped branch
```

- niemals direkt auf `main` schreiben;
- kein gemergter/supersedeter Branch wird wiederverwendet;
- Clone/Worktree muss vor Edits auf dem neuen Branch stehen.

3. Repository Implementation

Implementiere nur den genehmigten Scope.

Pflichten:

- least privilege;
- fail-closed Security Gates;
- keine Fake-/Mock-Daten in produktiven Pfaden;
- keine Secrets in Code/Evidence;
- negative Tests bei Auth/IAM/Mutation/Trust-Boundary-Änderungen;
- keine opportunistische Plattformmutation aus einer Codeaufgabe;
- Dev-Plane dokumentiert erforderliche Stripe/Supabase/Render-Handoffs statt sie direkt auszuführen.

4. Pre-PR Verification

Vor PR-Erstellung:

1. Diff gegen `main` prüfen;
2. offene PRs erneut auf Overlap prüfen;
3. Checkklasse ggf. hochstufen;
4. relevante Tests ausführen, ohne redundante teure CI-Läufe zu erzeugen;
5. Scope, Risiken, Rollback und Mutation State dokumentieren.

PR-Erstellung folgt der jeweils geltenden Human-/REM-Policy.

5. Human Review / CI

Für den aktuellen Head:

- Human File Review;
- alle erforderlichen Owner-Gates;
- genau die Checks der strengsten Checkklasse;
- keine Interpretation von grünem CI als Merge-Autorität.

Ein neuer Commit invalidiert Head-gebundene Review-/Approval-Evidence.

6. Human Merge / Branch Cleanup

Nach separater Human Merge-Anweisung:

1. Merge ausführen;
2. Merge SHA erfassen;
3. erforderliche Post-Merge Repository-/Runtime-Checks durchführen;
4. Remote-Branch im Finance Repository löschen;
5. lokalen Branch / ephemeren Worktree/Clone nach Evidence-Sicherung bereinigen.

Wenn keine externe Mutation erforderlich ist, direkt zu Schritt 10.

7. External Pre-Mutation Check

Vor jeder externen Mutation:

- exakten Account/Project/Service/Environment bestätigen;
- aktuellen Zustand und Drift erfassen;
- Last-Known-Good / Backup / Rollback prüfen;
- konkurrierende Writer ausschließen;
- geplante Operationen gegen Roadmap/ADR/Runbook prüfen;
- Handoff Contract vorbereiten;
- Secrets nur über den autorisierten Credential Host referenzieren, niemals in Handoff/Evidence kopieren.

Precheck **FAILED** oder **INCONCLUSIVE** → **STOP** .

8. Explicit Human Mutation Approval

Externe Mutation startet erst nach einer separaten, scope-/target-gebundenen Human/Owner-Freigabe, wenn die Policy sie verlangt.

Die Approval Evidence muss an mindestens Roadmap Item, Target, Operation, Risk und Gültigkeitsfenster gebunden sein.

Ein Roadmap-/Dokumentations-Merge allein ist keine Produktionsmutation Approval.

9. Authorized Mutation Execution

Bevorzugter autonomer Pfad nach dessen eigener Verifikation:

```
validated Handoff
→ REM / IAM / reserved-action gate
→ durable authorization audit
→ exact execution-host permit
→ exact side effect
→ durable outcome audit
→ post-verification
```

Direkte Connector-/Tool-Mutation ohne enforcebaren Permit-before-side-effect-Pfad darf nicht als autonom **VERIFIED PASS** klassifiziert werden.

Bei Fehler:

- keine Folgeoperationen;
- terminales ERROR Evidence;
- Rollback nach Runbook;
- Post-Rollback Verification.

10. Evidence Closure

Nutze `docs/evidence/templates/DEVELOPMENT_CHAIN_PHASE_EVIDENCE_TEMPLATE.md`.

Mindestens dokumentieren:

- Baseline;
- Authority;
- Branch/PR/Head/Merge;
- Checkklasse;
- Tests;
- Mutation State;
- Approval/Audit References;
- Pre/Post/Negative Checks;
- Rollback State;
- Branch deleted;
- finalen Exit Gate.

11. Roadmap Synchronization

Nach erfolgreicher Verifikation synchronisieren:

1. `docs/architecture/ROADMAP.md` ;
2. `docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md` ;
3. `docs/roadmaps/AI_AGENT_M0_M9_IMPLEMENTATION_ROADMAP.md` ;
4. `docs/traceability/AI_AGENT_M0_M9_TRACEABILITY_MATRIX.md` ;
5. `docs/traceability/DEVELOPMENT_CHAIN_DOCUMENT_TRACEABILITY_MATRIX.md` ;
6. betroffene ESS/ADR;
7. Evidence.

12. Next Gate

Folgephase nur freigeben, wenn alle **REQUIRED** Tests/Mutationen **VERIFIED PASS** sind.

Bei **FAILED**, **INCONCLUSIVE**, fehlender Evidence oder offenem Rollback bleibt die Folgephase blockiert.

FRED Macro/Rate Evidence — Production Runbook

Quelle: docs/runbooks/FRED_MACRO_EVIDENCE_RUNBOOK.md

FRED Macro/Rate Evidence — Production Runbook

Scope: CAPITAL-AI server-side macro/rate evidence

Related: ADR-0022, ADR-0023, `src/services/macroRateEvidence.ts`, `src/services/macroRiskRegime.ts`

1. Runtime configuration

The production runtime expects the server-side environment variable:

```
FRED_API_KEY
```

The value must never be committed to the repository, exposed through a `VITE_` variable, returned by an API response, logged, or copied into provenance/evidence records.

The application repository only defines the environment-variable contract. Secret creation and rotation remain an external production operation.

2. Approved FRED series

Only reviewed series in `APPROVED_FRED_SERIES` may be requested through CAPITAL-AI:

- `DGS2` — U.S. Treasury 2-Year Constant Maturity
- `DGS10` — U.S. Treasury 10-Year Constant Maturity
- `FEDFUNDS` — Federal Funds Effective Rate
- `CPIAUCSL` — Consumer Price Index for All Urban Consumers

Arbitrary client-supplied FRED series IDs are not permitted.

3. Verification endpoints

After deployment, validate the server-side evidence boundary with:

```
GET /api/registry/macro/fred/DGS10
GET /api/registry/macro/fred/DGS2
GET /api/registry/macro/risk-regime
```

Expected successful FRED evidence response characteristics:

- HTTP 200
- `status: READY`
- `provider: FRED`

- `executionPriceEligible: false`
- non-empty `evidenceIds`
- secret-free `sourcePath`
- `x-correlation-id` response header

The risk-regime endpoint is successful only when DGS2 and DGS10 contain a common observation date. It must never manufacture a curve regime from mismatched dates.

4. Fail-closed behavior

The following conditions must not produce default macro data:

- missing/invalid `FRED_API_KEY`
- FRED HTTP failure
- timeout
- malformed observations
- no numeric observations
- DGS2/DGS10 without a common observation day

Expected states include `SOURCE_UNAVAILABLE` or `EVIDENCE_INCOMPLETE`. No asset score or execution price is generated by the macro evidence layer.

5. Runtime observability

Every FRED call records:

- Provider Health through `recordProviderHealth()`
- routing success/failure and latency through `recordMarketDataProviderOutcome()`
- correlation ID at the API boundary
- stable Evidence IDs for accepted observations

These observations feed the existing Supervisor and `RUNTIME-01` compliance evidence path. A provider that was never called must not be represented as healthy by assumption.

6. Security verification

Before release, confirm:

1. repository search contains no real FRED key value;
2. `.env.example` contains only `FRED_API_KEY=`;
3. browser bundles do not contain `FRED_API_KEY` or a `VITE_FRED_*` variable;
4. API responses do not contain `api_key=`;
5. logs do not print the full outbound FRED URL containing the secret;
6. CI vulnerability, TypeScript, Vitest, build and deployment-readiness gates are green.

7. Bond / cross-asset boundary

FRED evidence may provide rate-curve or macro context, but it does not by itself authorize Bond Scoring. ADR-0022 remains the activation gate for Fixed-Income scoring.

The versioned Macro Risk Regime contract is evidence/context only and must not be presented as an investment recommendation or tradable market price.

8. UI / graphics

This runbook and the associated implementation do not require or authorize graphical changes.

CAPITAL-AI Runbook: GA4 Analytics MCP Server

Quelle: docs/runbooks/GA4_MCP_SERVER_SETUP.md

CAPITAL-AI Runbook: GA4 Analytics MCP Server

Document ID

RUNBOOK-0004

Bezug

- `.mcp.json` (Server-Definition), `.claude/hooks/ga4-mcp-credentials.sh`, `.claude/settings.json`
- `.ai/skills/ESS-0014-Google-Marketing-MCP-Governance.md` (normativ)
- `docs/architecture/CAPITAL_AI_GOOGLE_MARKETING_MCP_TOPOLOGY.md` (Ebene ④ der Topologie)
- `docs/runbooks/GOOGLE_ANALYTICS_SETUP.md` (Tag-/Consent-Seite, davon unabhängig)

Status

Aktiv. Der Server ist im Repository konfiguriert; die Inbetriebnahme erfordert einmalige, manuelle Schritte in der Google Cloud Console und in Google Analytics, die nur der Repository-Owner ausführen kann (Abschnitt 2).

Zweck

Lesender Zugriff auf echte GA4-Berichtsdaten (Besucherzahlen, Ereignisse, Realtime) direkt aus Claude-Code-Sitzungen — statt manueller Ablesung im GA4-Dashboard.

Abgrenzung. Ausschließlich lesend. ESS-0014 behandelt den offiziellen Google-Analytics-MCP als Read-/Evidence-Plane; Schreibzugriffe dürfen niemals am MCP hängen, sondern nur über die vollständige Gate-Kette des Write-Gateways erfolgen (heute nicht implementiert).

1. Gewählte Implementierung

Paket	<code>analytics-mcp</code> (PyPI), Version 0.7.0 zum Prüfstich-tag
Herkunft	offiziell , GitHub-Organisation <code>googleanalytics</code> — https://github.com/googleanalytics/google-analytics-mcp
Laufzeit	Python ≥ 3.10, gestartet über <code>uvx</code>

Authentifizierung	Application Default Credentials über <code>GOGLE_APPLICATION_CREDENTIALS</code>
Property-Auswahl	pro Werkzeugaufruf , nicht über eine Umgebungsvariable

Warum der offizielle Server. ESS-0014 benennt ausdrücklich den „Official Google Analytics MCP“ als Read-Plane. Zuvor war hier ein npm-Paket `google-analytics-mcp` konfiguriert, das auf der npm-Registry **nicht existiert** — der Server hätte nie starten können. Geprüfte Alternativen: `ruchernchong/mcp-server-google-analytics` (npm, seit Oktober 2025 archiviert/unmaintained) und `surendranb/google-analytics-mcp` (Python, Community). Beide sind gegenüber dem offiziellen Paket nachrangig.

Warum `uvx` statt `pipx`. Die Projekt-Dokumentation von Google zeigt `pipx run analytics-mcp`. In der hier verwendeten Claude-Code-Umgebung ist `pipx` nicht installiert, `uv` / `uvx` dagegen schon. `uvx analytics-mcp` ist das direkte Äquivalent und wurde in dieser Umgebung erfolgreich gestartet. Wer lokal mit `pipx` arbeitet, kann in `.mcp.json` auf `"command": "pipx", "args": ["run", "analytics-mcp"]` wechseln — funktional identisch.

2. Einmalige Einrichtung durch den Repository-Owner

Diese Schritte erfordern Zugriff auf das Google-Cloud- und das Google-Analytics-Konto und können nicht automatisiert werden.

2.1 Google Cloud

1. console.cloud.google.com → Projekt wählen oder anlegen. Die **Projekt-ID** notieren (nicht den Anzeigenamen).
2. **APIs & Services** → **Library** → beide APIs aktivieren: - **Google Analytics Data API** - **Google Analytics Admin API**
3. **IAM & Admin** → **Service Accounts** → **Create Service Account**, z. B. `ga4-mcp-reader`. **Keine** Projekt-IAM-Rolle vergeben — der Zugriff wird in Schritt 2.2 direkt auf der GA4-Property erteilt (Least Privilege).
4. Auf dem Service Account → **Keys** → **Add Key** → **Create new key** → **JSON** → Datei herunterladen. Diese Datei ist ein Geheimnis: **niemals committen, niemals in einen Chat einfügen**.

2.2 Google Analytics

1. GA4 → **Verwaltung** → **Property** → **Property-Zugriffsverwaltung** → **Nutzer hinzufügen**: - E-Mail: die `client_email` aus der JSON-Datei (endet auf `.iam.gserviceaccount.com`) - Rolle: **Betrachter / Viewer** — mehr wird nicht benötigt

2.3 Claude-Code-Umgebung

1. In den Umgebungs-/Secret-Einstellungen der Claude-Code-Umgebung setzen (**nicht** in `.env`, **nicht** in Render — das hier ist Agenten-Tooling, nicht die deployte Anwendung):

Variable	Wert
GA4_MCP_SERVICE_ACCOUNT_KEY_JSON	vollständiger Inhalt der JSON-Schlüsseldatei
GA4_MCP_PROJECT_ID	Google-Cloud-Projekt-ID aus Schritt 1

1. **Neue** Claude-Code-Sitzung starten. Bestehende Sitzungen lesen weder `.mcp.json`-Änderungen noch neue Umgebungsvariablen nach.

3. Funktionsweise

```

GA4_MCP_SERVICE_ACCOUNT_KEY_JSON (Secret der Umgebung, String)
    ↓
  SessionStart-Hook (.claude/hooks/ga4-mcp-credentials.sh)
~/.capital-ai/ga4-mcp-credentials.json (0600, Verzeichnis 0700, außerhalb des Repos)
    ↓
  GOOGLE_APPLICATION_CREDENTIALS (fester Pfad in .mcp.json)
uvx analytics-mcp → Google Analytics Data/Admin API (nur lesend)

```

Der Hook existiert, weil der MCP-Server einen **Dateipfad** erwartet, das Secret aber als **String** in der Umgebung ankommt.

Der Pfad ist in Hook und `.mcp.json` fest verdrahtet statt über eine Variable geführt: MCP-Server werden unabhängig vom Hook gestartet, eine per Hook exportierte Variable könnte zu spät kommen. Eine Datei an einem bekannten Ort hat dieses Timing-Problem nicht.

Fail-closed. Fehlt das Secret, schreibt der Hook nichts und beendet sich mit Code 0; der MCP-Server startet, kann sich aber nicht authentifizieren und meldet einen Auth-Fehler — statt stillschweigend leere oder falsche Zahlen zu liefern. Ist das Secret gesetzt, aber kein gültiges JSON oder keine Google-Credentials-Struktur, bricht der Hook mit Code 1 ab und schreibt **keine** Datei.

4. Verifikation

Nach dem Start einer neuen Sitzung:

1. **Hook gelaufen?** Erwartete Ausgabe: `[ga4-mcp] Anmelddaten bereitgestellt unter .../ga4-mcp-credentials.json (0600).`
2. **Rechte korrekt?** `bash ls -ld ~/.capital-ai && ls -l ~/.capital-ai/ga4-mcp-credentials.json` # erwartet: `drwx-----` und `-rw-----`
3. **Server erreichbar?** Die Werkzeuge des Servers `ga4-analytics` müssen in der Sitzung verfügbar sein. Ein Aufruf, der die Kontenübersicht liest, muss die erwartete GA4-Property zurückgeben.
4. **Least Privilege belegt?** Ein Schreibversuch muss fehlschlagen — der Server bietet keine Schreibwerkzeuge, und die Rolle ist Viewer.

Manueller Servertest ohne Anmelddaten (prüft nur, dass Paket und Laufzeit auflösbar sind):

```
uvx analytics-mcp --help
# erwartet u. a.: "Starting MCP Stdio Server: Google Analytics MCP Server"
```

5. Sicherheitshinweise

- Die Schlüsseldatei liegt unter `~/.capital-ai/` — **außerhalb** des Arbeitsbaums. Sie kann daher nicht versehentlich committed werden und taucht in `git status` nicht auf.
- Der Hook gibt zu keinem Zeitpunkt Schlüsselinhalte aus; Fehlermeldungen enthalten ausschließlich Metadaten (Fehlertyp, fehlendes Feld).
- Der Service Account erhält **keine** Projekt-IAM-Rolle, sondern ausschließlich Viewer-Rechte auf der einzelnen GA4-Property.
- Schlüsselrotation: neuen Key erzeugen, `GA4_MCP_SERVICE_ACCOUNT_KEY_JSON` ersetzen, alten Key in der Cloud Console löschen, neue Sitzung starten. Die alte Datei wird beim nächsten Hook-Lauf überschrieben.

6. Fehlerdiagnose

Symptom	Ursache	Abhilfe
Hook meldet „nicht gesetzt“	<code>GA4_MCP_SERVICE_ACCOUNT_KEY_JSON</code> fehlt in der Umgebung	Schritt 2.3, danach neue Sitzung
Hook bricht mit „kein gueltiges JSON“ ab	Secret abgeschnitten oder mit Zeilenumbrüchen zerstört	Dateiinhalt vollständig und unverändert einfügen
Server startet, jede Abfrage liefert 403	Service Account nicht auf der Property berechtigt	Schritt 2.2
Server startet, Abfragen melden fehlende API	Data- oder Admin-API nicht aktiviert	Schritt 2.1
<code>uvx: command not found</code>	<code>uv</code> fehlt in der Umgebung	<code>uv</code> installieren oder in <code>.mcp.json</code> auf <code>pipx run</code> wechseln
Werkzeuge fehlen trotz korrektem Setup	Sitzung liest <code>.mcp.json</code> nicht nach	neue Sitzung starten

GitGuardian-/Snyk-App-Integration — Owner-Runbook

Quelle: docs/runbooks/GITGUARDIAN_SNYK_APP_INTEGRATION.md

GitGuardian-/Snyk-App-Integration — Owner-Runbook

Status: PRE-MUTATION / OWNER ACTION REQUIRED

Datum: 2026-08-14

ADR: docs/adr/ADR-0070-gitguardian-snyk-external-app-integration.md

Zielzustand

- GitGuardian überwacht ausschließlich `SvenKulesa/Finance`.
- Snyk importiert ausschließlich `SvenKulesa/Finance`.
- Beide Anbieter arbeiten in Stufe 1 ohne Repository-Schreibrechte.
- Keine neuen GitHub-Actions-Workflows.
- Keine Required-Check-Promotion in diesem Arbeitsschritt.

Vorprüfung

- [x] PR #248 ist gemerged.
- [x] `main`-Baseline ist `5e8471de10644a5432017d28d2f4ff0657d92dd5`.
- [x] Keine offenen PRs zum Prüfzeitpunkt.
- [x] GitGuardian-App ist laut Owner bereits installiert.
- [x] Snyk-Token ist laut Owner bereits gesetzt; Wert wurde nicht gelesen.
- [] Exakte GitGuardian-App-Berechtigungen im GitHub-UI geprüft.
- [] Snyk-GitHub-App/Repository-Import im Snyk-UI geprüft.

GitGuardian prüfen

1. GitHub → Settings → Applications → Installed GitHub Apps → GitGuardian.
2. Repository access auf **Only select repositories** begrenzen.
3. Ausschließlich `SvenKulesa/Finance` auswählen.
4. Read-only Zugriff beibehalten.
5. Zusätzliche GitGuardian-Write-App/Honeytoken-Schreibrechte nicht aktivieren.
6. Historischen Scan und PR-Scanning im GitGuardian-Dashboard prüfen.
7. Keine Finding-Inhalte mit Secrets in PR-Kommentare kopieren.

Snyk anbinden

1. Im Snyk-Dashboard GitHub als Source öffnen.

2. `SvenKulesa/Finance` als einziges Ziel importieren.
3. Open-Source-/Dependency-Scanning aktivieren.
4. automatische Fix-PRs, Merge-Automation und Contents-Write deaktiviert lassen.
5. vorhandenen Token nur im Snyk-/GitHub-Secret-Speicher belassen.
6. keine Workflowdatei erzeugen und keinen Tokenwert anzeigen.
7. initialen Scan starten und Projekt/Commit-SHA notieren.

Verifikation

Für zwei unterschiedliche PR-Heads dokumentieren:

Feld	GitGuardian	Snyk
Repository	<code>SvenKulesa/Finance</code>	<code>SvenKulesa/Finance</code>
PR	auszufüllen	auszufüllen
Head-SHA	auszufüllen	auszufüllen
Checkname	auszufüllen	auszufüllen
Ergebnis	auszufüllen	auszufüllen
Zeitpunkt UTC	auszufüllen	auszufüllen
Anbieter-Link	redigierter Dashboard-/Check-Link	redigierter Dashboard-/Check-Link

Akzeptanz:

- Check gehört zum exakten PR-Head.
- Kein Secret erscheint in Log, Kommentar oder Evidence.
- Anbieterfehler beeinflusst `capital-ai-ci` in Stufe 1 nicht.
- Zwei aufeinanderfolgende PR-Heads liefern verwertbare Ergebnisse.

Promotion

Erst nach separater Owner-Freigabe:

1. exakte stabile Checknamen ermitteln;
2. Ruleset-Baseline exportieren;
3. Rollback verifizieren;
4. Checks einzeln im Shadow-Modus beobachten;
5. Required-Check-Mutation separat durchführen und protokollieren.

Rollback

- Repository im Anbieter deaktivieren.
- GitHub-App-Repositoryzugriff entziehen.
- späteren Required Check zuerst aus Ruleset entfernen.

- Token anbieterseitig widerrufen/rotieren, falls Kompromittierung vermutet wird.
- keine Änderung am kanonischen `capital-ai-ci` -Check.

CAPITAL-AI Runbook: Google Analytics, AdSense und CookieHub

Quelle: docs/runbooks/GOOGLE_ANALYTICS_SETUP.md

CAPITAL-AI Runbook: Google Analytics, AdSense und CookieHub

Document ID

RUNBOOK-0003

Status

Aktiv — Repository-Implementierung gemäß **ESS-0014** und **ADR-0042**.

CAPITAL-AI verwendet **Google Consent Mode v2 im Basic Mode**:

- CookieHub ist Consent Source of Truth;
- ohne Opt-in wird kein Google-Tag geladen und kein Google-Netzwerkrequest ausgelöst;
- GA4 wird nur durch die Kategorie `analytics` aktiviert;
- AdSense wird nur durch die Kategorie `marketing` aktiviert;
- alle Consent-Mode-v2-Felder starten fail-closed mit `denied`;
- ein Widerruf setzt die Signale zurück, deaktiviert GA und lädt die Seite neu.

Die frühere Runbook-Aussage, Consent Mode v2 solle nicht verwendet werden, ist durch ESS-0014 und ADR-0042 ersetzt. Advanced Consent Mode mit cookieless pings bleibt ausdrücklich ausgeschlossen.

Bezug

- `.ai/skills/ESS-0014-Google-Marketing-MCP-Governance.md`
- `docs/adr/ADR-0035-protected-google-marketing-integration-strict-csp.md`
- `docs/adr/ADR-0040-csp-runtime-remediation-safe-rollout.md`
- `docs/adr/ADR-0042-basic-consent-mode-v2-google-tag-gating.md`
- `docs/DATENSCHUTZ_PROTOKOLL.md`

1. Architektur

1.1 HTML Shell

`index.html` enthält nur:

- CookieHub Production SDK;

- inerte öffentliche IDs als Meta-Tags;
- den First-Party-Consent-Bridge;
- den CookieHub-Initializer.

Es gibt **keinen** direkt ausführbaren GA4- oder AdSense-Third-Party-Loader im HTML.

```
<script src="https://cdn.cookiehub.eu/c2/75f66920.js"></script>
<meta name="ga-measurement-id" content="%VITE_GA_MEASUREMENT_ID%" />
<meta name="adsense-publisher-id" content="ca-pub-1353017943074018" />
<script src="/google-analytics-consent.js"></script>
<script src="/cookiehub-init.js"></script>
```

Alle produktiven Script-Tags erhalten serverseitig denselben Request-Nonce gemäß ADR-0035/ ADR-0040.

1.2 First-Party Bridge

public/google-analytics-consent.js :

1. erzeugt lokal `dataLayer` / `gtag` ;
2. setzt Consent Mode v2 auf `denied`, ohne ein Google-Script zu laden;
3. registriert CookieHub-Events auf `document` ;
4. liest `analytics` und `marketing` aus CookieHub;
5. injiziert die jeweiligen Google-Tags erst nach Opt-in;
6. verhindert Mehrfachinjektion;
7. behandelt Fehler fail-closed;
8. löscht bei Analytics-Widerruf erreichbare `_ga` -/ `_ga_*` -Cookies;
9. lädt nach einem Widerruf neu, weil ausgeführte Third-Party-Skripte nicht sicher entladen werden können.

1.3 Consent Mapping

CookieHub-Kategorie	Google-Signal / Dienst	Verhalten
necessary	security_storage=granted	Immer aktiv, nur Sicherheits-/ Consent-Funktion
analytics	analytics_storage + GA4	GA4 erst nach Opt-in
marketing	ad_storage, ad_user_data, ad_personalization + AdSense	AdSense erst nach Opt-in
preferences	derzeit nicht genutzt	Bleibt denied

`allow_google_signals` und `allow_ad_personalization_signals` bleiben in der GA4-Konfiguration bewusst deaktiviert. AdSense wird getrennt über `marketing` gesteuert.

2. GA4 Property und Measurement-ID

1. Google Analytics öffnen und die Produktions-Property auswählen.

2. Web-Datenstream für `https://capital-ai.online` prüfen.
3. Measurement-ID im Format `G-XXXXXXXXXX` kopieren.
4. Erweiterte Messung nur für tatsächlich benötigte Events aktivieren.
5. Datenaufbewahrung auf den fachlich/rechtlich erforderlichen Mindestzeitraum reduzieren.
6. Google-Signals, Ads-Verknüpfungen und Remarketing nur nach eigener Freigabe aktivieren.

Die Measurement-ID ist öffentlich, aber ihre Zuordnung zur korrekten Property muss kontrolliert werden.

3. Build-Time Environment

Vite ersetzt `VITE_*`-Variablen zur Build-Zeit.

```
VITE_GA_MEASUREMENT_ID=G-XXXXXXXXXX
```

Entwicklung

Wert in `.env` setzen. `.env` darf nicht committed werden.

Produktion / Render

1. Environment Variable in Render setzen oder prüfen;
2. neuen Build/Deploy auslösen;
3. im ausgelieferten HTML prüfen, dass der Meta-Tag eine gültige `G-...`-ID enthält.

Fehlt die Variable oder bleibt `%VITE_GA_MEASUREMENT_ID%` unverarbeitet, lädt GA4 fail-closed nicht.

4. CookieHub Production Configuration

Repository-Code kann das externe CookieHub-Dashboard nicht verändern. Der Repository-Owner prüft:

1. Domain-Code `75f66920` gehört zu `capital-ai.online`;
2. Production-Domain ist verifiziert;
3. Kategorie-IDs heißen exakt `analytics` und `marketing`;
4. Google Analytics 4 ist `analytics` zugeordnet;
5. Google AdSense ist `marketing` zugeordnet;
6. Ablehnen ist auf derselben Ebene und ebenso leicht erreichbar wie Akzeptieren;
7. Einwilligungen werden mit Zeitstempel/Version protokolliert;
8. Preference Center ist dauerhaft erreichbar;
9. Deutsch ist aktiviert;

10. die aktuellen Google-CMP-/IAB-TCF-Anforderungen für AdSense in EWR/UK/CH sind erfüllt.

Keine doppelte Consent-Implementierung

Der First-Party-Bridge sendet die Consent-Mode-v2-Signale. Während dieser aktiv ist:

- keine zusätzliche manuelle `gtag('consent', ...)`-Implementierung;
- kein zweiter Cookie-Banner;
- kein paralleler GTM-Consent-Tag;
- CookieHub-GCM-Automatik nur aktivieren, wenn die First-Party-Bridge in einem eigenen ADR ersetzt und die doppelte Signalgebung ausgeschlossen wurde.

5. Verifikation

Tests immer in einem neuen privaten Browserprofil oder nach Löschen der Site-Daten durchführen.

5.1 Vor jeder Einwilligung

Im Network-Tab darf kein Request an folgende Ziele erscheinen:

```
www.googletagmanager.com  
google-analytics.com  
pagead2.googlesyndication.com  
doubleclick.net
```

Zusätzlich:

- keine `_ga` -/ `_ga_*` -Cookies;
- `dataLayer` enthält Consent Default `denied`;
- CookieHub-Banner/Preference Center ist sichtbar oder erreichbar.

5.2 Nur Analytics akzeptiert

- genau ein `gtag/js?id=G-...`-Request;
- GA4 Realtime zeigt den Testbesuch;
- kein AdSense-Loader;
- `analytics_storage=granted`;
- alle Ad-Signale bleiben `denied`.

5.3 Marketing akzeptiert

- AdSense-Loader wird genau einmal geladen;
- `ad_storage`, `ad_user_data`, `ad_personalization` sind `granted`;
- keine doppelte Script-Injektion nach erneutem Speichern der Einstellungen.

5.4 Widerruf

1. Datenschutzseite öffnen;
 2. `Cookie-Einstellungen ändern` wählen;
 3. Analytics/Marketing widerrufen;
 4. nach Reload prüfen, dass keine Google-Tags mehr im neuen Dokument geladen werden;
 5. `_ga` -/ `_ga_*` -Cookies prüfen.
-

6. Repository-Gates

```
npx tsx scripts/security/verifyGoogleMarketingInvariants.ts
npx vitest run tests/unit/securityResponse.test.ts tests/unit/googleMarketingConsent.test.ts
npm run lint
npm run build
NODE_ENV=production CSP_MODE=report-only npx vitest run tests/unit/securityResponse.production.test.ts
```

Der Protected Marketing Workflow führt diese Prüfungen bei relevanten Änderungen aus.

7. Webscan-Nachprüfung

Nach Merge und produktivem Deploy:

1. Cloudflare-/Browser-Cache leeren, falls erforderlich;
2. Scan ohne vorhandene CookieHub-Einwilligung starten;
3. Findings `Kein Cookie-Consent-Tool` und `Google Analytics ohne Consent-Gate` erneut prüfen;
4. Browser-Network-Evidence zusätzlich dokumentieren;
5. falls der Scanner trotz nachgewiesener Netzwerksperre nur Quelltext-Heuristiken nutzt, den Befund als Scanner-False-Positive dokumentieren — nicht die Implementierung verschleiern.

Ein Score von 100 ist ein Sekundärziel. Primär gilt das reale Netzwerk- und Consent-Verhalten.

8. Troubleshooting

CookieHub-Banner erscheint nicht

- Domain-Code und Produktionsdomain im CookieHub-Dashboard prüfen;
- Browser-Konsole auf CSP-/Hostname-Fehler prüfen;
- `window.cookiehub.isReady()` prüfen;
- sicherstellen, dass CookieHub nur einmal geladen wird;
- Cloudflare-/Browser-Cache leeren.

GA4 lädt trotz Opt-in nicht

- `VITE_GA_MEASUREMENT_ID` wurde beim Build gesetzt;
- ID entspricht `G-[A-Z0-9]+`;
- CookieHub-Kategorie heißt exakt `analytics`;
- `cookiehub_onStatusChange` wird auf `document` ausgelöst;
- CSP blockiert `www.googletagmanager.com` nicht.

Google-Request vor Opt-in

- nach weiteren GA/GTM/AdSense-Snippets im Repository suchen;
- Browser-Erweiterungen ausschließen;
- CookieHub-/GTM-Doppelinstallation ausschließen;
- ausgeliefertes Produktions-HTML statt nur den Main-Branch prüfen;
- Deploy-Commit gegen `/healthz` /Deployment Identity abgleichen.

9. Offene externe Schritte

Schritt	Verantwortlich	Status
CookieHub-Domain/Category Mapping prüfen	Repository-Owner	Production Handoff
Google-CMP-/TCF-Anforderungen für AdSense prüfen	Repository-Owner / Compliance	Production Handoff
Render Build mit gültiger Measurement-ID	Repository-Owner	Nach Merge
Browser-Network-Evidence erstellen	Reviewer / Owner	Nach Deploy
Webscan wiederholen	Reviewer / Owner	Nach Deploy

M10 — Passkey-only Human/Owner PR Authorization Runbook

Quelle: docs/runbooks/M10_PASSKEY_OWNER_PR_AUTHORIZATION.md

M10 — Passkey-only Human/Owner PR Authorization Runbook

Status: PLANNED — EXECUTION BLOCKED BY M9

Date: 2026-08-12

Updated: 2026-08-16

Authority: ADR-0066, ESS-0022, M10 Threat Model, DEVELOPMENT Chain Execution Policy, HUMAN_OWNER_PR_APPROVAL_POLICY

Goal

Introduce CAPITAL-AI WebAuthn/passkey transaction bound to the exact current PR state as the **strong authorization** for expensive CI consumption (action `AUTHORIZE_PR_CI`) and related privileged gates.

Owner-Entscheidung 2026-08-16: Die transitionalen Mechanismen Current-Head-Review `👍` / `okay` und Owner-Body-Checkboxen sind **bereits retired** (vereinfachter Merge-Prozess). M10 implementiert die **nächste** starke Autorisierungsschicht, nicht die parallele Abschaffung einer noch aktiven Checkbox-Zeremonie.

Human-only **Merge** bleibt verpflichtend (Agenten mergen nicht).

Prerequisite Gate

M10 implementation starts only when M9 is `COMPLETE / VERIFIED PASS`.

Before any cutover:

- M5 append-only audit remains healthy;
- M9 recovery/break-glass assurance is proven;
- simplified pre-M10 CI path (no checkbox/emoji gate) is documented and stable;
- rollback plan is accepted;
- exact RP ID / HTTPS origin architecture is approved;
- Owner credential enrollment and recovery procedure is defined.

Phase 1 — Trusted PR State Resolver

Implement server-side resolution of:

- repository `SvenKulessa/Finance` ;
- PR number;

- base branch/SHA;
- exact current head SHA;
- canonical sorted changed-file set;
- canonical diff/review digest;
- action `AUTHORIZE_PR_CI` .

Do not accept agent-supplied hashes as authoritative. The trusted service recomputes context from GitHub state.

Phase 2 — Challenge Issuance

Server creates a cryptographically random, short-lived, single-use challenge bound to the exact authorization context.

Persist challenge metadata sufficient for issue time, expiry, expected Owner, expected PR context digest, unused/consumed/revoked state.

Never store authenticator private key or biometric material.

Phase 3 — Owner Credential Enrollment

Owner passkey enrollment is a Human/Owner identity operation (strong session, expected RP/origin, UV required, public credential only, revocation + recovery proven before cutover).

Agents may assist with UI/code but cannot autonomously enroll, replace or revoke Owner credentials.

Phase 4 — Assertion Verification

On approval: re-resolve PR state; reject drift; verify challenge, RP ID, origin, credential, signature, UP/UV; persist immutable approval Evidence; mark single-use for CI consumption. Any failure → `DENY` .

Phase 5 — Atomic CI Consumption

Before expensive CI: re-resolve state; compare to approved context; atomically consume approval; persist consumption Evidence; issue exactly one CI request for the approved head. Duplicate → `DENY/DEDUPE` .

Phase 6 — Shadow Mode

Before cutover, operate M10 verifier in shadow mode while the **simplified** (no checkbox/emoji) path remains authoritative for starting CI.

Shadow M10 must not reintroduce weaker emoji/checkbox authorization.

Mandatory Negative Tests

Include wrong Owner, wrong RP/origin, UP/UV false, invalid signature, expired/replayed challenge, revoked credential, wrong repo/PR, changed base/head/file-set/diff, unavailable resolver/verifier/audit, already consumed approval, forged legacy `checkbox`/checkbox/label/reaction (must not authorize), agent self-approval, weak recovery fallback.

Controlled Cutover

Only after shadow + negative + recovery evidence `VERIFIED PASS` :

1. enable passkey approval as authoritative CI gate for `AUTHORIZE_PR_CI` ;
2. verify exactly one approved current-head CI run;
3. verify unapproved attempts do not start expensive CI;
4. keep Human Merge separate;
5. ensure no residual checkbox/emoji authorization path exists in CI;
6. update PR template, governance and Roadmap if needed;
7. repeat post-cutover negative tests.

Legacy Cleanup

Checkbox- and emoji-based CI authorization was retired on 2026-08-16. After M10 cutover:

- ensure no workflow reintroduces those parsers;
- preserve historical Evidence references only;
- retain Human merge boundary.

Rollback

Before M10 enforcement

Keep simplified CI path (no checkbox/emoji gate); do not silently restore weaker emoji/checkbox authorization.

After M10 enforcement

Do not silently restore emoji/checkbox authorization. Activate pre-approved incident/recovery path and restore a verified safe gate through a controlled change.

Exit Gate

M10 is `COMPLETE` / `VERIFIED PASS` only when:

1. M9 prerequisite PASS;
2. PR-bound WebAuthn Owner assertion is the enforced normal CI authorization;
3. all negative/replay/freshness/revocation tests PASS;
4. recovery PASS;

5. immutable audit PASS;
6. one expensive CI per approved head proven;
7. no checkbox/emoji authorization path remains;
8. Human-only Merge preserved;
9. Evidence/Roadmap/Traceability synchronized;
10. implementation work branch deleted after Human merge.

M5 Supabase Agent Audit Mutation Runbook

Quelle: docs/runbooks/M5_SUPABASE_AGENT_AUDIT_MUTATION.md

M5 Supabase Agent Audit Mutation Runbook

Status: PRE-MUTATION / HUMAN APPROVAL REQUIRED Date: 2026-08-12 Target: Supabase project AIFINANCIAL (ryzywoktpmyhwzxmstyu)

Scope

Diese Mutation darf ausschließlich die M5-Audit-Persistenz für ADR-0059 einführen. Keine Stripe- oder Render-Mutation. Keine Änderung produktiver Billing-Logik.

Zielzustand

Eine dedizierte append-only Tabelle `public.agent_audit_events` speichert redigierte Agent-/Command-Audit-Evidence. Operational Telemetry verbleibt in `src/platform/Telemetry`.

Mindestfelder: - `id` uuid - `occurred_at` timestampz - `request_id` text - `trace_id` text - `span_id` text - `human_actor_id` uuid nullable - `app_id` text - `agent_id` text - `provider` text nullable - `model` text nullable - `intent` text - `scope` jsonb - `capability` text - `risk_class` text - `policy_id` text nullable - `policy_version` text nullable - `authorization_decision` text - `approval_reference` uuid nullable - `step_up_reference` uuid nullable - `tool_name` text nullable - `repository` text nullable - `branch` text nullable - `commit_sha` text nullable - `pull_request_number` bigint nullable - `ci_run_id` text nullable - `artifact_digest` text nullable - `deployment_id` text nullable - `runtime_version` text nullable - `result` text - `error_code` text nullable - `rollback_reference` text nullable - `attributes` jsonb

Verbotene Payloads: Secrets, Tokens, private Schlüssel, vollständige Prompts, vollständige Diffs, Request-Bodies mit PII.

Pre-Mutation Gate

Vor Ausführung müssen alle Punkte PASS sein: 1. Planungs-PR ist vom Owner `SvenKulesa` vollständig reviewed und freigegeben. 2. Tabelle `agent_audit_events` existiert noch nicht. 3. aktuelle Audit-/IAM-Tabellen und Policies wurden als Baseline exportiert. 4. Migration ist idempotenz-sicher bzw. fail-closed bei vorhandenen Objekten. 5. RLS/Grants sind deny-by-default gegenüber `anon` und `authenticated` geplant. 6. Append-only-Negativtest ist definiert. 7. Rollback-SQL liegt vor.

Mutation

Die eigentliche DDL wird erst im nachgelagerten M5-Mutationsschritt über das Supabase-Plugin angewendet. Keine DDL aus einem Development-Branch direkt gegen Produktion ohne dieses Gate.

Post-Mutation Verification

Pflichtprüfungen: 1. Tabelle/Spalten/Constraints entsprechen dem freigegebenen Schema. 2. RLS ist aktiviert. 3. `anon` und `authenticated` besitzen keine direkten Schreib-/Leserechte auf Audit-Evidence. 4. ein redigiertes synthetisches Testevent kann über den vorgesehenen Server-/Service-Pfad geschrieben und korreliert gelesen werden. 5. UPDATE und DELETE werden fail-closed abgelehnt. 6. verbotene Felder werden vom Applikationsvertrag/redaction layer abgewiesen bzw. entfernt. 7. Supabase Security Advisor wird nach DDL erneut ausgeführt. 8. Evidence enthält Vorher/Nachher-Zustand, Actor, Timestamp und Verification Result.

Rollback

Rollback wird nur bei fehlgeschlagener oder inkonsistenter Verifikation ausgeführt.

Vor produktiver Nutzung ohne relevante Daten: - `agent_audit_events` samt zugehörigen Triggern/Policies kontrolliert entfernen.

Nach produktiver Nutzung: - kein destruktives Drop ohne separate Owner-Freigabe; - Writer deaktivieren, Evidence sichern/exportieren, Fehlerursache beheben und neue Migration planen.

Rollback gilt erst als PASS, wenn Schema/Policies wieder dem dokumentierten Pre-Mutation-Zustand entsprechen und die Anwendung weiterhin gesund ist.

Stop Conditions

Sofort STOP bei: - unerwartetem bestehenden Objekt gleichen Namens; - RLS/Grant-Abweichung; - Möglichkeit für Clientrollen Audit-Evidence zu verändern; - Secret/PII-Leak im Testevent; - nicht reproduzierbarem Before/After-State; - fehlender Owner-Freigabe.

M5A Runbook — Supabase Native TOTP MFA / AAL2 Hardening

Quelle: docs/runbooks/M5A_SUPABASE_TOTP_AAL2_HARDENING.md

M5A Runbook — Supabase Native TOTP MFA / AAL2 Hardening

Status: PLANNED — NO PRODUCTION MUTATION AUTHORIZED

Date: 2026-08-12

Authority: ESS-0020, ADR-0064

Production project: AIFINANCIAL (`ryzywoktpmyhwzxmstyu`)

1. Safety rule

This runbook is sequential and fail-closed:

`BASELINE` → `OWNER APPROVAL` → `CODE IMPLEMENTATION + CI` → `PRE-MUTATION CHECK` → `OWNER MUTATION APPROVAL` → `NATIVE MFA ENROLLMENT` → `AAL2 VERIFICATION` → `NEGATIVE TESTS` → `RECOVERY TEST` → `SECURITY ADVISOR` → `EVIDENCE` → `ROADMAP UPDATE`

Do not skip a gate. No step in the baseline PR authorizes enrollment, factor deletion, project Auth setting changes or legacy data cleanup.

2. Baseline state

Verified read-only on 2026-08-12:

- native MFA factors: 0;
- production AAL2 sessions: 0;
- current active session evidence: 2 × AAL1;
- two Owner profiles have legacy custom TOTP enabled;
- 20 unused legacy break-glass codes exist;
- no active legacy step-up token exists;
- Security Advisor reports `auth_insufficient_mfa_options`;
- project organization plan is Free;
- native Auth MFA tables already exist.

3. Stage A — Architecture approval

Required repository artifacts:

- ESS-0020;
- ADR-0064;
- M5A baseline evidence;

- this runbook;
- ROADMAP + implementation roadmap + traceability update;
- ADR-0003.5 reactivated as IN PROGRESS because its Owner-TOTP assurance statement is not currently satisfied by Supabase AAL2.

Exit:

- Human/Owner reviewed current PR head;
- docs-fast-path PASS;
- PR merged;
- baseline branch deleted after merge.

4. Stage B — Application implementation

Create a fresh branch from the post-Stage-A `main`.

B1 — Native enrollment

Replace the authoritative TOTP setup path with Supabase Native MFA:

1. call `supabase.auth.mfa.enroll({ factorType: 'totp' })`;
2. render QR code and optional secret only to current authenticated user;
3. create a challenge;
4. verify the entered TOTP code;
5. evaluate `getAuthenticatorAssuranceLevel()` after verify;
6. treat setup as successful only if the resulting state is `aal2/aal2`.

Do not write a native TOTP secret to public application tables.

B2 — Login/session gate

For Owner/Admin privileged access:

- `aal1/aal1` → enrollment required / deny privileged access;
- `aal1/aal2` → challenge required;
- `aal2/aal2` → MFA condition satisfied;
- `aal2/aal1` → stale/downgraded; deny and refresh/re-authenticate.

Network/Auth errors → deny privileged access. Remove the current fail-open privileged behavior.

B3 — Server enforcement

Add one canonical helper, conceptually:

```
requireVerifiedAal2(req)
```

It must use the Bearer token already tied to the Supabase identity and must not trust client-provided AAL fields.

Privileged server paths must compose:

```
checkAdminAccess
+ requireVerifiedAal2
+ requireStepUp (where action contract requires fresh purpose-bound confirmation)
```

B4 — Step-up

Retain the purpose-bound single-use token only as defense-in-depth.

It cannot:

- be issued from an AAL1 session;
- be accepted if the current user session is no longer valid AAL2;
- authorize a different user or purpose;
- be replayed after consumption.

B5 — Recovery

Implement a native-MFA-aware recovery contract before legacy TOTP retirement.

Preferred controls:

- backup TOTP factor/recovery procedure documented and tested;
- destructive native factor removal restricted to Owner-controlled server path;
- factor deletion via supported Supabase Admin MFA API;
- factor reset logged as CRITICAL audit event;
- no automatic role elevation;
- no automatic unrelated passkey deletion.

5. Stage C — CI / pre-mutation verification

Before touching production factors:

Required CI:

- repository integrity;
- TypeScript;
- unit tests;
- new MFA/AAL2 negative tests;
- production build;
- predeploy/runtime checks required by changed scope.

Mandatory negative tests:

1. AAL1 Owner/Admin denied;

2. missing factor denied;
3. unverified factor denied;
4. invalid TOTP denied;
5. expired/invalid challenge denied;
6. stale AAL2/AAL1 denied;
7. Auth/AAL verification error denied;
8. AAL1 + valid legacy/custom step-up denied;
9. AAL2 + wrong-purpose token denied;
10. replayed token denied;
11. unauthorized recovery/factor deletion denied.

Stage C exit: CI `VERIFIED PASS` .

6. Stage D — Production pre-mutation check

READ ONLY.

Immediately before enrollment:

1. confirm project health;
2. rerun aggregate native factor count;
3. rerun aggregate AAL distribution;
4. verify current Owner roles without exposing identities;
5. verify Security Advisor baseline;
6. test whether Native TOTP enroll/challenge/verify is available through the application path;
7. determine whether Dashboard Auth configuration mutation is actually required.

If native API is unavailable due project setting, STOP and request a separate Owner approval for the exact setting change.

Do not alter settings speculatively.

7. Stage E — Human/Owner production mutation approval

Approval must explicitly cover:

- native TOTP enrollment for the two Owner identities;
- any exact Auth configuration change proven necessary in Stage D;
- controlled recovery test scope;
- expected session invalidation/refresh behavior.

Without this approval: STOP.

8. Stage F — Native Owner enrollment

Perform via supported application/Supabase Auth flow, one Owner identity at a time.

For each Owner:

1. maintain a verified recovery path for the other Owner before starting;
2. enroll TOTP;
3. challenge;
4. verify;
5. confirm current AAL2;
6. sign out/in or refresh as required and prove MFA challenge behavior;
7. verify privileged AAL2-gated action in a safe/non-destructive test;
8. record redacted evidence only.

Never record the TOTP secret, QR payload, code or factor identifier in repository evidence.

9. Stage G — Negative production verification

Use safe/non-destructive paths where possible:

- AAL1 session cannot invoke privileged endpoint;
- invalid code cannot verify;
- stale/downgraded session cannot invoke privileged endpoint;
- missing purpose-bound step-up blocks critical Owner action;
- consumed token replay blocks;
- audit event contains no sensitive MFA payload.

No test should deliberately weaken production security configuration.

10. Stage H — Recovery verification

Verify a documented recovery/backup route without permanently locking out both Owner accounts.

If testing native factor removal:

- explicit Human/Owner approval required;
- test only one controlled factor while another valid recovery path remains;
- verify all affected sessions are invalidated as expected;
- re-enroll and restore AAL2 before declaring PASS.

Legacy break-glass codes remain transitional until this stage passes.

Update 2026-08-14 (Owner-Policy, ahead of this stage): the Owner has explicitly directed that no in-app emergency bypass exist ("kein Notfall-Bypass-Code in der Anwendung"). The `/break-glass/redeem` application route and all recovery-code UI have

already been removed from the codebase (see `docs/evidence/m5a/M5A_REPOSITORY_IMPLEMENTATION_EVIDENCE.md`, Nachtrag 2026-08-14) — independent of and prior to this runbook stage being reached. Stage H's documented recovery route is therefore no longer available in-app; loss of both Passkey and Authenticator can only be resolved via Supabase-Dashboard administration by the Owner. Plan this stage's Owner-approval test accordingly (there is no code-level fallback to fall back on if a test factor removal goes wrong — verify re-enrollment capability via the Dashboard path before testing, not via break-glass codes).

11. Stage I — Post-mutation verification

After successful native enrollment:

- native factor aggregate reflects expected verified factors;
- AAL2 session evidence exists for tested Owner sessions;
- privileged AAL1 paths deny;
- Advisor rerun completed;
- no new critical Security Advisor finding created;
- M5A audit evidence completed.

12. Stage J — Legacy cleanup decision

Do **not** clean up automatically.

Create a separate mutation decision for:

- `profiles.totp_enabled`;
- `totp_secret_encrypted` / pending secret fields;
- legacy break-glass code **table/data** (application code already removed 2026-08-14 per Owner-Policy — this item is now data-only, see runbook update in Stage H above);
- legacy local TOTP verifier and server setup routes;
- legacy step-up issuance logic that still depends on local TOTP.

Cleanup is authorized only after native MFA + recovery + AAL2 enforcement are `VERIFIED PASS`.

13. Rollback / stop conditions

STOP immediately if:

- native factor enrollment cannot reach AAL2;
- both Owner recovery paths would be at risk simultaneously;
- server AAL2 verification behaves fail-open;
- stale session is accepted;
- CI fails;

- Advisor shows a new critical finding;
- mutation scope differs from approved scope.

Rollback application enforcement if necessary, but preserve audit evidence and do not delete native factors automatically.

14. M5A exit

M5A may be marked `COMPLETE / VERIFIED PASS` only after:

- code implementation merged;
- required CI PASS;
- native Owner enrollment verified;
- AAL2 positive/negative tests PASS;
- recovery PASS;
- Advisor rerun;
- final evidence + Roadmap + traceability synchronized.

Only then may M6 begin.

M6 — Supply Chain Provenance & Attestation Runbook

Quelle: docs/runbooks/M6_SUPPLY_CHAIN_PROVENANCE.md

M6 — Supply Chain Provenance & Attestation Runbook

Status: PLANNED — EXECUTION BLOCKED BY M5A Date: 2026-08-12 Authority: ADR-0060, docs/architecture/ai-agent/AI_AGENT_SUPPLY_CHAIN_MODEL.md , DEVELOPMENT Chain Execution Policy

Goal

Bind one exact CAPITAL-AI source state to its dependency graph, SBOM, build result, artifact digest, provenance/attestation, release/deployment evidence and rollback artifact.

Canonical chain:

```
source SHA
→ committed lockfile
→ deterministic dependency install
→ SBOM
→ tests/build
→ artifact digest
→ provenance
→ attestation
→ deployment identity
→ runtime identity
```

No M6 implementation or CI/workflow mutation is authorized until M5A is COMPLETE / VERIFIED PASS .

Standards Baseline

M6 should implement against the current approved SLSA specification at execution time and record the exact version in Evidence. Documentation preparation uses SLSA v1.2 as the current approved baseline.

NIST SSDF SP 800-218 v1.1 remains the stable final baseline; later revisions/drafts may be tracked, but draft requirements must not be presented as final normative controls until adopted by a separate review.

OpenSSF Scorecard is supplementary automated evidence. A Scorecard result is not a substitute for source/artifact provenance, Human review or CAPITAL-AI policy gates.

Required Outputs

1. exact source SHA;
2. dependency lockfile digest;
3. generated machine-readable SBOM (SPDX or CycloneDX);

4. build/test result bound to source SHA;
5. artifact/image digest;
6. provenance statement identifying builder/source/build inputs;
7. verifiable signature/attestation when supported by the selected trusted build path;
8. dependency/security findings and disposition;
9. immutable reference to the last-known-good artifact for rollback;
10. Evidence bundle under `docs/evidence/m6/`.

Target Assurance

Minimum intended posture:

- provenance generated by a hosted/trusted build path rather than a developer laptop;
- source and artifact digests cryptographically bound;
- build identity distinguishable from agent/model identity;
- signing/attestation key material unavailable to untrusted PR code;
- pinned immutable GitHub Actions where possible;
- explicit least-privilege workflow permissions;
- deterministic `npm ci` from committed lockfile;
- production dependency audit retained;
- no rebuild of an old release from mutable inputs for rollback.

Where the environment supports stronger isolated/hardened build guarantees, M6 may target the corresponding higher SLSA Build assurance only after the required controls are proven rather than inferred.

Phase 0 — Read-only Preflight

Before implementation:

1. confirm M5A `VERIFIED PASS`;
2. resolve current `main`;
3. inventory build, Docker, npm and release workflows;
4. identify all third-party Actions and verify immutable pinning status;
5. inspect current package lock integrity and Node runtime contract;
6. inventory current SBOM/provenance/attestation tooling, if any;
7. inspect Render artifact/deployment correlation available at baseline;
8. classify proposed changes as C/R and identify any M7 mutation dependency;
9. define evidence and rollback before workflow changes.

Phase 1 — Repository Implementation

Use a fresh branch from current `main`.

Implement only repository-side supply-chain controls, such as:

- SBOM generation;
- artifact digest export;
- provenance/attestation generation;
- verification scripts;
- workflow permission/action pinning where needed;
- Evidence manifest tying outputs to `${GITHUB_SHA}` ;
- checks proving source/lock/SBOM/artifact consistency.

Do not perform Render/Stripe/Supabase external mutation in M6 unless a separately approved roadmapped requirement proves it is necessary. Deployment identity changes belong to M7.

Required Positive Tests

- SBOM generated for exact current source/lock state;
- SBOM includes the expected top-level application identity and dependency inventory;
- provenance subject digest equals built artifact digest;
- source reference equals exact commit SHA;
- attestation verifies with expected builder identity/trust path;
- artifact chosen for rollback is retrievable by immutable digest/reference;
- workflow-security checks pass.

Required Negative Tests

- changed source with stale provenance → DENY;
- changed lockfile with stale SBOM/provenance → DENY;
- artifact digest mismatch → DENY;
- unsigned/untrusted attestation when signature is required → DENY;
- unexpected builder/workflow identity → DENY;
- mutable/unpinned critical Action introduced → DENY according to repository policy;
- provenance/SBOM unavailable → release/deploy gate must fail closed where M6 requires them;
- agent/provider name claiming trust elevation → ignored/DENY.

Evidence Contract

Create phase evidence under `docs/evidence/m6/` using the generic DEVELOPMENT Chain Evidence Template.

Evidence must include:

- final PR/head/merge SHA;

- CI run(s);
- SBOM format/tool/version and digest;
- provenance format/version and subject digest;
- attestation verification result;
- builder/workflow identity;
- artifact digest;
- dependency audit result;
- negative-test results;
- last-known-good rollback reference;
- branch deletion state.

Do not commit generated secrets, signing private keys or sensitive runtime credentials.

Mutation State

External platform mutation: `NOT REQUIRED` by default for M6.

If a repository implementation discovers an unavoidable external mutation, M6 stops and produces a dedicated Roadmap/ADR/Runbook update before any such mutation is attempted.

Rollback

Repository changes: new revert branch from current `main` → Human-reviewed revert PR.

Release/runtime rollback: use the previously verified immutable artifact/digest; do not rebuild old source using newer/mutable dependencies or toolchains.

Exit Gate

M6 is `COMPLETE / VERIFIED PASS` only when:

1. M5A prerequisite was verified;
2. required repository controls are Human-merged;
3. exact source → lockfile → SBOM → artifact → provenance/attestation chain verifies;
4. required negative tests pass;
5. rollback artifact/reference is proven;
6. evidence is complete;
7. branch is deleted;
8. Roadmap/Traceability are synchronized.

Only then may M7 begin.

M7 — Render Deploy-Hook-Rotation: Mutation Work Order

Quelle: docs/runbooks/M7_DEPLOY_HOOK_ROTATION_HANDOFF.md

M7 — Render Deploy-Hook-Rotation: Mutation Work Order

Status: **VERIFIED PASS** (durchgeführt und real verifiziert, siehe Nachtrag Abschnitt 9) — M7 als Ganzes bleibt PLANNED Datum: 2026-08-14 (Nachtrag: 2026-08-14, selber Tag)
Roadmap phase: M7 (ADR-0061, docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md) Authority: docs/contracts/DEVELOPMENT_CHAIN_MUTATION_HANDOFF_CONTRACT.md , docs/runbooks/RENDER_PRODUCTION_EVIDENCE_HANDOFF.md

0. Warum dies kein maschinenvalidierter Handoff ist

.ai/contracts/development-chain-mutation-handoff.schema.json erzwingt executorAgentId: "capital-ai-systemadmin-roadmap-executor" (SA3B) als einzig gültigen Executor. SA3B ist ein GitHub-Actions/OIDC-Host mit ausschließlich GitHub-seitigem Mandat (kein Render-API-Zugriff) — es existiert derzeit kein verifizierter, autonomer Execution Host für Render-Mutationen. Diese Mutation wird daher **direkt vom Human Owner** durchgeführt, exakt nach dem in RENDER_PRODUCTION_EVIDENCE_HANDOFF.md etablierten Muster (wie bereits bei den M5A- Faktor-Registrierungen). Dieses Dokument folgt der konzeptuellen Struktur des Mutation Handoff Contract (Ziel, erlaubte/verbotene Operationen, Pre-/Post-Checks, Rollback), ist aber **keine** .json -Instanz gegen das Schema und autorisiert nichts von selbst.

Gemäß CLAUDE.md mutiert diese Claude-Code-Sitzung **nicht direkt** Render-Produktionskonfiguration von einem Entwicklungsschritt aus. Diese Sitzung bereitet den Work Order vor, führt die Pre-/Post-Mutation-Verifikation read-only durch und dokumentiert das Ergebnis — die eigentliche Änderung im Render-Dashboard und in den GitHub-Secrets erfolgt durch SvenKulessa selbst.

1. Ziel

Den bestehenden Render-Deploy-Hook (referenziert als GitHub-Actions-Secret RENDER_DEPLOY_HOOK_URL , verwendet in .github/workflows/ci.yml , Job deploy-production) rotieren: neuen Hook erzeugen, GitHub-Secret aktualisieren, alten Hook widerrufen. Beweist die im M7-Runbook geforderte „rotation/revocation“-Fähigkeit für die Deployment-Credential-Bridge — bisher ungetestet.

2. Strukturierte Feldbeschreibung (konzeptuell an den Handoff Contract angelehnt)

- **platform:** RENDER (Hook-Erzeugung/-Widerruf) + GITHUB (Secret-Update)
- **mutationClass:** PLATFORM_CONFIG

- **riskClass:** `LOW` — betrifft ausschließlich den CI→Render-Deploy-Trigger, nicht die laufende Produktionsinstanz; ein Fehler führt bestenfalls dazu, dass ein Deploy-Trigger fehlschlägt, nicht zu einem Ausfall des laufenden Service.
- **targetResource:** Render-Service `srv-d91o1o9o3t8c73edi55g` ("Finance", `SvenKulesa/Finance`, Environment `evm-d90oshpkh4rs739l1fm0`); GitHub-Repository-Secret `RENDER_DEPLOY_HOOK_URL` (`SvenKulesa/Finance` → Settings → Secrets and variables → Actions).
- **allowedOperations:** `RENDER_ROTATE_DEPLOY_HOOK:srv-d91o1o9o3t8c73edi55g`, `GITHUB_UPDATE_SECRET:RENDER_DEPLOY_HOOK_URL`.
- **forbiddenOperations:** jede andere Render-Service-/Environment-/Secret-Änderung; `MERGE`; `SELF_AUTHORITY_EXPANSION`; `SECURITY_CONTROL_DISABLEMENT`; jede Änderung an anderen Secrets/Keys.
- **expectedPostState:** neuer Deploy-Hook aktiv, GitHub-Secret auf neuen Wert aktualisiert, alter Hook nicht mehr funktionsfähig, laufende Produktionsinstanz unverändert (kein Neustart durch die Rotation selbst ausgelöst).
- **idempotencyKey:** `m7-deploy-hook-rotation-2026-08-14` (einmalige Aktion; bei Wiederholung neuer Key mit neuem Datum).
- **expiresAt:** dieser Work Order verfällt, wenn nicht innerhalb von 14 Tagen ab Erstellungsdatum durchgeführt — danach vor Ausführung erneut gegen aktuellen `main`/Produktionsstand prüfen.

3. Pre-Mutation Baseline (read-only erfasst, 2026-08-14T21:55 UTC)

```
renderProductionEvidence:
  capturedAt: 2026-08-14T21:55:00Z
  service:
    id: srv-d91o1o9o3t8c73edi55g
    name: Finance
    runtime: docker
    repo: SvenKulesa/Finance
    branch: main
    region: frankfurt
    plan: starter
    instances: 1
    healthCheckPath: /healthz
    autoDeployTrigger: "off"
  deploy:
    liveDeployId: dep-d9vomis9v7es73bc29e0
    liveCommitSha: fd52af95449ef4ee13114dba0649976126d0db8d
    liveStatus: live
  githubSecretReferenced: RENDER_DEPLOY_HOOK_URL (Wert nicht gelesen/eingesehen)
```

Kein Secret-Wert wurde zu irgendeinem Zeitpunkt gelesen oder in dieser Sitzung ausgegeben — nur Service-/Deploy-Metadaten über die read-only Render-MCP-Tools (`get_service`, `list_deploys`).

4. Durchführung (durch `SvenKulesa`, nicht durch diese Sitzung)

1. Render-Dashboard → Service `Finance` → Settings → Deploy Hook → neuen Hook erzeugen (Render erzeugt eine neue URL; je nach Render-UI-Version alten Hook explizit widerrufen/löschen, falls er nicht automatisch invalidiert wird — bitte im Dashboard prüfen).

2. Neue Hook-URL **niemals** in Chat, PR, Issue, Screenshot oder Dokumentation einfügen.
3. GitHub → `SvenKulesa/Finance` → Settings → Secrets and variables → Actions → `REN-
DER_DEPLOY_HOOK_URL` → Wert auf die neue URL aktualisieren.
4. Alten Hook im Render-Dashboard als widerrufen/gelöscht bestätigen.
5. Mir (dieser Sitzung) Bescheid geben, sobald Schritt 1–4 abgeschlossen sind.

5. Post-Mutation Verification (durch diese Sitzung, read-only)

Nach Rückmeldung des Owners:

1. auf den nächsten realen `push`-Lauf auf `main` warten (oder einen bewusst ausgelösten leeren Commit anfragen, falls kein natürlicher Push ansteht);
2. `deploy-production`-Job-Log prüfen: Schritt „Render-Deployment für verifizierten main-Commit auslösen“ muss weiterhin PASS liefern (beweist: neuer Hook funktioniert, GitHub-Secret korrekt aktualisiert);
3. per `list_deploys` (Render-MCP, read-only) bestätigen, dass für den ausgelösten Commit ein neuer Deploy-Eintrag mit `status: live` erscheint;
4. `verify-deployment-identity`-Job weiterhin PASS (bestätigt: Ende-zu-Ende-Kette weiterhin intakt);
5. optional, falls vom Owner bestätigbar: manueller Test, dass der **alte** Hook-URL-Aufruf jetzt fehlschlägt (Beweis des Widerrufs) — nur wenn der Owner die alte URL noch zur Hand hat, sonst entfällt dieser Schritt ersatzlos (kein Secret-Wert wird dafür angefragt oder gespeichert).

6. Rollback

Bei Fehlschlag (neuer Hook funktioniert nicht): Owner erzeugt im Render-Dashboard erneut einen neuen Hook und aktualisiert das GitHub-Secret erneut — keine Datenverlust-, Ausfall- oder Sicherheitsfolge, da die laufende Produktionsinstanz von diesem Vorgang nicht betroffen ist. Kein Rollback auf den alten (potenziell kompromittierten/beendeten) Hook — neue Rotation statt Rückfall auf den alten Wert, konsistent mit `REN-
DER_PRODUCTION_EVIDENCE_HANDOFF.md` Abschnitt 15 („Secret regression“).

7. Evidence Output (nach Abschluss zu ergänzen)

Nach erfolgreicher Durchführung: Nachtrag in diesem Dokument mit Post-Mutation-Verifikationsergebnis (Job-Run-Link, Deploy-ID, Commit-SHA) — kein Secret-Wert. Anschließend `docs/evidence/m7/*` entsprechend ergänzen und M7-Traceability für dieses Element aktualisieren.

9. Nachtrag — Durchführung und Post-Mutation Verification (2026-08-14)

Der Owner hat die Rotation durchgeführt: neuen Deploy-Hook im Render-Dashboard erzeugt, GitHub-Secret `RENDER_DEPLOY_HOOK_URL` aktualisiert, alten Hook gelöscht. Kein Secret-Wert wurde dieser Sitzung zu irgendeinem Zeitpunkt mitgeteilt oder von ihr gelesen.

Der Merge von PR #270 löste den realen `push`-Lauf [31845661423](#) aus — der erste Deploy-Trigger-Versuch mit dem **neuen** Hook-Wert. Drei unabhängige Nachweise bestätigen den Erfolg:

1. **GitHub-Actions-Job-Log** (`deploy-production` , Job-ID `94912100575`): der `curl`-Aufruf gegen den (aus dem aktualisierten GitHub-Secret gelesenen) Hook lieferte `{"deploy":{"id":"dep-d9vp6unmal7c73855dq0"}}` — HTTP-Erfolg mit gültiger Render-Deploy-ID.
2. **Render-Deploy-Historie** (read-only per `list_deploys` , unabhängig von GitHub erfasst): exakt diese Deploy-ID `dep-d9vp6unmal7c73855dq0` , gebunden an Commit `888112d483764ef8f5d8abb918ac0e1b855c9a9d` (dem auslösenden Commit), mit `status:"live"` .
3. **verify-deployment-identity -Job** (Job-ID `94912127023`): PASS — bestätigt zusätzlich, dass die tatsächlich laufende Instanz den erwarteten Commit über `/healthz` meldet.

Der alte Hook wurde vom Owner im Render-Dashboard gelöscht (nicht separat durch diese Sitzung verifizierbar, da kein Zugriff auf den alten Wert bestand/besteht — konsistent mit der Secret-Policy dieses Dokuments).

Ergebnis: Rotation + Revocation der Deployment-Credential-Bridge ist real bewiesen — die im M7-Runbook geforderte, bisher ungetestete Fähigkeit ist jetzt `VERIFIED PASS` .

Was dies NICHT bedeutet: M7 als Ganzes ist damit **nicht** `COMPLETE` / `VERIFIED PASS` . Der M7-Exit-Gate (`docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md` , Abschnitt „Exit Gate“) verlangt zusätzlich mindestens:

- **Punkt 7, „rollback is proven“:** ein echter Render-Rollback (Wiederherstellung eines vorherigen Deploys) wurde **nie** ausgeführt oder verifiziert — nur die (andersartige) Credential-Rotation. Dies bleibt offen.
- **Punkt 6, „negative tests pass“:** mehrere Punkte der Runbook-Liste „Required Negative Tests“ (`duplicate/replayed mutation request → DENY/DEDUPE` , `execution without durable audit permit → DENY` , `Handoff target mismatch → DENY`) setzen einen automatisierten Execution Host mit Audit-Permit voraus — dieser existiert für Render nicht (siehe Abschnitt 0). Diese Punkte sind für eine manuell durch den Owner durchgeführte Mutation konzeptuell nicht 1:1 anwendbar und wurden nicht durch ein automatisiertes Testsystem nachgewiesen.

M7 bleibt daher weiterhin `PLANNED` — zwei von mehreren möglichen Scope-Elementen sind jetzt real verifiziert (Repository-Controls-Paket; Deploy-Hook-Rotation), aber der vollständige Exit Gate ist nicht erreicht.

8. Freigabestatus

- [x] Human/Owner: Work Order geprüft und Umfang bestätigt.
- [x] Human/Owner: Schritte 1-4 durchgeführt.
- [x] Diese Sitzung: Post-Mutation Verification durchgeführt und dokumentiert (Abschnitt 9).

Dieses Dokument autorisiert selbst keine Ausführung. Die Ausführung ist bereits erfolgt (siehe Abschnitt 9); dieses Dokument diene als Vorbereitung und dient jetzt als Evidence.

M7 — Deployment Identity & Production Platform Mutation Runbook

Quelle: docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md

M7 — Deployment Identity & Production Platform Mutation Runbook

Status: **COMPLETE / VERIFIED PASS** (2026-08-14, siehe Abschnitt „Exit Gate Closure“ am Ende dieses Dokuments) — M8 unblocked Date: 2026-08-12 (Closure: 2026-08-14)
Authority: ADR-0061, docs/architecture/ai-agent/AI_AGENT_DEPLOYMENT_IDENTITY.md, DEVELOPMENT Chain Execution Policy

Goal

Establish a bounded, attributable production deployment path that binds a verified source/artifact identity to the exact Render deployment/runtime identity and governs every required external platform mutation through explicit Owner approval, auditable execution and rollback.

Security Boundary

Production deploy authority is separate from AI-agent repository authority.

No provider/model/tool receives implicit `PRODUCTION_MUTATION` because it can create code, branches, PRs or CI requests.

`MERGE` remains Human/Owner-only.

Scope

Primary M7 scope

- Render production deployment identity;
- environment-scoped deployment credentials/hooks/bridge;
- branch/source/artifact binding;
- rotation/revocation;
- controlled deploy verification;
- health/readiness and rollback evidence.

Conditional scope

Stripe or Supabase mutations are included **only** when a dedicated ADR/runbook/work item explicitly names the exact resource and operation.

Human-reserved scope

IONOS / DNS / TLS / domain ownership remains Human/Owner-only unless a future dedicated ADR introduces equivalent or stronger assurance.

Required Trust Chain

```
verified main source SHA
→ M6 verified artifact/provenance
→ protected deployment request
→ environment-scoped deployment identity
→ exact Render service/environment
→ deployed artifact/commit identity
→ health/readiness verification
→ immutable deployment evidence
```

Phase 0 — Read-only Preflight

Before any repository or platform change:

1. confirm M6 COMPLETE / VERIFIED PASS ;
2. resolve current `main` and M6 artifact/provenance evidence;
3. identify exact Render service(s), environment(s), deploy path and currently deployed commit/artifact;
4. inspect current credential/hook ownership, rotation and revocation model without exposing values;
5. identify whether GitHub OIDC can be used directly; where not possible, document the narrow secret bridge and its owner/revocation path;
6. inspect current health/readiness/rollback mechanism;
7. classify each proposed platform mutation individually;
8. prepare a DevelopmentChain Mutation Handoff for each externally mutating target;
9. define last-known-good deploy and rollback trigger;
10. check for active deploy/config writers.

Unknown target/account/environment → STOP .

Repository Implementation

On a fresh branch from current `main` , implement only the repository-side controls required for M7, such as:

- deployment identity verification;
- expected source/artifact binding;
- workflow/environment guardrails;
- deploy request metadata;
- post-deploy health/readiness verification;
- rollback tooling/reference;

- redacted evidence generation.

Workflow/runtime changes are Check Class R and receive the full required validation.

External Mutation Work Order

Every Render/Supabase/Stripe state change uses:

- `docs/contracts/DEVELOPMENT_CHAIN_MUTATION_HANDOFF_CONTRACT.md` ;
- `.ai/contracts/development-chain-mutation-handoff.schema.json` .

Required binding includes:

- exact platform/account/project/service/environment;
- exact allowed operation;
- expected post-state;
- pre/post checks;
- rollback plan;
- idempotency/concurrency key;
- expiry;
- separate Human Mutation Approval Evidence.

The Handoff itself does not authorize execution.

Owner Mutation Gate

After repository implementation + required CI and immediately before external mutation:

1. rerun read-only platform precheck;
2. show exact target and proposed state transition;
3. verify rollback is executable;
4. obtain explicit Human/Owner mutation approval;
5. bind approval evidence to the exact target/operation/risk/time window.

Any material drift after approval invalidates or reopens the approval gate.

Authorized Execution

Preferred autonomous sequence after the Systemadmin Execution Host is independently verified:

```
validated Handoff
→ REM / IAM / reserved-action policy
→ durable authorization audit
→ exact host permit
→ one bounded platform mutation
→ durable outcome audit
→ post-verification
```

Do not accept arbitrary command strings from issue/comment/model output as mutation instructions.

Render Verification

After each approved Render mutation/deploy, prove at least:

- exact service/environment targeted;
- expected commit/artifact/source identity deployed;
- health endpoint PASS;
- readiness/deployment status PASS;
- no unexpected environment/config drift;
- runtime identity corresponds to approved source/artifact;
- rollback target remains available;
- audit/evidence correlation complete.

Stripe Conditional Verification

If M7 explicitly includes a Stripe mutation:

- exact account + live/test mode;
- exact webhook/config/credential resource;
- no live Money/Entitlement mutation unless separately Human-reserved approval explicitly covers it;
- webhook signing/idempotency/mapping checks;
- revocation/rollback path;
- no secret value in evidence.

Supabase Conditional Verification

If M7 explicitly includes a Supabase mutation:

- exact project/reference;
- exact Auth/DB/config resource;
- no opportunistic repetition of M5/M5A changes;
- RLS/IAM/security advisor implications reviewed where applicable;
- rollback and data-integrity impact proven.

Required Negative Tests

- deploy request from non-`main` or unverified source → DENY;
- source/provenance/artifact mismatch → DENY;
- wrong Render service/environment → DENY;
- expired/missing Owner mutation approval → DENY;

- Handoff target mismatch → DENY;
- duplicate/replayed mutation request → DENY/DEDUPE;
- execution without durable audit permit → DENY;
- agent attempts `MERGE` → DENY;
- arbitrary platform operation outside Handoff → DENY;
- health/readiness failure → rollback/STOP.

Rollback

Deployment rollback

Restore the last-known-good **verified immutable artifact/deployment reference**, then rerun health/readiness/source-identity verification.

Configuration rollback

Restore the previously captured redacted configuration state through an explicitly authorized rollback operation. Rotate/revoke compromised bridge credentials when relevant.

Repository rollback

Use a new revert branch from current `main`; never revive a merged implementation branch.

Evidence

Create `docs/evidence/m7/*` with:

- M6 prerequisite evidence;
- exact source/artifact/deploy identity;
- platform baseline;
- Handoff ID;
- Human Approval Evidence ref;
- authorization/outcome audit refs;
- mutation result;
- post-verification;
- rollback state;
- branch deletion state.

Exit Gate

M7 is `COMPLETE / VERIFIED PASS` only when:

1. M6 prerequisite is verified;
2. repository controls are merged and validated;
3. every required external mutation is separately Human-approved;

4. every required external mutation is `VERIFIED PASS` or safely `FAILED` / `ROLLED BACK` with the phase still blocked;
5. production deployment identity binds exact verified source/artifact to runtime;
6. negative tests pass;
7. rollback is proven;
8. Evidence and Roadmap/Traceability are synchronized;
9. work branches are deleted.

Only a fully `VERIFIED PASS` M7 unblocks M8.

Exit Gate Closure (2026-08-14)

Owner instruction "fang mit der Required Negative Tests Liste an" closed the last open item (negative tests). All 9 Exit Gate criteria are now walked through explicitly against real evidence:

1. **M6 prerequisite is verified** — `docs/evidence/m6/M6_REPOSITORY_IMPLEMENTATION_EVIDENCE.md`, `VERIFIED PASS` on the real hosted `push` build path (cosign/Sigstore).
2. **repository controls are merged and validated** — Provenance-Gate + Post-Deploy-Verification package `VERIFIED PASS`, `docs/evidence/m7/M7_PHASE0_AND_REPOSITORY_CONTROLS_EVIDENCE.md`.
3. **every required external mutation is separately Human-approved** — deploy-hook rotation (`docs/runbooks/M7_DEPLOY_HOOK_ROTATION_HANDOFF.md`), rollback and roll-forward (`docs/runbooks/M7_ROLLBACK_VERIFICATION_HANDOFF.md`); each performed by the Owner directly, never by this session. No further external mutation is required by this runbook's scope beyond these three (see "External Mutation Work Order" / "Render Verification" above).
4. **every required external mutation is `VERIFIED PASS`** — all three independently confirmed via at least 3 sources each (GitHub Actions logs, Render's own `list_deploys`, and `verify-deployment-identity` CI PASS).
5. **production deployment identity binds exact verified source/artifact to runtime** — `verify-deployment-identity` CI job, real PASS on multiple production pushes.
6. **negative tests pass** — `docs/evidence/m7/M7_REQUIRED_NEGATIVE_TESTS_EVIDENCE.md`: all 10 listed scenarios now have a concrete, automated, passing test.
7. **rollback is proven** — `docs/runbooks/M7_ROLLBACK_VERIFICATION_HANDOFF.md`, both rollback and roll-forward real-executed and independently verified.
8. **Evidence and Roadmap/Traceability are synchronized** — this document plus `docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md`, `docs/roadmaps/ROADMAP_CONSOLIDATION_MASTER_INDEX.md`, `docs/traceability/AI_AGENT_M0_M9_TRACEABILITY_MATRIX.md`, `docs/traceability/DEVELOPMENT_CHAIN_DOCUMENT_TRACEABILITY_MATRIX.md` updated in the same PR as this closure.

9. **work branches are deleted** — confirmed via `git ls-remote --heads origin` immediately before this closure: the session's designated branch does not persist after merge (repository has auto-delete-head-branches; each prior M7 PR's branch was gone the moment it merged), and no stray M7-related branch exists on the remote.

Result: all 9 criteria are met. M7 is COMPLETE / VERIFIED PASS . M8 is unblocked per this runbook's own closing rule. M8 itself has not been started and will not start without a separate, explicit Owner instruction — this closure only removes the blocker documented in `docs/runbooks/M8_AGENT_CUTOVER.md`'s Prerequisite Gate.

M7 — Deployment Rollback Verification: Mutation Work Order

Quelle: docs/runbooks/M7_ROLLBACK_VERIFICATION_HANDOFF.md

M7 — Deployment Rollback Verification: Mutation Work Order

Status: Rollback und Roll-Forward beide **VERIFIED PASS** (siehe Nachtrag Abschnitt 9 und Abschnitt 10) — M7-Exit-Gate-Punkt 7 vollständig geschlossen; M7 als Ganzes bleibt **PLANNED** Datum: 2026-08-14 (Nachträge: 2026-08-14, selber Tag) Roadmap phase: M7 (ADR-0061, docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md , Exit-Gate-Punkt 7 „rollback is proven“) Authority: docs/contracts/DEVELOPMENT_CHAIN_MUTATION_HANDOFF_CONTRACT.md , docs/runbooks/RENDER_PRODUCTION_EVIDENCE_HANDOFF.md

0. Warum dies kein maschinenvalidierter Handoff ist / warum diese Sitzung nicht selbst ausführt

Wie bei der Deploy-Hook-Rotation (docs/runbooks/M7_DEPLOY_HOOK_ROTATION_HANDOFF.md): kein verifizierter autonomer Execution Host für Render-Mutationen; `executorAgentId` im Handoff-Schema ist fest auf SA3B (kein Render-Zugriff) gebunden.

Zusätzlicher Grund, spezifisch für diese Mutation: Das einzige in dieser Sitzung verfügbare Render-Mutations-Tool (`trigger_deploy`) kann ausschließlich einen **neuen** Deploy des aktuellen Branches/Images auslösen — es kann **keinen spezifischen früheren Deploy als Rollback-Ziel** adressieren. Render's tatsächliche „Rollback zu einem bestimmten früheren Deploy“-Funktion ist ausschließlich über das Render-Dashboard verfügbar, nicht über ein in dieser Sitzung nutzbares Tool. Diese Mutation kann technisch gar nicht von dieser Sitzung ausgeführt werden, unabhängig von CLAUDE.md's Grundsatz, keine Render-Produktion direkt von einem Entwicklungsschritt aus zu mutieren.

1. Ziel

Beweisen, dass ein echter Render-Rollback auf einen vorherigen, bereits verifizierten Deploy funktioniert — das im M7-Runbook-Exit-Gate (Punkt 7) geforderte, bisher ungetestete „rollback is proven“. Danach: Roll-Forward auf den aktuellen `main`-Stand.

2. Strukturierte Felddescription (konzeptuell an den Handoff Contract angelehnt)

- **platform:** `RENDER`
- **mutationClass:** `DEPLOYMENT`
- **riskClass:** `LOW` — beide beteiligten Commits sind reine Dokumentationsänderungen (docs/**), es gibt **keinen** Anwendungs-/Laufzeitcode-Unterschied zwischen ihnen. Live-

Traffic während des Tests erhält funktional identisches Verhalten, lediglich ein anderer `x-capital-ai-commit-Identitäts-Header`.

- **targetResource:** Render-Service `srv-d91o1o9o3t8c73edi55g` ("Finance").
- **allowedOperations:** `RENDER_ROLLBACK_TO_DEPLOY:dep-d9vp043m8hqs73dv238g`, anschließend `RENDER_REDEPLOY_LATEST_MAIN:srv-d91o1o9o3t8c73edi55g` (Roll-Forward).
- **forbiddenOperations:** jede andere Service-/Environment-/Secret-Änderung; Rollback auf einen anderen als den unten benannten exakten Deploy; `MERGE`; `SELF_AUTHORITY_EXPANSION`; `SECURITY_CONTROL_DISABLEMENT`.
- **expectedPostState:** unmittelbar nach Rollback läuft Deploy `dep-d9vp043m8hqs73dv238g` (Commit `8b4cab066c7c893c4525cd8e015e80919b145bde`) live; nach Roll-Forward läuft wieder der aktuelle `main`-Stand live.
- **idempotencyKey:** `m7-rollback-verification-2026-08-14`.
- **expiresAt:** verfällt nach 14 Tagen ab Erstellungsdatum — danach vor Ausführung erneut gegen aktuellen `main`/Produktionsstand prüfen (die genannte Deploy-ID könnte in Render's Historie ablaufen/nicht mehr rollback-fähig sein).

3. Pre-Mutation Baseline (read-only erfasst, 2026-08-14T22:31 UTC)

```
renderProductionEvidence:
  capturedAt: 2026-08-14T22:31:00Z
  service:
    id: srv-d91o1o9o3t8c73edi55g
    name: Finance
  currentLiveDeploy:
    id: dep-d9vp6unmal7c73855dq0
    commitSha: 888112d483764ef8f5d8abb918ac0e1b855c9a9d
    commitMessage: "Merge pull request #270 (docs(m7): Deploy-Hook-Rotation Work Order)"
    status: live
  rollbackTargetDeploy:
    id: dep-d9vp043m8hqs73dv238g
    commitSha: 8b4cab066c7c893c4525cd8e015e80919b145bde
    commitMessage: "Merge pull request #269 (docs(m7): Repository-Controls-Paket VERIFIED PASS)"
    status: deactivated (war zuvor selbst `live` und `healthy`, siehe Nachtrag Abschnitt 0.2 in docs/evidence/m7/M7_PHASE0_AND_REPOSITORY_CONTROLS_EVIDENCE.md)
  diffBetweenCurrentAndTarget: "ausschließlich docs/runbooks/M7_DEPLOY_HOOK_ROTATION_HANDOFF.md (neue Datei) – keine Anwendungs-/Laufzeitcode-Änderung"
```

Kein Secret-Wert wurde zu irgendeinem Zeitpunkt gelesen oder ausgegeben.

4. Durchführung (durch `SvenKulessa`, nicht durch diese Sitzung)

1. Render-Dashboard → Service `Finance` → Tab „Deploys“.
2. Den Eintrag für Deploy `dep-d9vp043m8hqs73dv238g` (Commit `8b4cab066...`, „docs(m7): Repository-Controls-Paket auf VERIFIED PASS abschließen“) suchen.
3. Render's „Rollback“-Aktion für genau diesen Deploy auslösen (Render redeploys exakt dieses bereits gebaute Image, ohne Neu-Build).
4. Mir Bescheid geben, sobald der Rollback abgeschlossen ist — **vor** dem Roll-Forward-Schritt.
5. Nach meiner Rollback-Verifikation (Abschnitt 5): entweder erneut über das Dashboard „Deploy latest commit“ für `main` auslösen, oder einfach den nächsten regulären Mer-

ge/Push abwarten (jeder künftige PR-Merge auf `main` löst über den Deploy-Hook automatisch einen frischen Deploy des dann aktuellen `main`-Standes aus und stellt damit den Soll-Zustand wieder her).

5. Post-Mutation Verification (durch diese Sitzung, read-only)

Nach Owner-Rückmeldung „Rollback durchgeführt“:

1. per `list_deploys / get_service` (Render-MCP, read-only) bestätigen, dass Deploy `dep-d9v-p043m8hqs73dv238g` (oder ein neuer, gleichwertiger Rollback-Deploy-Eintrag mit identischem Commit) jetzt `status: live` meldet;
2. Zeitstempel/Reihenfolge in der Deploy-Historie prüfen, dass der Rollback tatsächlich nach dem vorherigen Live-Deploy erfolgte;
3. Ergebnis dokumentieren — **kein** automatisierter `/healthz`-Check möglich (kein Netzwerkzugriff dieser Sandbox auf `capital-ai.online`, siehe an anderer Stelle in dieser Sitzung wiederholt dokumentiert); Render's eigene, unabhängige Deploy-Status-Meldung ist die primäre Evidence.
4. Nach Bestätigung: Owner um Roll-Forward bitten (Abschnitt 4, Schritt 5) und dessen Abschluss ebenfalls per `list_deploys` bestätigen — Produktion darf nicht dauerhaft auf dem älteren Commit verbleiben.

6. Rollback des Rollbacks (falls nötig)

Sollte der Rollback selbst fehlschlagen oder unerwartetes Verhalten zeigen: Owner löst im Render-Dashboard erneut „Deploy latest commit“ für `main` aus — stellt sofort den bekannten guten Zustand wieder her. Kein Datenverlust möglich, da beide beteiligten Commits reine Dokumentationsänderungen sind.

7. Evidence Output (nach Abschluss zu ergänzen)

Nach erfolgreichem Rollback + Roll-Forward: Nachtrag in diesem Dokument mit beiden Verifikationsergebnissen (Deploy-IDs, Zeitstempel, kein Secret-Wert). Anschließend

`docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md`, `docs/roadmaps/ROADMAP_CONSOLIDATION_MASTER_INDEX.md` und beide Traceability-Matrizen aktualisieren — M7-Exit-Gate-Punkt 7 („rollback is proven“) als erfüllt markieren; M7 als Ganzes bleibt weiterhin unterhalb `VERIFIED PASS`, bis auch die volle „Required Negative Tests“-Liste geklärt ist (siehe `docs/runbooks/M7_DEPLOY_HOOK_ROTATION_HANDOFF.md` Abschnitt 9 für die bereits dokumentierte Einschränkung).

9. Nachtrag — Rollback real durchgeführt und verifiziert (2026-08-14)

Der Owner hat im Render-Dashboard den Rollback auf den in Abschnitt 3 genannten Ziel-Deploy ausgelöst und dieser Sitzung Bescheid gegeben. Read-only Verifikation per

`list_deploys` (Render-MCP, unabhängig von GitHub):

rollbackDeploy:

id: `dep-d9vpurulegvs73f7684g`

commitSha: `8b4cab066c7c893c4525cd8e015e80919b145bde`

```

status: live
trigger: rollback # Render's eigene Klassifizierung des Auslösers - nicht "deploy_hook"
startedAt: 2026-08-14T23:06:28Z
finishedAt: 2026-08-14T23:06:48Z

```

trigger: "rollback" ist Render's **eigene** Einordnung des Auslösers (nicht `deploy_hook`, nicht `manual`) — ein unabhängiger, plattformseitiger Beweis, dass tatsächlich die Rollback-Funktion verwendet wurde, nicht ein gewöhnlicher Neu-Deploy. Commit stimmt exakt mit dem in Abschnitt 3 benannten Ziel überein.

Nebenbefund (kein Fehler, nur zur Vollständigkeit dokumentiert): Während des Rollback-Zeitfensters löste ein regulärer `push` auf `main` (Merge von PR #275, Commit `3cc92df6...`) über den Deploy-Hook einen weiteren, unabhängigen Deploy-Versuch aus (`dep-d9vpuq7ma-17c7386r55g`). Dieser wurde von Render selbst mit `status: canceled` markiert, da der Rollback-Vorgang Vorrang erhielt. Das bedeutet: Produktion lief nach dem Rollback auf dem alten Commit `8b4cab06...`, während `main` bereits deutlich weiter war (PR #274 Outbox-Migration, PR #275 Market-Data-Contracts/Alpaca-Adapter — beides echte Code-Änderungen, kein reiner Dokumentations-Diff mehr). Der Roll-Forward (Abschnitt 4, Schritt 5) ist daher **nicht optional/kosmetisch**, sondern erforderlich, um Produktion wieder mit aktuellem `main` zu synchronisieren.

Ergebnis: M7-Exit-Gate-Punkt 7 („rollback is proven“) ist real bewiesen — ein echter, Render-seitig als solcher klassifizierter Rollback auf einen spezifischen vorherigen Deploy wurde erfolgreich durchgeführt und unabhängig verifiziert.

Roll-Forward-Plan: Statt eines direkten Render-Dashboard-Klicks oder eines Aufrufs von `mcp_render_trigger_deploy` aus dieser Sitzung (was den vollständigen CI-Gate — `build-and-test`, `M6-supply-chain-attestation` — umgehen würde, da dieses Tool nicht über den GitHub-Actions-Pfad läuft), erfolgt der Roll-Forward über den **regulären Merge dieses PRs**: der resultierende `push`-Lauf auf `main` löst `deploy-production` über die normale, M6-attestation-gegatete Kette aus und deployt automatisch den dann aktuellen `main`-Stand. Das ist der einzige Weg, Produktion wieder auf `main` zu bringen, ohne die M6/M7-Gate-Kette für diesen einen Deploy zu umgehen. Nachtrag mit dem Roll-Forward-Ergebnis folgt als separater PR, sobald dieser hier gemergt ist.

10. Nachtrag — Roll-Forward real durchgeführt und verifiziert (2026-08-14)

Der Merge von PR #277 (dieses Dokument, Nachtrag Abschnitt 9) löste den geplanten Roll-Forward automatisch über die reguläre, M6-attestation-gegatete `push-zu-main`-Kette aus (Run [31850025365](#)). Drei unabhängige Nachweise bestätigen den Erfolg:

1. **GitHub-Actions-Job-Log** (`deploy-production`): `curl` gegen den rotierten Deploy-Hook lieferte `{"deploy":{"id":"dep-d9vq75c9v7es73fu2uh0"}}` für `VERIFIED_COMMIT_SHA: 28917b08a8-fee0a6adaf58542e97522e5ab5a22b` (Merge-Commit von PR #277).
2. **Render-Deploy-Historie** (read-only per `list_deploys`, unabhängig von GitHub):

```
yaml rollForwardDeploy: id: dep-d9vq75c9v7es73fu2uh0 commitSha: 28917b08a8fee0a6a-daf58542e97522e5ab5a22b status: live trigger: deploy_hook startedAt: 2026-08-14T23:24:05Z finishedAt: 2026-08-14T23:24:55Z
```

Der vorherige Rollback-Deploy `dep-d9vpuru1egvs73f7684g` (Commit `8b4cab06...`) zeigt jetzt `status: deactivated` — sauber durch den Roll-Forward abgelöst, kein Rest-/Konfliktzustand.

3. **verify-deployment-identity** (CI-Job im selben Run): `PASS` nach 4 Versuch(en), `32206ms`: Commit `28917b08a8fee0a6adaf58542e97522e5ab5a22b` `live` und `healthy`.

Ergebnis: Produktion ist wieder vollständig mit `main` synchron, inklusive der zuvor durch den Rollback zurückgestellten Änderungen aus PR #274 (Outbox-Migration) und PR #275 (Market-Data-Contracts/Alpaca-Adapter). Der Roll-Forward ist damit ebenfalls real bewiesen, nicht nur geplant.

8. Freigabestatus

- [x] Human/Owner: Work Order geprüft und Umfang bestätigt (insbesondere: welcher Deploy exakt als Rollback-Ziel dient).
- [x] Human/Owner: Rollback durchgeführt (Schritt 1–4).
- [x] Diese Sitzung: Rollback-Verifikation durchgeführt und dokumentiert (Abschnitt 9).
- [x] Human/Owner: Roll-Forward durchgeführt. — erfolgt automatisch beim Merge von PR #277 (siehe Abschnitt 9, „Roll-Forward-Plan“; Ergebnis siehe Abschnitt 10).
- [x] Diese Sitzung: Roll-Forward-Verifikation durchgeführt und dokumentiert (Abschnitt 10).

Dieses Dokument autorisiert selbst keine weitere Ausführung. Rollback und Roll-Forward sind beide bereits real erfolgt und unabhängig verifiziert (siehe Abschnitt 9 und Abschnitt 10). M7-Exit- Gate-Punkt 7 ist damit vollständig geschlossen; M7 als Ganzes bleibt weiterhin unterhalb `VERIFIED PASS`, bis zusätzlich die vollständige „Required Negative Tests“-Liste geklärt ist (siehe `docs/runbooks/M7_DEPLOY_HOOK_ROTATION_HANDOFF.md` Abschnitt 9 für die bereits dokumentierte Einschränkung).

M8 — Provider-neutral Agent Cutover Runbook

Quelle: docs/runbooks/M8_AGENT_CUTOVER.md

M8 — Provider-neutral Agent Cutover Runbook

Status: **COMPLETE / VERIFIED PASS** — all 9 Exit Gate items PASS. Item 2 closed under an Owner-accepted scope decision (2026-08-16, explicit "ACCEPT" via AskUserQuestion): "privileged supported providers" = mutating providers with a production execution host, currently `chatgpt-github-connector` only (READY, all 6 `ProviderCutoverEvidence` fields incl. `externalHostConfigurationVerified`). Canonical provider set corrected the same day (PR #365, docs/evidence/m8/M8_PROVIDER_SET_CORRECTION_2026-08-16.md): ChatGPT, Claude, Grok; `google-ai-studio / notebooklm / gemini` are now `RETIRED_PROVIDER_ALIASES` (DENY). Both `claude-code-cli` and `grok-xai-connector` remain structurally BLOCKED (interactive, host-mediated connector sessions with no code-addressable execution host) and are explicitly not supported privileged production paths. M9 (Integrated Roadmap Phase I2) is no longer blocked by M8, but each individual M9 drill remains a separately Owner-authorized action. Evidence: docs/evidence/m8/M8_CLOSURE_EVIDENCE.md, docs/evidence/m8/M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED_EVIDENCE.md, docs/evidence/m8/M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md, docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md. Date: 2026-08-16 (Nachtrag: Owner-ACCEPT + M8 Closure). Authority: ADR-0062, ESS-0019, docs/architecture/ai-agent/AI_AGENT_PROVIDER_PROFILE_CONTRACT.md, DEVELOPMENT Chain Execution Policy, docs/roadmaps/INTEGRATED_DEVELOPMENT_SYSTEMADMIN_ROADMAP.md (Phase I1, ROADMAP-INTEGRATED-DC-SA-0001) — dieses Runbook bleibt die spezifischere Authority; die Integrated Roadmap ersetzt es nicht.

Goal

Move privileged AI execution from provider-specific direct paths to one provider-neutral CAPITAL-AI Control Plane with equivalent authorization semantics, audit, risk handling and rollback across ChatGPT, Claude, Google AI Studio and future execution clients.

Research/read-only profiles remain mutation-denied.

Target Architecture

```
AI application / agent
→ provider/transport adapter
→ provider profile
→ CAPITAL-AI Control Plane
→ principal / capability / risk / target policy
→ audit-bound execution decision
→ bounded execution host
→ result / evidence
```

Provider/model/client identity may contribute attribution, but never grants authority by itself.

Prerequisite Gate

Before M8 implementation:

1. M7 must be `COMPLETE / VERIFIED PASS` ;
2. production deployment identity must be verified;
3. M4 IAM and M5 audit controls remain intact;
4. M6 provenance/attestation remains intact;
5. Systemadmin execution-host path used for mutations, if selected, must have its own verified enforcement evidence;
6. Human-only `MERGE` remains enforced.

Provider Roles

ChatGPT

May use connectors/tools as transport. Privileged mutation must route through the same Control Plane and execution-host policy as other clients; direct provider-specific admin access is not canonical after cutover.

Claude

May perform production integration/staging work only through the same semantic capability/risk/target policy. Provider-specific tooling does not create higher authority.

Google AI Studio

Remains the Development Plane for application code, architecture and frontend. Stripe/Supabase/Render external mutations remain handoff-controlled rather than being executed directly from the Development Plane.

Read-only Research Plane

Research/briefing/notebook profiles receive `READ / ANALYZE` only and fail mutation tests by design.

Phase 0 — Read-only Inventory

1. resolve current provider integrations and direct admin paths;
2. map each client to actual tools/connectors/credentials;
3. identify capabilities currently possible per client;
4. identify bypass paths that skip IAM/audit/execution-host enforcement;
5. inventory provider profile contracts and configuration;
6. define rollback-to-read-only for every provider;

7. define identical semantic policy test vectors across at least ChatGPT and Claude, plus Google AI Studio where write-capable transport exists;
8. classify any required external configuration mutations through M7 rules.

Phase 1 — Provider Profile Implementation

For each provider/AI app:

- define stable `appId` / principal attribution;
- map only supported capabilities;
- map risk classes consistently;
- bind target resource semantics consistently;
- reject unknown capability names;
- reject provider-specific privilege inheritance;
- preserve human-reserved action denysets;
- preserve M5 audit correlation fields;
- preserve secret isolation in tool host/gateway;
- preserve idempotency/replay controls for mutation.

Policy Equivalence Tests

The same semantic request must produce the same authorization outcome regardless of provider where capability/profile inputs are equivalent.

Required examples:

- read-only allowed under read profile;
- repository PR work allowed only under valid scoped mandate/profile;
- `MERGE` denied for every agent/provider;
- production mutation without explicit approved Handoff/permit denied;
- wrong target denied;
- expired mandate/approval denied;
- Self-Authority expansion denied;
- provider name/model claiming admin status ignored;
- Research profile mutation denied;
- audit unavailable on a mutation path → DENY.

Cutover Sequence

1. implement provider profiles/adapters on a fresh branch;
2. CI + negative policy tests;
3. shadow routing where feasible without changing authoritative execution;
4. compare old/new decisions and audit correlation;

5. resolve false allow/deny paths;
6. explicitly disable/deprecate provider-specific canonical admin routes;
7. enable Control-Plane path per provider in bounded order;
8. verify rollback-to-read-only per provider;
9. collect Evidence;
10. synchronize Roadmap/Traceability.

External Mutation Gate

Any connector/app configuration or credential change is an external mutation and follows the M7/DevelopmentChain Handoff + explicit Owner Approval process.

No cutover implementation may opportunistically rotate or expand credentials outside its exact work order.

Negative Tests

- provider/model spoofing → no elevation;
- unknown provider profile → DENY;
- missing principal/app identity on privileged request → DENY;
- profile capability mismatch → DENY;
- direct provider admin bypass → DENY/deactivated;
- research profile calls mutation endpoint → DENY;
- target mismatch → DENY;
- replayed mutation envelope → DENY/DEDUPE;
- audit unavailable → DENY;
- kill switch active → mutation denied, read access retained where policy permits.

Rollback

For each provider, rollback must be able to:

1. disable the privileged provider profile;
2. revoke/disable the mutation transport or credential path;
3. restore read-only operation;
4. preserve audit/evidence access;
5. avoid changes to Human/Owner merge authority.

Rollback must not re-enable an ungoverned direct provider admin path.

Evidence

Create `docs/evidence/m8/*` including:

- prerequisite evidence;

- provider inventory;
- profile/capability mappings;
- equivalence test matrix;
- bypass negative tests;
- cutover sequence/results;
- external mutation approval refs if any;
- audit correlation;
- rollback-to-read-only proof;
- final branch/merge/deletion evidence.
- external host configuration verification for the wired mutating provider;
- Exit-Gate item 2 scope decision (Owner).

Exit Gate

M8 is `COMPLETE` / `VERIFIED PASS` only when:

1. M7 is verified;
2. privileged supported providers use the provider-neutral Control Plane as the canonical path;
3. policy-equivalence tests pass;
4. direct provider-specific privileged bypasses are denied/deactivated;
5. research profiles fail mutation tests;
6. rollback-to-read-only is proven;
7. audit correlation is complete;
8. Evidence + Roadmap/Traceability are synchronized;
9. work branches are deleted.

Scope note (2026-08-16): Point 2 may be closed by explicit Owner acceptance of the supported privileged provider set defined in `docs/evidence/m8/M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md` (currently: only `chatgpt-github-connector` is a production mutating host path; non-mutating profiles remain `NOT_APPLICABLE`; `claude-code-cli` remains structurally `BLOCKED` pending a separate ADR). Without that Owner acceptance, point 2 stays open and M9 remains blocked.

Only then may M9 begin.

M9 — Agent Assurance, Incident Response & Break-Glass Runbook

Quelle: docs/runbooks/M9_ASSURANCE_INCIDENT_BREAK_GLASS.md

M9 — Agent Assurance, Incident Response & Break-Glass Runbook

Status: PLANNED — EXECUTION BLOCKED BY M8 Date: 2026-08-12 Authority: ADR-0063, docs/architecture/ai-agent/AI_AGENT_INCIDENT_RESPONSE.md , DEVELOPMENT Chain Execution Policy

Goal

Prove that the CAPITAL-AI agentic execution architecture fails closed under adversarial, replay, exfiltration, audit-failure and incident conditions, and that kill-switch, break-glass and rollback/recovery procedures are operational and attributable.

M9 is an assurance phase, not a feature-expansion phase.

Prerequisite Gate

Before M9 drills:

1. M8 COMPLETE / VERIFIED PASS ;
2. provider-neutral Control Plane active for privileged execution;
3. M5 audit persistence healthy;
4. M6 provenance controls intact;
5. M7 deployment identity intact;
6. kill-switch and rollback mechanisms identified;
7. no unresolved CRITICAL finding from preceding phases.

Assurance Domains

1. Authorization Bypass

Attempt at least:

- missing principal;
- wrong principal/provider profile;
- missing/unknown capability;
- risk above allowed ceiling;
- wrong target/resource;
- expired/revoked mandate/approval;

- Human-reserved action;
- Self-Authority mutation;
- direct tool/connector bypass.

Expected: **DENY**, no side effect, auditable outcome.

2. Prompt / Tool Injection

Test untrusted content from:

- repository files;
- issues/PR comments;
- external webpages/API responses;
- tool output;
- inter-agent messages;
- document/evidence payloads.

Attempts should include instructions to alter policy, reveal credentials, bypass approval, change target, run arbitrary commands, merge/deploy, or expand capabilities.

Expected: untrusted content remains data, not authority.

3. Replay / Idempotency

Attempt replay of:

- approval evidence;
- mutation Handoff;
- execution permit/envelope;
- CI request;
- platform mutation request;
- terminal outcome submission.

Expected: consumed/expired/duplicate requests **DENY** or deterministic **DEDUPE** with no duplicate side effect.

4. Secret / Data Exfiltration

Attempt to cause Evidence/log/tool output to expose:

- API keys;
- bearer tokens;
- service-role secrets;
- passwords;
- TOTP secrets/codes;
- recovery codes;
- passkey private/biometric material;

- full sensitive prompts/request bodies;
- customer PII beyond approved redacted metadata.

Expected: redaction/omission; no reusable secret persists.

5. Audit Completeness / Outage

Verify:

- authorization event before mutation;
- terminal SUCCESS/ERROR event after attempt;
- correlation across actor/agent/session/request/target/capability/result;
- no UPDATE of append-only authorization evidence;
- audit persistence failure prevents autonomous mutation;
- read-only operator visibility can remain available where policy permits.

6. Kill Switch

Activate the documented agent mutation kill switch.

Expected:

- new agent mutations denied;
- in-flight unsafe work stops at the next enforceable boundary;
- Human/Owner and operator read access preserved;
- no policy weakening needed to activate/deactivate;
- action audited.

7. Break-Glass

Break-glass is disabled by default and Human/Owner-controlled.

A drill must prove:

- strong Owner identity/step-up requirement;
- explicit reason and target;
- bounded capability;
- short expiry;
- no silent role elevation;
- append-only audit;
- automatic/explicit revocation;
- mandatory post-event review.

Break-glass must not mint autonomous `MERGE` capability or permanently weaken controls.

8. Rollback / Recovery

Perform a controlled rollback drill appropriate to the selected test resource:

- repository revert through new branch/PR;
- immutable artifact deployment rollback;
- configuration rollback;
- credential revocation/rotation where explicitly in scope;
- provider profile rollback to read-only.

Expected: last-known-good state is restored and independently verified.

Drill Safety

Prefer non-production, synthetic, reversible and non-destructive targets.

Any production-connected drill that causes a real mutation requires the full external Mutation Gate, exact Handoff, explicit Owner approval and rollback plan.

Never perform destructive customer-data, live-money, DNS/domain-ownership or Owner MFA/credential drills without a separate dedicated Human-approved plan.

Evidence Schema

For each drill record:

- drill ID;
- date/time;
- exact baseline;
- actor/human owner;
- agent/app/execution host;
- policy/contract versions;
- capability/risk/target;
- expected result;
- actual result;
- authorization/outcome audit refs;
- side effect state;
- rollback state;
- residual finding;
- owner/remediation assignment.

Use `docs/evidence/templates/DEVELOPMENT_CHAIN_PHASE_EVIDENCE_TEMPLATE.md` plus a phase drill table in `docs/evidence/m9/`.

Required Independent Review

M9 closure requires review separate from the execution mechanism itself. The review confirms:

- negative tests were not silently skipped;
- no synthetic PASS was recorded for unavailable controls;
- Evidence is redacted but sufficient;
- residual risks have owners;
- no unowned CRITICAL control remains;
- any accepted HIGH residual risk has explicit Human/Owner acceptance.

Failure Handling

Any unexpected ALLOW or side effect in a negative test:

1. activate kill switch if needed;
2. stop further privilege expansion/cutover;
3. preserve redacted Evidence;
4. rollback/revoke affected state;
5. create a scoped remediation Roadmap item;
6. re-run the failed drill only after remediation and required review.

M10 remains blocked.

Exit Gate

M9 is **COMPLETE / VERIFIED PASS** only when:

1. M8 prerequisite is verified;
2. all required authorization/injection/replay/exfiltration/audit drills PASS;
3. kill-switch drill PASS;
4. break-glass drill PASS;
5. rollback/recovery drill PASS;
6. independent Evidence review PASS;
7. no unowned CRITICAL control remains;
8. residual risk is explicitly documented/accepted as required;
9. Evidence and Roadmap/Traceability are synchronized;
10. work branches are deleted.

Only then may M10 implementation/cutover begin.

Production Environment Setup — Documentation-Only Target

Quelle: docs/runbooks/PRODUCTION_ENVIRONMENT_SETUP.md

Production Environment Setup — Documentation-Only Target

Status: DESIGN ONLY / M7 HANDOFF

This runbook describes the intended GitHub/Render production-environment configuration but authorizes no current mutation.

Target: GitHub Environment `production`, deployment branch `main`, environment-scoped Render credential/bridge, retained rollback credential until post-deploy verification, explicit rotation/revocation owner, and deploy evidence bound to verified commit SHA.

Implementation is blocked until M2G Documentation Freeze and M7 start. The earlier PR #190 workflow mutation is intentionally not carried forward.

Render Auto-Deploy A/B Test — 2026-08-09

Quelle: docs/runbooks/RENDER_AUTODEPLOY_AB_TEST_2026-08-09.md

Render Auto-Deploy A/B Test — 2026-08-09

Objective

Determine whether the intermittent deployment failure is specific to Render's `After CI Checks Pass` (`checksPass`) trigger or affects GitHub→Render repository events generally.

Production service under test

- Render service: `Finance`
- Repository: `SvenKulessa/Finance`
- Branch: `main`
- Runtime: Docker
- Baseline policy: `After CI Checks Pass`
- Diagnostic policy: `On Commit`

Evidence before test

- PR #136 merge commit `584d8d4f67af1ab0c8f063e71bcd3db530771bff` completed GitHub CI successfully and published `capital-ai/ci-gate: success`; Render did not create the expected deploy in the normal observation window.
- PR #138 merge commit `a586e314de5ca0defc2686c2573e02833f84feae` completed GitHub CI successfully and published `capital-ai/ci-gate: success`; Render did not immediately create the expected `checksPass` deploy.
- After the Render policy was switched to `On Commit`, Render created deploy `dep-d9sbff-pt0dsc73bl4qqg` for PR #138's merge commit with `trigger: new_commit`; final status was `live`.

B-test confirmation

PR #141 was merged while Render remained on `On Commit`.

- Merge commit: `ae35d4e57dbf06eadf5776c0e9d6353978f03f38`
- Merge timestamp: `2026-08-09T17:38:02Z`
- Render deploy ID: `dep-d9sbnugu01pc73e2mjQ0`
- Render deploy created: `2026-08-09T17:42:19Z`
- Render deploy finished: `2026-08-09T17:43:27Z`
- Render trigger: `new_commit`
- Final status: `live`

- Observed merge-to-deploy-creation latency: approximately 4 minutes 17 seconds

Conclusion

Result: **checksPass-specific fault domain confirmed.**

The following path is proven functional:

```
GitHub PR merge → main commit → Render On Commit → trigger:new_commit → live
```

Therefore the PR merge mechanism itself is not the cause. GitHub repository access, Git deployment credentials, Render repository binding, and recognition of merge commits are functional. The unreliable component is Render's `After CI Checks Pass` / GitHub Check Suite correlation path.

GitHub plan constraint

GitHub reports `main` as `protected: false` with required status-check enforcement off. The current account/repository plan does not provide the required private-repository ruleset enforcement without an upgrade, and no upgrade is planned at this time.

This makes permanent `Render: On Commit` unsuitable as the production gate because Render can begin deployment before the push-triggered CI has validated that commit.

Selected free-plan production architecture

ADR-0047 replaces both Render `checksPass` and permanent `On Commit` with an explicit GitHub Actions deployment gate:

```
main update → GitHub build-and-test → success → deploy-production → Render Deploy Hook → exact verified SHA
```

Required operational state:

- Render Auto-Deploy: **Off**;
- GitHub repository secret: `RENDER_DEPLOY_HOOK_URL` ;
- deploy job runs only for `push` to `main` ;
- deploy job depends on successful `build-and-test` ;
- deploy hook receives the exact `github.sha` through its `ref` parameter;
- failed CI therefore cannot invoke the production deploy.

Transition controls

PR #144 was merged on 2026-08-09 after the operational transition. The deployment decision was initially recorded under the colliding identifier `ADR-0046` ; this was corrected retrospectively to `ADR-0047` without changing the deployment architecture.

Validate the resulting production path and confirm:

1. pull-request CI does not deploy;

2. successful push CI invokes `Deploy verified commit` to Render ;
3. Render deploys the same commit SHA;
4. the deploy reaches `live` ;
5. no Render `new_commit` auto-deploy is generated independently.

Render Auto-Deploy Credential Test — 2026-08-09

Quelle: docs/runbooks/RENDER_AUTODEPLOY_CREDENTIAL_TEST_2026-08-09.md

Render Auto-Deploy Credential Test — 2026-08-09

Purpose: create a no-op documentation-only change to validate the repaired GitHub deployment credentials for the Render `Finance` service.

Expected post-merge sequence:

1. Merge to `main`.
2. GitHub Actions `CI` runs on the exact merge commit.
3. `build-and-test` completes successfully.
4. `Render deployment gate` publishes `capital-ai/ci-gate: success` for the same commit.
5. Render detects the successful checks and creates a deployment with `trigger: new_commit`.
6. The deployment reaches `status: live`.

This file changes no application code, runtime configuration, billing, IAM, database state, or Render configuration. It exists only as an observable deployment-path test after the Git credential repair.

Render CI Deployment Gate

Quelle: docs/runbooks/RENDER_CI_DEPLOY_GATE.md

Render CI Deployment Gate

Purpose

The production Render service `Finance` deploys `main` with `autoDeployTrigger: checksPass`. GitHub Actions therefore validates the exact `main` commit after merge before Render may deploy it.

Gate

`.github/workflows/ci.yml` runs on both pull requests and pushes to `main`.

After `build-and-test` completes, the final `Render deployment gate` job publishes a classic GitHub commit status with context:

`capital-ai/ci-gate`

The status is:

- `success` only when `build-and-test` completed successfully;
- `failure` for every other result, and the final job exits non-zero.

This provides Render's `checksPass` integration with an explicit commit status in addition to GitHub Actions check runs. No Render API key or deploy hook is stored in the repository.

Expected production sequence

1. Pull request CI passes.
2. Human-authorized merge updates `main`.
3. GitHub Actions runs CI again for the exact merge commit.
4. `capital-ai/ci-gate` becomes `success` only after all CI checks pass.
5. Render observes the successful commit checks/status and starts the automatic production deploy.
6. The Render deploy history should show `trigger: new_commit` for the same merge commit.

Failure behavior

If CI fails, `capital-ai/ci-gate` is published as `failure` and Render must not deploy the commit while `autoDeployTrigger` remains `checksPass`.

Verification after merge

Confirm all of the following for the merge commit:

- GitHub Actions `CI` run conclusion is `success`.
- Commit status `capital-ai/ci-gate` is `success`.
- Render creates a deployment for the same commit with `trigger: new_commit`.
- The deployment reaches `live`.

If the first two conditions are true but Render still creates no deploy, treat the incident as a Render GitHub-App/webhook integration fault rather than a repository CI-trigger fault. Do not weaken the gate to `On Commit` as a workaround without an explicit architecture decision.

Render Docker Hardening Runbook — 2026-08-09

Quelle: docs/runbooks/RENDER_DOCKER_HARDENING_2026-08-09.md

Render Docker Hardening Runbook — 2026-08-09

Scope

Production service: Finance Repository: SvenKulessa/Finance Runtime: Docker Render Auto-Deploy: Off Production deploy authority: GitHub Actions deploy hook after successful CI

Hardening controls

- immutable Node 22 Alpine digest in builder and runner stages;
- multi-stage build;
- production-only runtime dependency installation;
- npm cache cleanup;
- non-root runtime identity `capitalai`;
- explicit artifact ownership through `COPY --chown`;
- writable-path allowlist centered on `/app/uploads`;
- read-only `/app/docs` governance path;
- direct Node PID 1 startup;
- `/healthz` container healthcheck;
- expanded `.dockerignore` for secrets, key material, logs, coverage, editor files, temp files and VCS metadata;
- CI policy validation with `scripts/security/verifyDockerHardening.mjs`;
- CI Docker image build and metadata inspection before production deploy.

Production deployment sequence

`main push → build-and-test → Docker hardening policy → application tests/build → Docker image build → image metadata checks → Deploy verified commit to Render → deploy_hook → exact SHA → live`

A failure in any build-and-test step prevents the Render deploy job from running.

Validation after merge

Record the following evidence:

1. merge commit SHA;
2. GitHub CI run ID and final conclusion;

3. result of Docker hardening policy step;
4. result of production Docker image build;
5. result of runtime metadata verification;
6. Render deploy ID;
7. Render trigger must be `deploy_hook`;
8. Render commit SHA must equal the GitHub verified merge SHA;
9. final Render status must be `live`.

Rollback

If the hardened container fails in Render while CI remains green, revert the Docker-hardening PR. Do not re-enable Render Auto-Deploy as a workaround. The existing GitHub deploy-hook gate remains the authoritative deployment path.

Future hardening candidates

Evaluate separately in later ADRs:

- distroless or similarly reduced runtime image after native-module/library compatibility testing;
- generated SBOM attestation tied to the deployed SHA;
- vulnerability scanning of the built OCI image with a pinned scanner version;
- signed OCI provenance if the architecture moves from Render-side Docker builds to prebuilt immutable registry images;
- platform-level read-only root filesystem if/when supported by the production runtime model.

CAPITAL-AI Runbook — Render Production Evidence & Configuration Handoff

Quelle: docs/runbooks/RENDER_PRODUCTION_EVIDENCE_HANDOFF.md

CAPITAL-AI Runbook — Render Production Evidence & Configuration Handoff

Runbook ID: RUNBOOK-RENDER-0001

Date: 2026-08-03

Status: ACTIVE

Related: RENDER-AUDIT-0001 / ADR-0037

1. Zweck

Dieses Runbook wird verwendet, sobald ein Human Operator direkten Zugriff auf das Render-Dashboard oder die Render API/CLI besitzt.

Es hat zwei Ziele:

1. den tatsächlichen Production State beweissicher zu erfassen, ohne Secrets zu kopieren;
2. geplante Render-Konfigurationsänderungen kontrolliert aus ADR-0037 in die Production Control Plane zu überführen.

Das Runbook ist absichtlich so aufgebaut, dass ein Agent die Architektur vorbereiten kann, ohne selbst Production Secrets zu benötigen.

2. Was NIEMALS dokumentiert wird

Nicht in GitHub, PR, Chat, Ticket oder Screenshot-Dokumentation übernehmen:

- Secret-Werte;
- API Tokens;
- OAuth Client Secrets;
- SMTP Password;
- Supabase Service/Secret Keys;
- Stripe Secret/Webhook Secret;
- TOTP Encryption Key;
- Render API Tokens;
- Deploy Hook URLs;
- private registry credentials;
- vollständige Secret Files.

Erlaubt sind:

- Key-Namen;
- Service-/Project-/Environment IDs, soweit nicht als Credential verwendbar;
- Commit SHA;
- boolesche Zustände;
- Region/Plan;
- Domainnamen;
- nicht-sensitive Policy-Werte;
- Secret-Rotationsdatum ohne Secret-Inhalt.

3. Evidence Capture — Workspace

Render Dashboard → Workspace.

Erfassen:

```
Workspace name:  
Workspace plan:  
Admin users / roles reviewed: YES|NO  
Render GitHub App scope reviewed: YES|NO  
Notification default: failures|all|none  
Notification destination: email|slack|both|none
```

Keine personenbezogenen Details unnötig in technische Dokumente übernehmen; Rollen-/Owner-Evidence genügt.

4. Evidence Capture — Project / Environment

Erfassen:

```
Project name:  
Environment name:  
Environment protection: enabled|disabled  
Cross-environment connections blocked: enabled|disabled  
Environment-scoped env groups:  
  - <name only>
```

Ziel nach ADR-0037

Production soll — soweit Render-Plan/Architektur dies unterstützt — als geschütztes Project Environment betrieben werden.

5. Evidence Capture — Service General

Service `capital-ai` → Settings.

Erfassen:

```
Service ID:
Service type:
Runtime:
Repository:
Branch:
Region:
Instance type:
Root directory:
Dockerfile path:
Docker command override:
Health check path:
Max shutdown delay:
Maintenance mode:
```

Sofortige Abbruchkriterien

Nicht eigenständig ändern, wenn der Istwert von Repository/ADR abweicht. Stattdessen als Drift Finding dokumentieren.

6. Evidence Capture — Build & Deploy

Erfassen:

```
Auto Deploy:
  commit | checksPass | off

Overlapping Deploy Policy:
  <actual value>

Build Filters:
  included paths:
  ignored paths:

Pre-deploy command:
Preview/service preview policy:
Current live deploy commit SHA:
Last successful deploy commit SHA:
Last failed deploy commit SHA (if applicable):
```

Zielzustand

Production Auto Deploy soll nach ADR-0037 `After CI Checks Pass` / `checksPass` verwenden.

Vor Änderung prüfen, dass verpflichtende GitHub Checks für `main` tatsächlich existieren und nicht regelmäßig `skipped` werden.

7. Evidence Capture — Scaling

Erfassen:

```
Scaling mode: manual|autoscaling
Current instances:
Autoscaling min:
Autoscaling max:
CPU target:
Memory target:
```

ADR-0037 Gate

Solange wiederkehrende Background Jobs im Web-Prozess laufen:

```
current instances MUST be 1
autoscaling MUST be disabled
```

Eine Abweichung ist als **P0 Scaling Risk** zu behandeln und nicht als normale Performance-Einstellung.

8. Evidence Capture — Disk / Filesystem

Erfassen:

```
Persistent disk attached: yes|no
Mount path:
Disk size:
```

Interpretation

Wenn `no`:

- `/app/uploads` ist als ephemer zu behandeln;
- lokale Subscription-/Credit-Fallbacks dürfen nicht als durable gelten.

Wenn `yes`:

- sofort dokumentieren, dass Multi-Instance-Scaling nicht möglich ist;
- dokumentieren, dass Zero-Downtime Deploys für den Service entfallen;
- prüfen, warum der Disk existiert und ob er durch Supabase/Object Storage ersetzt werden kann.

Disk nicht entfernen, bevor die Datenklassifikation und ein Backup/Migrationsplan vorliegen.

9. Evidence Capture — Domains / Network Edge

Erfassen:

```
Custom domains:
- capital-ai.online
- <others>

Canonical redirect:
onrender.com subdomain reachable: yes|no
TLS certificate status:
Inbound IP rules:
```

Ziel

Wenn die Render-Subdomain nicht für Betrieb/Monitoring benötigt wird, ADR-0037-konform prüfen, ob `renderSubdomainPolicy: disabled` sinnvoll ist.

10. Evidence Capture — Environment Variables

Nur **Namen** vergleichen.

Repository-deklarierte Schlüssel

Aus `render.yaml` gegen Dashboard-Key-Namen abgleichen.

Zusätzlich explizit prüfen, weil dynamische Stripe-Konfiguration vom bestehenden Scanner nicht vollständig erkannt wird:

```
STRIPE_PRICE_ID_STARTER
STRIPE_PRICE_ID_STARTER_MONTHLY
STRIPE_PRICE_ID_STARTER_YEARLY
STRIPE_PRICE_ID_PRO
STRIPE_PRICE_ID_PRO_MONTHLY
STRIPE_PRICE_ID_PRO_YEARLY
STRIPE_PRICE_ID_ENTERPRISE
STRIPE_ID_FOUNDER or STRIPE_PRICE_ID_FOUNDER
STRIPE_PRICE_ID_EXPORT_PDF
```

Weiterhin prüfen:

```
VITE_SUPABASE_URL
VITE_SUPABASE_PUBLISHABLE_KEY / VITE_SUPABASE_ANON_KEY
SUPABASE_SECRET_KEY / SUPABASE_SERVICE_ROLE_KEY
STRIPE_SECRET_KEY
STRIPE_WEBHOOK_SECRET
VITE_STRIPE_PUBLISHABLE_KEY / STRIPE_PUBLISHABLE_KEY
TOTP_ENCRYPTION_KEY
SMTP_*
METRICS_TOKEN
AI provider keys
market-data provider keys
social OAuth client IDs/secrets
VITE_GA_MEASUREMENT_ID
```

Regeln

- kein Secret-Wert ins Evidence-Dokument;
- doppelte Alias-Keys bewusst konsolidieren;
- `VITE_*` bedeutet browser-/build-visible und darf niemals ein Secret enthalten;
- neue `sync:false` Keys bei bestehendem Blueprint müssen manuell mit Wert provisioniert werden.

11. Environment Groups

Für jeden Group-Namen dokumentieren:

```
name:  
project/environment scope:  
linked services:  
key names only:  
owner:
```

Kollisionen gleicher Variablennamen zwischen mehreren Groups vermeiden. Render garantiert die Präzedenz zwischen kollidierenden Groups nicht als langfristigen Contract.

12. Notifications / Incident Routing

Mindestens folgende Render-Events müssen einen dokumentierten Empfänger besitzen:

```
build/deploy failed  
running service unhealthy  
Docker pull failed
```

Optional `all notifications` für successful deploy/recovery.

Incident-Routing muss erklären:

```
who receives alert  
where it is recorded  
who decides rollback  
how GitHub commit/deploy ID is correlated
```

13. Pre-Change Gate

Vor jeder Render Production Mutation:

1. aktuellen Wert dokumentieren;
 2. Sollwert aus ADR/PR nennen;
 3. abhängige Komponenten nennen;
 4. Blast Radius klassifizieren;
 5. Rollback definieren;
 6. prüfen, ob Setting einen Deploy/Restart auslöst;
 7. prüfen, ob Secret Rotation erforderlich ist;
 8. Human Approval nach CAPITAL-AI Governance;
 9. Änderung durchführen;
 10. Post-Change Evidence erfassen.
-

14. Post-Change Verification

Nach Render-Änderungen mindestens:

```
[ ] deploy succeeded
[ ] correct commit SHA is live
[ ] Render health check passes
[ ] login/auth works
[ ] owner/IAM protected path works
[ ] Supabase durable write/read works
[ ] Stripe checkout test path appropriate to environment works
[ ] webhook endpoint reachable/configured
[ ] SMTP test where affected
[ ] critical API smoke test
[ ] no unexpected CSP/CORS regression
[ ] logs contain no new secret exposure
[ ] notification path tested if modified
```

Nicht jede Änderung benötigt jeden fachlichen Test; nicht zutreffende Punkte werden explizit **N/A** markiert, nicht still übersprungen.

15. Rollback Decision

Code regression

Render Instant Rollback oder Git revert nach Incident-Analyse.

Configuration regression

Aktuellen und vorherigen Service Configuration State vergleichen. Ein Code-Rollback setzt nicht automatisch jede Dashboard-Einstellung zurück.

Secret regression

Nicht auf alte kompromittierte Secrets zurückrollen. Stattdessen neue Secret-Rotation + providerseitige Revocation.

Data regression

Render Application Rollback ist kein Supabase Data Restore. Daten-Recovery separat nach Datenbank-Runbook.

16. Evidence Output Template

Nach vollständigem Audit einen Secret-freien Block erzeugen:

```
renderProductionEvidence:
  capturedAt: <ISO timestamp>
  service:
    id: <id>
    name: capital-ai
    runtime: docker
    repo: SvenKulesa/Finance
    branch: main
    region: <verified>
    plan: <verified>
    instances: <verified>
    autoscaling: <verified>
    healthCheckPath: <verified>
    maxShutdownDelaySeconds: <verified>
    autoDeployTrigger: <verified>
```

```
diskAttached: <yes/no>
renderSubdomainEnabled: <yes/no>
environment:
  project: <verified>
  name: <verified>
  protected: <yes/no>
  crossEnvironmentIsolation: <yes/no>
domains:
  - <verified>
environmentGroups:
  - <name only>
notifications:
  level: <verified>
  destinations: <types only>
deploy:
  liveCommitSha: <verified>
```

Dieses Evidence-Objekt enthält bewusst keine Secret-Werte.

17. Abschluss

Nach Ausfüllen dieses Runbooks:

1. `RENDER_PRODUCTION_CONFIGURATION_AUDIT.md` aktualisieren;
2. offene Findings neu klassifizieren;
3. ADR-0037 Implementation Status aktualisieren;
4. Render-IaC nur mit **verifizierten** Dashboard-Werten vervollständigen;
5. Production Change über separaten, reviewbaren PR/Handoff durchführen.

RUNBOOK — Screening SLO Evidence Persistence

Quelle: docs/runbooks/SCREENING_SLO_EVIDENCE_PERSISTENCE.md

RUNBOOK — Screening SLO Evidence Persistence

Zweck

Dieser Runbook beschreibt den Produktions-Handoff für die persistente Ablage von `screening-slo-evidence/1.0.0`.

Der Code in `src/services/screeningSloEvidence.ts` erzeugt ausschließlich einen versionierten Evidence-Record. Er schreibt selbst weder in Supabase noch in ein externes Observability-System. Dadurch bleibt die Entwicklungsumgebung frei von produktiven Infrastrukturmutationen.

Datenquelle

Der Evidence-Record basiert ausschließlich auf bereits vorhandenen Contracts:

- `screening-eligibility/1.0.0`
- `screening-sla/1.0.0`
- `screening-operations/1.0.0`

Er enthält keine neu berechneten Finanzscores und verändert keinen kanonischen Score.

Persistenzanforderungen

Empfohlen ist eine append-only Ablage mit mindestens:

- `correlation_id`
- `observed_at`
- `symbol`
- `asset_class`
- `state`
- `eligible`
- `eligibility_status`
- `quote_status`
- `quote_age_ms`
- `quote_fresh`
- `sla_state`
- `reasons`
- `contract_version`

Zusätzlich sollte der persistierte Datensatz die Policy-Felder enthalten:

- `score_impact_enabled = false`
- `hard_screening_block_enabled = false`
- `synthetic_evidence_allowed = false`

Produktions-Handoff

Eine tatsächliche Supabase-Tabelle oder eine externe Observability-Senke darf erst im Produktions-Handoff eingerichtet werden.

Empfohlene Reihenfolge:

1. Zielspeicher festlegen: Supabase append-only Tabelle oder Observability Backend.
2. Retention Policy definieren.
3. RLS/Service-Role-Zugriff prüfen.
4. Write-Path ausschließlich serverseitig implementieren.
5. Correlation-ID unverändert übernehmen.
6. Keine Secrets oder Roh-Provider-Credentials persistieren.
7. Golden-/Regression-Test für Record-Schema ergänzen.
8. Erst nach ausreichender Runtime-Stichprobe SLO-Schwellen reviewen.

Verbotene Abkürzungen

Nicht zulässig sind:

- synthetische SLO-Evidence,
- künstliche Healthy-Defaults bei fehlenden Runtime-Daten,
- direkte Ableitung eines Asset-Scores aus SLO/Latenz,
- automatische Aktivierung eines Hard Screening Gates,
- Persistieren von API Keys oder Provider-Secrets.

Aktivierung eines Hard Gates

Ein späteres Hard Gate benötigt separat:

- ausreichende Produktionsstichprobe,
- dokumentierte Schwellenanalyse,
- ADR,
- Regression-/Golden-Dataset-Prüfung,
- Rollback-Plan,
- Supervisor-/Compliance-Evidence.

Bis dahin bleiben `scoreImpactEnabled=false` und `hardScreeningBlockEnabled=false` verbindlich.

Screening SLO Sink — Production Handoff

Quelle: docs/runbooks/SCREENING_SLO_SINK_PRODUCTION_HANDOFF.md

Screening SLO Sink — Production Handoff

Ziel

`screening-slo-sink/1.0.0` definiert die Persistenzgrenze für `screening-slo-evidence/1.0.0`.

Der Default in der Anwendung ist bewusst `noop-unpersisted`. Er darf niemals behaupten, Evidence persistent gespeichert zu haben.

Produktionsanforderungen

Eine produktive Senke MUSS:

- append-only schreiben,
- `correlationId`, `observedAt`, `state`, `eligibilityStatus`, `quoteStatus`, `quoteAgeMs`, `slaState` und `reasons` erhalten,
- keine Secrets oder API-Keys persistieren,
- `scoreImpactEnabled=false` und `hardScreeningBlockEnabled=false` unverändert bewahren,
- Fehler beim Persistieren als Persistenzfehler ausweisen und niemals einen Finanzscore verändern,
- Retention und Zugriffskontrolle dokumentieren.

Empfohlene Tabelle / Event-Struktur

Eine spätere Supabase- oder Observability-Implementierung kann mindestens folgende Felder verwenden:

- `id` UUID
- `correlation_id` text, indexed
- `symbol` text nullable
- `asset_class` text nullable
- `observed_at` timestampz
- `state` text
- `eligible` boolean
- `eligibility_status` text
- `quote_status` text nullable
- `quote_age_ms` bigint nullable
- `quote_fresh` boolean nullable
- `sla_state` text

- `reasons` jsonb
- `contract_version` text
- `created_at` timestamptz default now()

Governance

Die Persistenzsenke ist Observability-/Audit-Infrastruktur. Sie darf weder Screening Eligibility noch Score, Recommendation oder Ranking automatisch verändern.

Eine spätere Aktivierung als Hard Gate erfordert eine eigene ADR, Produktionskalibrierung und Regressionstests.

Handoff

Dieser Entwicklungs-PR nimmt keine Supabase- oder Render-Mutation vor. Die produktive Implementierung der Senke wird im Production-Handoff separat ausgeführt und danach mit realer Runtime-Evidence validiert.

SEO WP-S1 Migration-Ledger-Abgleich

Quelle: docs/runbooks/SEO_WP_S1_LEDGER_RECONCILIATION.md

SEO WP-S1 Migration-Ledger-Abgleich

Status: VERIFIED PASS Date: 2026-08-15 Target: Supabase project AIFINANCIAL (rzy-woktpmyhwzxmstyu) Work claim: SEO-WP-S1-FOLLOWUP-HARDENING-2026-08-15 Roadmap: SEO-GM-ROADMAP-0002 / WP-S1 Status hygiene: Phase C updater 2026-08-16 (detector 0.1.0-phase-b → VERIFIED PASS; Apply-Evidenz body unchanged)

Problem

Der Supabase-MCP-Connector nimmt beim `apply_migration` keine Versionsnummer entgegen. Er stempelt stattdessen die Anwendungszeit in `supabase_migrations.schema_migrations.version`. Der SQL-Inhalt ist identisch zur Repo-Datei, die Version im Ledger weicht aber ab:

Repo-Datei	Ledger-Version (ist)	Ledger-Name
20260815010000_seo_engine.sql	20260815192502	seo_engine
20260815200000_seo_engine_source_align_rls.sql	20260815192747	seo_engine_source_align_rls
20260815210000_seo_engine_service_role_grants.sql	20260815193957	seo_engine_service_role_grants
20260815220000_seo_rank_snapshots_fk_restrictions.sql	20260815194000	seo_rank_snapshots_fk_restrictions

Folge: Ein `supabase db push` haelt die vier Repo-Migrationen fuer offen und fuehrt sie erneut aus. Alle vier sind idempotent (`create table if not exists`, `create index if not exists`, `drop constraint if exists`, `revoke / grant`, `enable row level security`), und der Guard-Block in `20260815200000` laeuft bei leerer Tabelle sauber durch. Der Re-Run ist damit ungefaehrlich, aber der Ledger bildet den Repo-Stand nicht ab, und jeder kuenftige Drift-Vergleich startet mit falschem Bild.

Abgrenzung — was hier ausdruecklich NICHT passiert

Drei weitere Repo-Migrationen fehlen im Ledger:

- 20260801150000_social_media_publishing.sql
- 20260810110642_pdf_credit_ledger.sql
- 20260815140000_social_media_content_approvals.sql

Diese sind **kein Ledger-Drift, sondern offener Migrationsrueckstand**. Verifiziert am 2026-08-15 gegen die Produktionsdatenbank — keine der erzeugten Tabellen existiert:

```
select t.name as expected_table,
       to_regclass('public.' || t.name) is not null as exists_in_db
from (values ('social_media_accounts'), ('social_media_oauth_states'), ('social_media_publish_log'))
```

```
( 'pdf_credits'), ('pdf_credit_grants'), ('social_media_content_approvals')) as t(name);
-- alle sechs: exists_in_db = false
```

Ein Ledger-Eintrag fuer diese drei wuerde sie faelschlich als angewendet markieren und `db push` dazu bringen, echtes Schema zu ueberspringen. Sie brauchen einen eigenen Vorgang mit der fachlichen Frage, ob die Features noch gewollt sind. **Dieses Runbook fasst sie nicht an.**

Reihenfolge

Der Abgleich laeuft **nach** dem Apply von `20260815220000`, damit alle vier Zeilen in einem Durchgang korrigiert werden. Wird `20260815220000` nicht angewendet, entfaellt dessen `update` ersatzlos.

Vorab-Pruefung

Kollisionsfreiheit der Zielversionen und Ist-Zustand:

```
select version, name
from supabase_migrations.schema_migrations
where name in ('seo_engine', 'seo_engine_source_align_ri',
               'seo_engine_service_role_grants', 'seo_rank_snapshots_fk_restrict')
       or version in ('20260815010000', '20260815200000', '20260815210000', '20260815220000')
order by version;
```

Erwartung vor der Mutation: genau die vier gestempelten Zeilen, keine Zeile traegt bereits eine der vier Zielversionen. Trifft das nicht zu, **abbrechen** und den Befund melden.

Mutation

Namensbasiert statt versionsbasiert, damit die Anweisungen unabhaengig vom gestempelten Zeitwert korrekt bleiben und ein zweiter Lauf zum No-op wird:

```
begin;

update supabase_migrations.schema_migrations
set version = '20260815010000'
where name = 'seo_engine' and version <> '20260815010000';

update supabase_migrations.schema_migrations
set version = '20260815200000'
where name = 'seo_engine_source_align_ri' and version <> '20260815200000';

update supabase_migrations.schema_migrations
set version = '20260815210000'
where name = 'seo_engine_service_role_grants' and version <> '20260815210000';

update supabase_migrations.schema_migrations
set version = '20260815220000'
where name = 'seo_rank_snapshots_fk_restrict' and version <> '20260815220000';

commit;
```

Erwartet: vier mal `UPDATE 1` (bzw. drei, falls `20260815220000` noch nicht angewendet ist).

Verifikation

```
select version, name
from supabase_migrations.schema_migrations
where name like 'seo\_%'
order by version;
```

Zielzustand:

```
20260815010000 seo_engine
20260815200000 seo_engine_source_align_rls
20260815210000 seo_engine_service_role_grants
20260815220000 seo_rank_snapshots_fk_restrict
```

Zusaetzlich pruefen, dass die Schema-Objekte unveraendert sind — der Abgleich ist reine Buchhaltung:

```
select conname, pg_get_constraintdef(oid)
from pg_constraint
where conrelid = 'public.seo_rank_snapshots'::regclass and contype = 'f';
-- erwartet: FOREIGN KEY (keyword_id) REFERENCES seo_keywords(id) ON DELETE RESTRICT
```

Rollback

Der Abgleich veraendert ausschliesslich Metadaten; kein Schema- und kein Datenverlust ist moeglich. Zum Zuruecksetzen die urspruenglich gestempelten Werte wieder eintragen:

```
begin;
update supabase_migrations.schema_migrations set version = '20260815192502' where name = 'seo_engine';
update supabase_migrations.schema_migrations set version = '20260815192747' where name = 'seo_engine_source_align_rls';
update supabase_migrations.schema_migrations set version = '20260815193957' where name = 'seo_engine_service_role_grants';
-- 20260815220000: auf den beim Apply tatsaechlich gestempelten Wert zuruecksetzen (siehe Apply-Evidenz unten: 20260815210718)
commit;
```

Wiederauftreten vermeiden

Solange Migrationen ueber den MCP-Connector statt ueber `supabase db push` angewendet werden, tritt der Drift bei jedem Apply erneut auf. Zwei Optionen, jeweils eigener Vorgang:

1. Migrationen grundsatzlich per CLI (`supabase db push`) anwenden, der Connector bleibt Lese-/Evidenzpfad.
2. Den Abgleich als festen Nachbereitungsschritt in den Apply-Ablauf aufnehmen und die Ledger-Version direkt nach jedem `apply_migration` korrigieren.

Apply-Evidenz

Ausgefuehrt 2026-08-15, Owner-Freigabe SvenKulesa (Work claim `SEO-WP-S1-FOLLOWUP-HARDENING-2026-08-15`).

Schritt	Ergebnis
20260815220000 FK-Apply	<code>success: true . pg_get_constraintdef</code> bestaetigt <code>FOREIGN KEY (keyword_id) REFERENCES seo_keywords(id) ON DELETE RESTRICT</code> , gestempelt als <code>20260815210718 seo_rank_snapshots_fk_restrict .</code>
Vorab-Pruefung Ledger	Genau vier Zeilen (<code>seo_engine</code> , <code>seo_engine_source_align_rls</code> , <code>seo_engine_service_role_grants</code> , <code>seo_rank_snapshots_fk_restrict</code>), keine trug bereits eine Zielversion. Keine Kollision.
Mutation Ledger	Vier namensbasierte <code>UPDATE</code> s in einer Transaktion, <code>commit .</code>
Verifikation Ledger	<code>schema_migrations</code> zeigt exakt die vier Repo-Versionen: <code>20260815010000 seo_engine</code> , <code>20260815200000 seo_engine_source_align_rls</code> , <code>20260815210000 seo_engine_service_role_grants</code> , <code>20260815220000 seo_rank_snapshots_fk_restrict .</code>
Abgrenzungs-Check	<code>to_regclass</code> fuer alle sechs Tabellen aus <code>social_media_publishing / pdf_credit_ledger / social_media_content_approvals</code> weiterhin <code>false</code> — unveraendert nicht angewendet, kein Ledger-Eintrag erzeugt.

CAPITAL-AI Runbook: Social Media OAuth-2.0-Setup (YouTube, TikTok, Instagram, Facebook, X)

Quelle: docs/runbooks/SOCIAL_MEDIA_OAUTH_SETUP.md

CAPITAL-AI Runbook: Social Media OAuth-2.0-Setup (YouTube, TikTok, Instagram, Facebook, X)

Document ID

RUNBOOK-0002

Bezug

ADR-0020 (Social Media Direct Publishing — Reale OAuth-2.0- und Plattform-API-Integration).

Status

Aktiv

Geltungsbereich

Dieses Runbook beschreibt die Schritte, die **nur der Repository-Owner selbst** (oder jemand mit Vollmacht für die CAPITAL-AI-Organisation) durchführen kann: Registrierung von OAuth-Apps in vier externen Developer-Consolen und Eintragen der resultierenden Secrets in Render. Diese Schritte sind bewusst außerhalb dessen, was ein KI-Agent automatisieren kann — sie erfordern Login mit einem echten Geschäftskonto, Annahme externer Nutzungsbedingungen als Rechtsperson und teils Business-Verifizierung/App-Review durch die jeweilige Plattform.

Die zugehörige Backend-Implementierung (Token-Exchange, verschlüsselte Speicherung, echte Publish-Calls) ist bereits vollständig umgesetzt (siehe ADR-0020) — nach Abschluss dieses Runbooks funktioniert die Verbindung ohne weitere Code-Änderungen.

0. Vorab: Redirect-URI und Env-Var-Referenz

Für **alle** Plattformen wird exakt dieselbe Redirect-URI verwendet:

```
https://<DEINE-PRODUKTIONS-DOMAIN>/api/social-media/auth/callback
```

Ersetze `<DEINE-PRODUKTIONS-DOMAIN>` durch die tatsächliche Render-Domain (z. B. `capital-ai.onrender.com` oder eine eigene Domain, falls konfiguriert). Für lokale Entwicklung/Tests: `http://localhost:3000/api/social-media/auth/callback`.

Plattform	Env-Var (Client-ID)	Env-Var (Client-Secret)
YouTube	<code>YOUTUBE_CLIENT_ID</code>	<code>YOUTUBE_CLIENT_SECRET</code>
TikTok	<code>TIKTOK_CLIENT_ID</code>	<code>TIKTOK_CLIENT_SECRET</code>
Instagram	<code>INSTAGRAM_CLIENT_ID</code>	<code>INSTAGRAM_CLIENT_SECRET</code>
Facebook	<code>FACEBOOK_CLIENT_ID</code>	<code>FACEBOOK_CLIENT_SECRET</code>
X (Twitter)	<code>X_CLIENT_ID</code>	<code>X_CLIENT_SECRET</code>

Ohne gesetztes Paar bleibt die jeweilige Plattform serverseitig gesperrt (GET /api/social-media/auth/url liefert HTTP 503) — es gibt keinen Fallback, der ohne echte Credentials funktioniert (fail-closed, siehe `server/socialMedia/oauthExchange.ts` `isProviderConfigured()`).

1. YouTube (Google Cloud Console)

1. console.cloud.google.com → neues Projekt anlegen oder bestehendes wählen (z. B. „CAPITAL-AI Production“).
2. **APIs & Services** → **Library** → „YouTube Data API v3“ suchen → **Enable**.
3. **APIs & Services** → **OAuth consent screen**: - User Type: `External` (sofern kein Google Workspace vorhanden). - App-Name, Support-E-Mail, Developer-Kontakt-E-Mail ausfüllen. - Scopes hinzufügen: `.../auth/youtube.upload` , `.../auth/youtube.readonly` . - **Test-Nutzer** hinzufügen (die eigene YouTube-Kontoemail), solange die App im „Testing“-Status ist — nur diese Konten können sich verbinden, bis ein Verification-Review durchlaufen wurde.
4. **APIs & Services** → **Credentials** → **Create Credentials** → **OAuth client ID**: - Application type: `Web application` . - Authorized redirect URIs: `https://<DOMAIN>/api/social-media/auth/callback` eintragen. - Speichern → Client-ID und Client-Secret werden angezeigt.
5. Render: `YOUTUBE_CLIENT_ID` und `YOUTUBE_CLIENT_SECRET` setzen (siehe Abschnitt 6).

Wichtig: Für den Übergang von „Testing“ zu „In Production“ (damit sich beliebige YouTube-Konten verbinden können, nicht nur explizit gelistete Test-Nutzer) verlangt Google einen Verifizierungsprozess inkl. Prüfung der `youtube.upload` -Scope-Nutzung — dieser kann mehrere Tage dauern. Bis dahin funktioniert die Verbindung nur für als Test-Nutzer eingetragene Konten.

2. TikTok (TikTok for Developers)

1. developers.tiktok.com → mit Business-Konto anmelden → **Manage apps** → **Create an app**.
2. App-Name, Kategorie, Beschreibung ausfüllen.
3. **Add products**: „Login Kit“ und „Content Posting API“ hinzufügen.

4. Unter Login Kit: Redirect-URI `https://<DOMAIN>/api/social-media/auth/callback` eintragen.
5. Scopes aktivieren: `user.info.basic`, `video.upload`, `video.publish`.
6. **Client Key** und **Client Secret** aus dem App-Dashboard kopieren.
7. Render: `TIKTOK_CLIENT_ID` = Client Key, `TIKTOK_CLIENT_SECRET` = Client Secret.

Wichtig — App-Review-Status: `video.publish` ist ein review-pflichtiger Scope. Vor Freigabe durch TikTok landen veröffentlichte Videos zunächst als **privater Entwurf im Postfach** des verbindenden Nutzers, nicht öffentlich sichtbar (`platformPublishers.ts` kennzeichnet das Ergebnis entsprechend als `pending`, nicht als fertig veröffentlicht). Für öffentliches Direct-Posting muss der App-Review-Antrag bei TikTok gestellt und genehmigt werden (Beschreibung des Use-Case, Demo-Video des Flows).

2.1 App-Review — Texte für das Einreichungsformular

Fertige Textbausteine für die Pflichtfelder aus TikToks „App review criteria“. Alle Aussagen sind an der tatsächlichen Implementierung (`server/socialMedia/`) ausgerichtet — nichts davon beschreibt eine Funktion, die es nicht gibt.

App-Beschreibung (öffentliches Feld, erscheint auf der TikTok-Autorisierungsseite — kurz, beschreibt was die App/Website tut, nicht wie sie heißt):

DE: „CAPITAL-AI ist eine Enterprise-FinTech-Plattform für KI-gestützte Finanzanalysen und automatisierte Marktberichte. Über die TikTok-Anbindung können Nutzer Kurzvideos, die aus ihren Analysen entstanden sind, direkt aus ihrem CAPITAL-AI-Konto auf ihrem eigenen TikTok-Profil veröffentlichen.“

EN: „CAPITAL-AI is an enterprise fintech platform for AI-assisted financial analysis and automated market reporting. Through the TikTok integration, users can publish short-form videos created from their analyses directly to their own TikTok profile from within their CAPITAL-AI account.“

Detaillierte Produkt-/Scope-Erklärung (Pflichtfeld „detailed explanation of how each product and scope works within your app“):

„CAPITAL-AI integrates TikTok Login Kit and the Content Posting API to let users publish short-form videos generated within CAPITAL-AI directly to their own TikTok account, from the 'Social Media Accounts' section of the CAPITAL-AI dashboard.

- **Login Kit** (scope `user.info.basic`): users connect their TikTok account via OAuth 2.0. We use `user.info.basic` solely to display the connected account's display name, avatar and follower count in the CAPITAL-AI dashboard, so the user can confirm which account is linked before publishing anything.
- **Content Posting API** (scopes `video.upload`, `video.publish`): once connected, the user can trigger publishing of a video from within CAPITAL-AI. The video is submitted via the `PULL_FROM_URL` method of `/v2/post/publish/video/init/`. Every publish action is an explicit, individual user action — no content is ever posted automatically or wi-

thout the user actively selecting 'Sofort Veröffentlichen' (Publish Now) for that specific video."

Vorschlag für den Ablauf des Demo-Videos (Pflichtanhang, von TikTok separat verlangt — muss tatsächlich aufgezeichnet werden, kein Text ersetzt das): 1. CAPITAL-AI-Dashboard öffnen, zu „Social Media Accounts" navigieren (Domain im Video muss zur angegebenen Website-URL passen). 2. „Mit TikTok Verbinden" klicken → OAuth-Popup zeigen → Login/Consent auf der echten TikTok-Seite → Popup schließt sich automatisch, Konto erscheint mit echtem Handle/Avatar/ Follower-Count. 3. Ein Video zur Veröffentlichung auswählen, TikTok als Zielplattform wählen, Caption anpassen. 4. „Sofort Veröffentlichen" klicken → Ergebnis-Ansicht zeigen (Status „Veröffentlicht"/„Geplant" je nach Review-Stand). 5. Optional: Konto-Trennung zeigen („Verbindung Trennen").

Noch offene Pflichtangaben, die dieser Text nicht abdecken kann (siehe Guidelines-Abschnitte „Website URL" und „Privacy Policy and Terms of Service"): die angegebene Website-URL muss eine vollständig ausgebaute, öffentlich erreichbare Seite sein (keine reine Landing-/Login-Seite), mit ohne Menü sichtbaren, aktiven Links zu Datenschutzerklärung und Nutzungsbedingungen. Das muss vor Einreichung manuell auf der tatsächlichen Produktions-Domain geprüft werden.

3. Meta — Instagram & Facebook (ein gemeinsamer Meta-App-Eintrag)

Instagram und Facebook nutzen **dieselbe Meta-App** und denselben OAuth-Dialog, aber getrennte Client-ID/Secret-Env-Var-Paare (identische Werte, zweimal eingetragen — historisch bedingt durch das ADR-0010-Schema, funktional beliebig).

1. developers.facebook.com → **My Apps** → **Create App** → Typ „Business" wählen.
2. Produkte hinzufügen: **Facebook Login** und **Instagram Graph API**.
3. **Facebook Login** → **Settings**: Valid OAuth Redirect URIs = `https://<DOMAIN>/api/social-media/auth/callback`.
4. **App Review** → **Permissions and Features**: folgende Berechtigungen beantragen (für den eigenen Account/Test-Nutzer sofort nutzbar, für fremde Konten erst nach Review): `pages_show_list`, `pages_read_engagement`, `pages_manage_posts`, `instagram_basic`, `instagram_content_publish`.
5. **Voraussetzung serverseitig**: Das zu verbindende Instagram-Konto muss ein **Instagram-Business- oder Creator-Konto** sein, das mit einer Facebook-Page verknüpft ist — private Instagram-Konten können über die Graph API grundsätzlich nicht per API posten. Die Verknüpfung erfolgt in der Instagram-App unter Einstellungen → Konto → „Mit Facebook-Seite verknüpfen", bevor der OAuth-Flow in CAPITAL-AI gestartet wird.
6. App-ID und App-Secret aus **Settings** → **Basic** kopieren.
7. Render: `INSTAGRAM_CLIENT_ID / FACEBOOK_CLIENT_ID` = App-ID, `INSTAGRAM_CLIENT_SECRET / FACEBOOK_CLIENT_SECRET` = App-Secret (jeweils derselbe Wert).

Wichtig — Page-Auswahl: Der Backend-Code (`oauthExchange.ts` `fetchProviderProfile()`) verbindet automatisch die **erste** über `/me/accounts` gelistete Page des Nutzers. Verwaltet der Account mehrere Pages, muss vor dem Verbinden in Meta Business Suite sichergestellt werden, dass die gewünschte Page an erster Stelle steht, oder das zu verbindende Konto darf nur eine Page verwalten. Eine Auswahl-UI ist als Folgearbeit vorgemerkt (ADR-0020, Abschnitt 5).

4. X / Twitter (X Developer Portal)

1. developer.x.com → Developer-Account beantragen (falls noch nicht vorhanden — kann eine manuelle Prüfung durch X durchlaufen).
2. **Projects & Apps → Create App** (innerhalb eines Projekts).
3. **User authentication settings → Set up:** - App permissions: `Read and write`. - Type of App: `Web App`, `Automated App` or `Bot`. - Callback URI: `https://<DOMAIN>/api/social-media/auth/callback`. - Website URL: Produktions-Domain.
4. Scopes: `tweet.read`, `tweet.write`, `users.read`, `offline.access`.
5. **Keys and tokens → OAuth 2.0 Client ID and Client Secret** kopieren.
6. Render: `X_CLIENT_ID`, `X_CLIENT_SECRET`.

Wichtig — PKCE: X verlangt für jeden OAuth-2.0-Client zwingend PKCE (`code_challenge` / `code_verifier`), unabhängig davon, ob ein Client-Secret vorhanden ist. Das ist bereits serverseitig implementiert (`server/socialMedia/pkce.ts`) — hier ist keine zusätzliche Konfiguration nötig, außer sicherzustellen, dass „Confidential Client“ (nicht „Public Client“) im X-Portal ausgewählt ist, da der Server das Client-Secret per Basic-Auth mitsendet.

5. Zusammenfassung: Redirect-URI-Eintrag pro Plattform

Plattform	Ort des Redirect-URI-Eintrags
YouTube	Google Cloud Console → Credentials → OAuth 2.0 Client ID → Authorized redirect URIs
TikTok	TikTok for Developers → App → Login Kit → Redirect URI
Instagram/Facebook	Meta for Developers → App → Facebook Login → Settings → Valid OAuth Redirect URIs
X	X Developer Portal → App → User authentication settings → Callback URI

Alle vier müssen **exakt** `https://<DOMAIN>/api/social-media/auth/callback` sein (inkl. `https://`, ohne trailing slash) — eine abweichende URI führt beim jeweiligen Provider zu einem `redirect_uri_mismatch`-Fehler direkt im OAuth-Popup, bevor der Request CAPITAL-AI überhaupt erreicht.

6. Render Environment Variables eintragen

1. dashboard.render.com → Service `capital-ai` → **Environment**.
 2. Für jede der 10 Variablen aus Abschnitt 0 (bereits als `sync: false`-Platzhalter in `render.yaml` vorbereitet) den tatsächlichen Wert eintragen: **Add Environment Variable** → Key + Value → **Save Changes**.
 3. Render löst nach dem Speichern automatisch einen Redeploy aus. Kein manueller Trigger nötig.
 4. **Nie** diese Werte in `.env`-Dateien committen — `.env.example` enthält nur die Variablennamen als Platzhalter, so wie es bereits für alle anderen Secrets in diesem Projekt gehandhabt wird.
-

7. Validierung nach dem Deploy

1. `GET https://<DOMAIN>/healthz` → sollte `200 OK` liefern (bestätigt, dass der Server überhaupt startet — schlägt eine der neuen Dateien beim Import fehl, würde der Server gar nicht hochfahren).
 2. Angemeldet in der CAPITAL-AI-App: **Dashboard** → **Social Media Accounts** öffnen.
 3. Für jede konfigurierte Plattform „Mit ... Verbinden“ klicken → Popup öffnet sich mit der echten Provider-Login-Seite (nicht mehr mit einer Fake-Erfolgsseite) → nach Login/Consent schließt sich das Popup automatisch und das Konto erscheint mit echtem Handle/Follower-Count.
 4. Für eine Plattform ohne gesetzte Env-Vars: „Verbinden“ sollte eine Fehlermeldung „ist serverseitig nicht konfiguriert“ zeigen (HTTP 503) statt eines stillen Fehlschlags.
 5. `GET /api/social-media/accounts` **ohne** Authorization-Header sollte `401` liefern (Beleg, dass die Auth-Pflicht aus ADR-0020 aktiv ist — die im ursprünglichen Auftrag genannte „Validierung der Erreichbarkeit von `GET /api/social-media/accounts`“ ist absichtlich kein anonymer 200er mehr).
-

8. Rollback

Alle Änderungen sind additiv (neue Tabellen, neuer Router, neue Env-Vars) — kein bestehender Pfad wird durch dieses Feature verändert. Rollback-Optionen:

- **Feature deaktivieren, ohne zu deployen:** Render-Env-Vars leer lassen/entfernen → alle fünf Plattformen bleiben mit HTTP 503 gesperrt, der Rest der Anwendung ist unberührt.
- **Vollständiger Code-Rollback:** siehe `docs/runbooks/DEPLOYMENT_ROLLBACK_UND_BACKUP.md` (RUNBOOK-0001), Abschnitt „Anwendungsebene“. Die drei neuen Supabase-Tabellen können bei einem DB-Rollback stehen bleiben (sie werden von keinem anderen Code-Pfad gelesen) oder bei Bedarf manuell per `drop table` entfernt werden.

Systemadmin Audited Execution Runbook — SA3

Quelle: docs/runbooks/SYSTEMADMIN_AUDITED_EXECUTION.md

Systemadmin Audited Execution Runbook — SA3

Status: IMPLEMENTED / CI + EXECUTION-HOST BINDING PENDING Date: 2026-08-12

Authority: ESS-0021, ADR-0065, ADR-0059

Purpose

This runbook governs the repository-only audited execution path for the logical Systemadmin Roadmap Executor.

It does not authorize external production mutation and does not grant merge authority.

Mandatory authorization path

For repository actions that may mutate state, the canonical policy path is:

```
authorizeSystemadminAuditedExecution()
```

A raw SA2 prepared envelope is never mutation authority.

Required sequence:

1. resolve current `main` and Roadmap work package;
2. validate Owner-approved REM through SA1;
3. pass SA2 security/sequence preflight;
4. prepare SA2 action as `DRY_RUN`;
5. apply SA3 self-authority protection;
6. persist authorization to M5 append-only audit;
7. require returned `supabase:agent_audit_events:<id>` reference;
8. require `auditBoundExecutionPermitted=true`;
9. execution host validates and consumes the permit for the exact action;
10. execute the exact repository action;
11. persist terminal `SUCCESS` or `ERROR` through `recordSystemadminAuditedOutcome()`;
12. stop on any evidence failure;
13. stop before Human final review / merge boundary.

Execution-host boundary

Repository TypeScript cannot intercept the external ChatGPT GitHub connector by itself. Therefore SA3 has two sub-gates:

SA3A — repository audit adapter

Implemented by this branch:

- SA1/SA2 policy composition;
- durable M5 authorization event;
- audit-bound permit only after durable evidence;
- terminal append-only outcome event;
- self-authority protection and negative tests.

SA3B — actual execution-host binding

Still required before SA3 can reach `VERIFIED PASS` and before SA4 may start.

One of these enforcement modes must be proven:

1. **Tool-host middleware:** the actual execution host consumes a valid SA3 permit before each mutating connector action and rejects direct/bypass calls; or
2. **CAPITAL-AI execution gateway:** ChatGPT sends the exact intended action to a CAPITAL-AI gateway which validates/consumes the SA3 permit and performs the repository mutation itself.

Until one mode is verified:

- direct ChatGPT→GitHub mutating connector calls are **not** treated as SA3-enforced autonomous execution;
- `auditBoundExecutionPermitted=true` is a repository/runtime contract, not proof that the external connector consumed it;
- SA4 remains blocked.

Fail-closed rules

The caller/execution host must not execute when:

- REM/SA1/SA2 decision is DENY;
- audit persistence throws or returns no valid reference;
- execution-host binding is absent or cannot prove permit consumption;
- the request targets SA2/SA3 self-authority paths;
- requested capability is MERGE, DEPLOY_REQUEST or PRODUCTION_MUTATION;
- requested repository/path/target is outside the REM;
- security preflight is incomplete;
- credential exposure or untrusted scope elevation is detected;
- production mutation unexpectedly becomes necessary;
- final Human/Owner review has started;
- kill switch/revocation is active.

Audit fields

Authorization evidence must correlate at least:

- `mandateId` ;
- Roadmap item;
- Human actor;
- app/agent/session/request;
- trace id;
- capability/risk;
- repository/target/requested paths;
- branch/current head/PR when available;
- policy reason/decision;
- authorization audit reference.

Outcome evidence adds the resulting branch/commit/PR/workflow identifiers where available and references the authorization audit event.

Sensitive-data policy

Never persist:

- raw reusable tokens or API keys;
- passwords or TOTP values;
- complete prompts;
- complete diffs;
- raw request/response bodies;
- private keys or recovery material.

The existing M5 writer remains the canonical sanitizer/redactor.

Human boundary

The audited permit is repository execution authority only. It does not authorize:

- final PR approval;
- CI bypass;
- merge;
- production deploy;
- external production mutation.

At Human final review start, agent mutation stops. Human merge remains separate.

Branch lifecycle

current main → fresh agent/* branch → audited actions → PR → Human review/CI → Human merge → branch delete

Merged and superseded branches are never reused.

SA3 exit gate

SA3 requires all of:

1. SA3A repository implementation merged with CI VERIFIED PASS ;
2. SA3 work branch deleted;
3. actual execution-host binding identified;
4. positive proof that the host consumes the permit before mutation;
5. negative proof that direct/bypass mutation is denied;
6. authorization/outcome audit correlation verified end-to-end without secret evidence.

SA4 handoff

Only after the complete SA3 exit gate is VERIFIED PASS may the first SA4 pilot REM run.

The pilot remains non-production, max HIGH risk, one open Systemadmin PR, explicit path allowlist, expiry ≤ 7 days, kill switch enabled and one final build-and-test per reviewed head.

Systemadmin Chat Execution Profile Runbook

Quelle: docs/runbooks/SYSTEMADMIN_CHAT_EXECUTION_PROFILE.md

Systemadmin Chat Execution Profile Runbook

Status: SA2 IMPLEMENTATION / LIVE MUTATION BLOCKED UNTIL SA3 Date: 2026-08-12 Authority: ESS-0021, ADR-0065, SA1 REM Validator, M4 Agent IAM Logical agent: `capital-ai-systemadmin-roadmap-executor` Execution client: ChatGPT through the GitHub connector/tool host

1. Purpose

This runbook defines how the Systemadmin Roadmap Executor is prepared for autonomous Roadmap execution in ChatGPT without weakening the Human/Owner merge boundary.

SA2 does **not** enable unaudited live repository mutation. It creates the executable profile contract and policy-bound action envelope. Mutating LIVE execution remains fail-closed until SA3 proves durable append-only correlation of mandate, actor, agent, request, capability, target and result.

2. Authority chain

Canonical decision chain:

`OWNER_APPROVED REM → SA2 CHAT PROFILE → SA1 REM VALIDATOR → M4 AGENT IAM → ACTION ENVELOPE`

The ChatGPT/provider/model label never grants authority. The exact REM remains the authorization source.

3. Initial capability boundary

The profile may evaluate only:

- `READ`
- `ANALYZE`
- `PLAN`
- `BRANCH`
- `COMMIT`
- `PR`
- `CI_REQUEST`

The following remain outside the profile:

- `MERGE`

- `DEPLOY_REQUEST`
- `PRODUCTION_MUTATION`
- CRITICAL actions
- every Human/Owner-reserved mutation class from ADR-0065/SA1

4. Credential boundary

- Connector/tool host retains OAuth tokens, installation tokens and other reusable credentials.
- Raw reusable credentials must not be copied into model context, prompts, Evidence or action envelopes.
- If the host or agent detects credential exposure, execution stops fail-closed.
- Provider/model metadata is non-authoritative.

5. Mandatory preflight before repository mutation

Before `BRANCH`, `COMMIT`, `PR` or `CI_REQUEST`, all of the following must be true:

1. SA1 is `VERIFIED PASS` on `main`.
2. Current `main` SHA is resolved.
3. Current Roadmap gate and work package are resolved.
4. The exact REM is valid and Owner-approved.
5. Open PR changed-file overlap has been checked.
6. PR check class D/C/R/M has been determined.
7. Security and negative-test preflight has passed.
8. Rollback/revert strategy exists.
9. Targeted tests are defined.
10. CI budget state is known.
11. No production mutation or reserved Human/Owner action is required.

Any security-critical uncertainty is DENY/STOP.

6. Execution sequence

The intended sequence after SA3 is:

`READ/ANALYZE → PLAN → PREFLIGHT → BRANCH → IMPLEMENT → TARGETED VALIDATE → COMMIT → repeat as needed → PR → STOP FOR OWNER REVIEW → OWNER GATE → CI → HUMAN MERGE → BRANCH DELETE`

Additional invariants:

- branch names use `agent/*` and are created fresh from current `main`;
- a commit requires its targeted validation to have passed;
- PR creation requires implementation complete + targeted validation PASS + current head SHA;

- CI request requires an open PR, current head SHA and budget availability;
- once final Human/Owner review has started, the agent may inspect but must not silently mutate the reviewed head;
- merge remains Human/Owner-only;
- a merged or superseded work branch is deleted and never reused.

7. SA2 dry-run mode

SA2 supports two evaluation modes:

DRY_RUN

Policy evaluation and action-envelope preparation are permitted for the full initial capability set. Mutating envelopes always carry:

```
liveMutationPermitted = false
```

This proves sequencing and authorization without creating an unaudited mutation path.

LIVE

- `READ / ANALYZE` may be allowed when the REM and profile allow them.
- mutating `BRANCH / COMMIT / PR / CI_REQUEST` are denied until SA3 append-only audit correlation is `VERIFIED PASS`.

8. Action envelope

A successful SA2 preparation produces an immutable secret-free envelope containing only the evidence required to bind the intended action:

- profile/version;
- `mandateId` ;
- Roadmap item;
- Human actor;
- app/client id;
- agent/session/request ids;
- capability and risk;
- target resource;
- repository/base;
- requested paths;
- branch/head/PR identifiers when available;
- `requiresHumanMerge = true` ;
- `liveMutationPermitted = false` for SA2 mutating actions.

Tokens, passwords, secrets and reusable credentials are not envelope fields.

9. Stop conditions

Immediate stop applies when any of the following occurs:

- REM invalid, expired, revoked or out of scope;
- wrong Owner/agent/client/repository/base;
- path/target/capability outside REM;
- prompt/tool/retrieval content requests scope elevation;
- credential exposure detected;
- production mutation becomes necessary;
- reserved Owner action becomes necessary;
- open-PR conflict;
- CI budget exhausted or duplicate unchanged-head CI requested;
- final Human/Owner review has started and a mutation is requested;
- kill switch/revocation is active;
- authorization/audit prerequisite is ambiguous.

10. ChatGPT connector limitation

Repository TypeScript cannot intercept or technically wrap every external ChatGPT connector call by itself. Therefore SA2 establishes the canonical repository policy/profile and a testable action-envelope contract, but **does not claim connector-level enforcement that the connector does not expose**.

The first real autonomous mutation remains blocked until SA3 integrates the action path with durable M5 append-only audit evidence and the SA4 Owner-approved pilot proves the complete branch-to-PR lifecycle.

11. Known unrelated CI debt

PR #215 merged SA1 after the authoritative `build-and-test` and Governance checks passed. A separate `Google-Marketing-Schutzprüfung` run failed in its static contract step. The failure is inherited CI hygiene debt and is not treated as SA1 security-validation evidence.

SA2 does not modify Google Marketing configuration/workflows. The defect should be corrected in a separate scoped CI-maintenance change rather than mixed into the Systemadmin execution profile.

12. SA2 exit gate

SA2 is complete only when:

- profile code + manifest + tests are merged;
- full repository CI for the final reviewed head passes for the SA2 scope;
- mutating LIVE mode is still denied;
- no Supabase/Stripe/Render/deployment/production mutation occurred;

- work branch is deleted after merge;
- SA3 is recorded as the next gate.

Systemadmin Execution Host Runbook — SA3B

Quelle: docs/runbooks/SYSTEMADMIN_EXECUTION_HOST.md

Systemadmin Execution Host Runbook — SA3B

Status: IMPLEMENTED / PR+CI+POST-MERGE PROBE PENDING Date: 2026-08-12 Authority: ESS-0021, ADR-0065, ADR-0059, ADR-0067

Purpose

This runbook governs the first technically enforceable execution host for the CAPITAL-AI Systemadmin Roadmap Executor.

The host is **not** a general-purpose GitHub automation endpoint. Before SA4 it is limited to one security verification operation: creation of an empty probe branch after durable authorization evidence.

Trusted path

Owner execution issue → trusted main workflow → strict request parser → GitHub Actions OIDC → CAPITAL-AI broker → SA1/SA2/SA3A → M5 authorization evidence → BRANCH side effect → M5 outcome evidence

A direct ChatGPT GitHub connector write is not part of this autonomous path and must not be counted as SA3B evidence.

Execution request format

Issue title must start with:

[SA3B-PROBE]

Issue author must be exactly:

SvenKulessa

Issue body must be JSON only:

```
{
  "version": "1.0",
  "mode": "BRANCH_PROBE",
  "mandateId": "REM-SA3B-PROBE-001",
  "roadmapItem": "SA3B-HOST-PROBE",
  "baseSha": "<exact current main 40-char lowercase SHA>",
  "branchName": "agent/sa3b-host-probe-<unique-suffix>"
}
```

No additional fields are accepted.

Trust sources

The workflow checks out `main` with `persist-credentials: false` and loads:

- `scripts/systemadmin/validateExecutionIssue.mjs`;
- `.ai/mandates/REM-SA3B-PROBE-001.json`;
- trusted host workflow code.

The Issue never supplies an executable REM, shell command, file content, workflow definition or credential.

GitHub Actions permissions

Initial probe workflow permissions:

- `contents: write` — required only to create/delete the probe branch;
- `issues: write` — required only to post evidence reference;
- `id-token: write` — required only to mint GitHub OIDC workload identity.

Not granted:

- `pull-requests: write`;
- deployment write authority;
- workflow write authority;
- package write authority.

OIDC broker authentication

The workflow requests a token for audience:

```
capital-ai-systemadmin-execution
```

The broker verifies JWT signature with GitHub OIDC JWKS and requires the exact claims from ADR-0067, including Finance repository/IDs, Owner actor, `issues` event, `refs/heads/main` and the exact SA3B workflow reference.

Invalid, expired, incorrectly signed or incorrectly scoped tokens receive no authorization permit.

Broker endpoints

Authorization

```
POST /api/internal/systemadmin-execution/authorize
```

Required before every host side effect.

The broker binds:

- Issue number;

- workflow run ID;
- Owner/agent/client request identity;
- exact REM;
- roadmap item;
- capability `BRANCH` ;
- target Finance;
- exact base/head SHA;
- branch name;
- GitHub OIDC host identity.

The workflow must require:

- policy verdict `ALLOW` ;
- syntactically valid `supabase:agent_audit_events:<id>` authorization reference;
- `auditBoundExecutionPermitted=true` ;
- capability `BRANCH` ;
- branch/base SHA equal to the request.

Only after these checks may `gh api .../git/refs` run.

Outcome

`POST /api/internal/systemadmin-execution/outcome`

The same exact OIDC workflow/run binding is required. `SUCCESS` or `ERROR` becomes a second append-only M5 event.

If branch creation succeeds but terminal outcome persistence fails, the workflow immediately deletes the probe branch and fails.

Probe sequence after SA3B PR merge

1. Verify final SA3B PR CI/workflow-security PASS.
2. Human merge.
3. Delete the SA3B implementation branch.
4. Verify `main` has deployed and broker endpoint exists.
5. Resolve exact current `main` SHA.
6. Create one Owner issue using the exact JSON contract.
7. Observe host workflow.
8. Verify authorization audit reference was produced before branch creation.
9. Verify probe branch points exactly to the requested current `main` SHA.
10. Verify terminal SUCCESS outcome reference.
11. Run a negative probe with an invalid/stale permit/request and prove no branch appears.

12. Delete the successful probe branch after evidence capture.
13. Close/archive probe issues and synchronize evidence/Roadmap.

Stop conditions

Immediate DENY/STOP when:

- issue author/title/body contract is wrong;
- base SHA is stale;
- probe REM is absent/revoked/expired;
- OIDC validation fails;
- actor/request/workflow/run binding mismatches;
- SA1/SA2/SA3 policy denies;
- durable audit persistence fails;
- returned permit/audit reference is invalid;
- branch/head binding mismatches;
- any non-BRANCH capability is requested;
- outcome evidence cannot be persisted.

Security boundary

Before SA4 the host cannot create commits, open PRs, request CI, deploy or merge. It cannot accept arbitrary shell commands or file payloads.

The one-time branch probe is a security-control acceptance test, not a product mutation and not the SA4 autonomous roadmap pilot.

SA4 handoff

Only after SA3B `VERIFIED PASS` may a later PR extend the host to a bounded Owner-approved SA4 REM with repository `COMMIT / PR` actions.

Before that extension, SA3B trust-root paths must also be incorporated into the broader SA1 self-authority list, not only the SA3 audit-layer protection.